



A Blockchain and GAT with Temporal Autoencoder based Mitigation Scheme for Mobility Pattern and Trajectory Poisoning Attacks in Software Defined Vehicular Networks

An undergraduate project progress report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

**Degree of the Bachelor of the Science of Engineering
Honours**

by

H.M.S. Muthukumari	-	EG/2021/4685
G.W.N. Niluminda	-	EG/2021/4691
I.J. Niroshan	-	EG/2021/4695
C.Y. Samarasinghe	-	EG/2021/4775

.....
Dr. Tharindu Gamage
(Supervisor)

.....
Dr. P.A.D. S. Nilmantha Wijesekara
(Co-Supervisor)

Abstract

Mobility pattern and trajectory poisoning attacks corrupt the spatio-temporal learning models of SDVN controllers by injecting kinematically plausible yet collectively falsified beacon sequences over time, inducing safety-critical misrouting, phantom congestion, and suppression of collision warnings without triggering conventional detection mechanisms. Existing defenses are fundamentally inadequate against this threat class: threshold-based mobility validators calibrate detection parameters under static observation assumptions that collapse at high vehicle speeds; cryptographic authentication schemes verify identity but provide no guarantee of data truthfulness, leaving authenticated insiders free to poison learning models; individual vehicle analysis misses coordinated multi-source attacks whose spatial inconsistency is only detectable relationally; temporal learning approaches rely on generic network traffic features rather than mobility-specific sequential dynamics; and no reviewed framework eliminates the centralized trust bottleneck at the RSU or controller that a single compromise can silently subvert. To address this, we propose MPTD-PQS, a dual-mode detection and mitigation framework that operates in a lightweight mode via rule-based attack signature detection and HMAC beacon integrity enforcement at the RSU, and in a full mode via a Graph Attention Network (GAT) for spatial anomaly detection over dynamic vehicle-RSU graphs, an LSTM temporal autoencoder for sequential trajectory anomaly scoring, and a post-quantum Threshold Ring Signature (TRS) scheme requiring t -of- n RSU collaborative signing to structurally prevent minority-compromised RSU aggregates from reaching the controller. each RSU encrypts its local aggregate under post-quantum Fully Homomorphic Encryption (FHE) via OpenFHE before TRS ring coordination, preventing compromised ring members from observing neighbouring RSU aggregate contents, with the cloud performing blind homomorphic computations across combined encrypted aggregates without decryption. A permissioned blockchain with smart contracts (SC-Trust, SC-Revoke) provides Byzantine fault-tolerant trust management and real-time revocation across both modes, with IPFS integration bridging real-time and offline detection. Evaluated via NS-3 and SUMO under trajectory poisoning, Sybil-based mobility pattern poisoning, MitM, and control-plane compromise scenarios.

Contents

Acronyms	xiii
1 Introduction	1
1.1 Overview of SDVN	1
1.2 Significance and Impact of Mobility Pattern and Trajectory Poisoning Attacks	1
1.3 Problem Introduction	2
1.4 Limitations of Existing Approaches	2
1.5 Problem Statement	3
1.6 Objectives and Scope	3
1.6.1 Research Questions	4
1.7 Contributions	5
1.8 Novelty Statement	7
1.9 Implications of the Study	8
1.10 Organization of the Report	9
2 Literature Review	10
2.1 Background	10
2.2 Previous Work	11
2.2.1 SDVN Architecture and Security Foundations	11
2.2.2 Security Threats in SDVN	12
2.2.3 Network Topology Defense	12
2.2.4 Authentication and Intrusion Detection	13
2.3 Related Work	14
2.3.1 Mobility Pattern Analysis	14
2.3.2 Detection and Mitigation Techniques in Related Domains	16
2.3.2.1 Graph Attention Networks for Intrusion Detection	16
2.3.2.2 Data-Centric Machine Learning for Vehicular Misbehavior Detection	17
2.3.2.3 Blockchain-Based Trust and Distributed Consensus in Vehicular Networks	17
2.3.2.4 Threshold Ring Signatures and Post-Quantum Cryptography for VANET	17
2.4 Gaps in Literature	18

2.4.1	Literature Gap Analysis	18
2.4.1.1	The Missing Link: Mobility Data Content Verification	18
2.4.1.2	Centralized Trust Creates Single Points of Failure	18
2.4.1.3	Individual Analysis Misses Coordinated Attacks	19
2.4.1.4	No Temporal Deep Learning for Sequential Patterns	19
2.4.1.5	Reactive Detection Allows Damage before Response	20
2.4.1.6	No Integration - Security Mechanisms Work in Isolation	20
2.4.1.7	Detection Thresholds Calibrated Without Mobility Constraints	21
2.4.1.8	Cryptographic Protections Restricted to Classical Hardness Assumptions	21
2.4.1.9	Absence of Deployment Feasibility Evaluation Across Heterogeneous RSU Infrastructure . . .	21
2.4.2	Literature Comparison	22
2.5	Summary	22
3	Methodology	24
3.1	Research Design	24
3.2	Threat Model	24
3.3	SDVN System Modeling	27
3.4	Attack Modeling Based on Defined Scenarios	27
3.4.1	Trajectory Poisoning Attacks	27
3.4.2	Mobility Pattern Poisoning Attacks	31
3.4.3	Mobility-Induced Threat Amplification in SDVN	36
3.4.3.1	RSU Contact Duration and Beacon Sampling Window	36
3.4.3.2	Handover-Induced Learning Discontinuity . . .	37
3.4.3.3	Mobility-Disguised Poisoning Drift Rate	37
3.4.3.4	Sybil Identity Capacity Under Mobility Constraints	38
3.4.4	Attack Signature Characterization	39
3.4.4.1	Trajectory Poisoning Signatures	40
3.4.4.2	Mobility Pattern Poisoning Signatures	43
3.4.4.3	Composite Rule-Based Anomaly Score	44
3.5	Proposed Mitigation Framework	45
3.5.1	Cryptographic Mitigation Scope Analysis	45
3.5.2	Group Key Management via Logical Key Hierarchy . . .	47
3.5.2.1	LKH Tree Structure	47
3.5.2.2	Vehicle Session Key Establishment	48
3.5.2.3	Key Update on Vehicle Leave or Revocation . .	48

3.5.2.4	RSU Ring Key Generation via Distributed Key Generation (DKG)	49
3.5.3	Dual-Mode Detection and Mitigation Pipeline	49
3.5.3.1	GAT Architecture Design for Vehicle-RSU Mobility Graphs	57
3.5.3.2	Temporal Autoencoder Design for Mobility Sequences	60
3.5.3.3	Lightweight Mode: HMAC Beacon Integrity and Rule-Based Detection	61
3.5.3.4	Full Mode: GAT-Based Spatial Anomaly Detection	65
3.5.3.5	Full Mode: Temporal Autoencoder Sequential Anomaly Detection	66
3.5.3.6	Combined Detection Score and Fusion	67
3.5.4	Full Mode Mitigation: Post-Quantum TRS and FHE Aggregate Protection	70
3.5.4.1	RSU Aggregate Construction and Pre-Coordination FHE Encryption	73
3.5.4.2	Post-Quantum Threshold Ring Signature on Combined Encrypted Aggregate	74
3.5.4.3	Cloud-Side Blind Homomorphic Computation and Result Decryption	75
3.5.4.4	Threshold FHE Decryption at RSU Cluster	75
3.5.5	Blockchain Trust Layer with Smart Contract Design	77
3.5.5.1	Registration Smart Contract (SC-Register)	78
3.5.5.2	Trust Score Smart Contract (SC-Trust)	78
3.5.5.3	Revocation Smart Contract (SC-Revoke)	81
3.5.5.4	Real-Time vs. Offline Detection Mode	85
4	Experimental Setup	88
4.1	Baseline Selection and Benchmarking Rationale	88
4.1.1	External Baselines	88
4.1.2	Ablation Study Design	88
4.2	Primary Evaluation Metrics	93
4.2.1	Detection Performance Metrics	94
4.2.2	Mitigation Impact Metrics	95
4.2.3	Attack Impact Metrics	96
4.2.4	System Overhead and Scalability Metrics	98
4.3	Simulation Settings and Hyper-parameter Configuration	100
4.3.1	Network and Physical Layer	100
4.3.2	Lightweight Detection Thresholds	102
4.3.3	Attack Model Parameters	104
4.3.4	Parameter Selection Rationale and Empirical Calibration	104
4.3.5	Full-Mode Model Selection (GAT and LSTM-AE)	108

4.3.5.1	GAT Architecture Selection	108
4.3.5.2	LSTM-AE Architecture Selection	109
4.3.5.3	Selected Full-Mode Configuration	110
4.3.6	Full-Mode Sensitivity Analysis	110
4.3.6.1	Sensitivity of GAT Hyperparameters Inherited from the Source Architecture	112
4.3.7	Run Control and Reproducibility	114
4.3.8	State-of-the-Art Adaptation	114
4.3.9	Mobility Scenarios	114
4.3.10	Simulation Environment	115
4.3.11	Components Planned for Full-Mode Evaluation	116
4.3.12	Full-Mode Pipeline Settings	119
4.4	Experimental Scenarios	124
4.4.1	Experiment 1: Attack Penetration Rate $\times$ Attack Intensity	124
4.4.2	Experiment 2: Vehicle Speed Regime	125
4.4.3	Experiment 3: Network Scalability	126
4.4.4	Experiment 4: Combined Attack Threat Level	126
4.4.5	Experiment 5: Per-Attack Variant Comparison Under Default Settings	127
5	Results	129
5.1	Per-Attack-Variant Comparison (Experiment 5)	129
5.2	RSU-Side State Contamination Fix	133
5.3	Ablation Study Results	133
5.3.1	AB1 — Rule-Based Signatures Removed	133
5.3.2	AB2 — HMAC Removed	135
5.3.3	AB3 — GAT Disabled	136
5.3.4	AB4 — LSTM-AE Disabled	138
5.3.5	AB5 — Full AI Layer Removed (Speed Regime)	141
5.3.6	AB6 — TRS Threshold Signing Gate Removed	144
5.3.7	AB7 — FHE Pre-Coordination Encryption Removed	146
5.3.8	AB9 — Multi-Controller Architecture Removed	148
5.3.9	AB10 — Blockchain Trust Layer Removed	150
5.3.10	AB11 — LKH Replaced with Unicast Key Distribution	153
6	Timeline and Resource Required	155
6.1	Timeline	155
6.2	Resource Required	156
6.2.1	Software Resources	156
6.2.2	Hardware Resources	156
6.2.3	Estimated Cost	156
7	Conclusion	157

List of Figures

3.1	Malicious RSU based data-plane trajectory poisoning	28
3.2	Malicious vehicle based data-plane trajectory poisoning	29
3.3	Control-plane trajectory poisoning	30
3.4	Sybil based attack via compromised RSU	31
3.5	Sybil based attack via malicious vehicle impersonation	32
3.6	Data plane MitM based mobility pattern poisoning	33
3.7	Control plane mobility pattern poisoning	35
3.8	Mobility-Induced Threat Amplification.	39
3.9	Dual-mode system architecture of the proposed MPTD-PQS frame- work.	51
3.10	UML sequence diagram for lightweight real-time detection across one beacon epoch.	56
3.11	UML sequence diagram for the full operating mode across one de- tection epoch. Five lifelines	71
3.12	Detection flow diagrams for TP-DETECT and SYB-DETECT. . . .	86
3.13	Detection flow diagrams for MITM-DETECT and CP-DETECT. . .	87
4.1	Simulation environment across the three mobility scenarios.	116
5.1	a1 — trajectory poisoning via malicious vehicle: MPTD-PQS MCC vs. attack penetration, against B1–B3.	129
5.2	a2 — trajectory poisoning via compromised RSU: MPTD-PQS MCC vs. attack penetration, against B1–B3.	130
5.3	a3 — Sybil mobility pattern poisoning via compromised RSU: MPTD- PQS MCC vs. attack penetration, against B1–B3.	130
5.4	a4 — Sybil mobility pattern poisoning via vehicle impersonation: MPTD-PQS MCC vs. attack penetration, against B1–B3.	131
5.5	a5 — trajectory poisoning via malicious controller (stealthy control- plane; B1–B3 have no control-plane defense and are blind to this variant): MPTD-PQS MCC vs. attack penetration, against B1–B3. .	131
5.6	a6 — MitM mobility pattern poisoning: MPTD-PQS MCC vs. at- tack penetration, against B1–B3.	132
5.7	a7 — control-plane mobility pattern poisoning (stealthy control- plane; B1–B3 have no control-plane defense and are blind to this variant): MPTD-PQS MCC vs. attack penetration, against B1–B3. .	132

5.8	AB1 ablation: rule-based signatures removed — (a) MCC vs. attacker penetration ρ_a .	133
5.9	AB1 ablation: rule-based signatures removed — (b) TTD vs. attacker penetration ρ_a .	134
5.10	AB1 ablation: rule-based signatures removed — (c) PBPO vs. attacker penetration ρ_a .	134
5.11	AB2 ablation: HMAC removed — (a) MCC vs. MitM penetration ρ_{MitM} .	135
5.12	AB2 ablation: HMAC removed — (b) TTD vs. MitM penetration ρ_{MitM} .	136
5.13	AB2 ablation: HMAC removed — (c) CDER vs. MitM penetration ρ_{MitM} .	136
5.14	AB3 ablation: GAT disabled (preliminary, $n = 3$ seeds) — (a) MCC vs. coordinated Sybil identities n_{coord} .	137
5.15	AB3 ablation: GAT disabled (preliminary, $n = 3$ seeds) — (b) TTD vs. coordinated Sybil identities n_{coord} .	137
5.16	AB3 ablation: GAT disabled (preliminary, $n = 3$ seeds) — (c) CDER vs. coordinated Sybil identities n_{coord} .	138
5.17	AB4 ablation: LSTM-AE removed — (a) MCC vs. stealth drift bound ϵ_{max} .	139
5.18	AB4 ablation: LSTM-AE removed — (b) TTD vs. stealth drift bound ϵ_{max} .	139
5.19	AB4 ablation: LSTM-AE removed — (c) CDER vs. stealth drift bound ϵ_{max} .	140
5.20	AB4 ablation: LSTM-AE removed — (d) TPE vs. stealth drift bound ϵ_{max} .	140
5.21	AB5 ablation: full AI layer removed — (a) MCC vs. vehicle speed.	141
5.22	AB5 ablation: full AI layer removed — (b) CDER vs. vehicle speed.	141
5.23	AB5 ablation: full AI layer removed — (c) TTD vs. vehicle speed.	142
5.24	AB5 ablation: full AI layer removed — (d) TDEE vs. vehicle speed.	142
5.25	AB5 ablation: full AI layer removed — (e) TPE vs. vehicle speed.	143
5.26	AB5 ablation: full AI layer removed — (f) PBPO vs. vehicle speed.	143
5.27	AB6 ablation: TRS threshold gate weakened to single-signer — (a) PARR vs. compromised ring-member fraction f/n .	144
5.28	AB6 ablation: TRS threshold gate weakened to single-signer — (b) CDER vs. compromised ring-member fraction f/n .	145
5.29	AB6 ablation: TRS threshold gate weakened to single-signer — (c) COO vs. compromised ring-member fraction f/n .	145
5.30	AB7 ablation: post-quantum vs. classical cryptographic overhead — (a) per-epoch crypto cost ($\text{COO}_{\text{epoch}}$) vs. signing-ring size n .	146
5.31	AB7 ablation: post-quantum vs. classical cryptographic overhead — (b) TRS signing latency (Δt_{TRS}), PQ vs. ECDSA-class, vs. signing-ring size n .	147

5.32	AB7 ablation: post-quantum vs. classical cryptographic overhead — (c) byte-overhead ratio ($\text{BWO}_{\text{ratio}}$) vs. signing-ring size n	147
5.33	AB7 ablation: post-quantum vs. classical cryptographic overhead — (d) one-time DKG + keygen setup latency vs. signing-ring size n	148
5.34	AB9 ablation: multi-controller architecture removed — (a) CDER vs. controller compromise onset $t_{\text{comp}}/T_{\text{sim}}$	149
5.35	AB9 ablation: multi-controller architecture removed — (b) MCC vs. controller compromise onset $t_{\text{comp}}/T_{\text{sim}}$	149
5.36	AB10 ablation: blockchain trust layer removed — (a) CDER vs. attack penetration ρ_a	150
5.37	AB10 ablation: blockchain trust layer removed — (b) $\text{FRR}_{\text{revoke}}$ vs. attack penetration ρ_a	151
5.38	AB10 ablation: blockchain trust layer removed — (c) $\text{FRR}_{\text{demote}}$ vs. attack penetration ρ_a	151
5.39	AB10 ablation: blockchain trust layer removed — (d) $\text{TCL}_{\text{confirm}}$ vs. attack penetration ρ_a	152
5.40	AB10 ablation: blockchain trust layer removed — (e) MCC vs. attack penetration ρ_a	153
5.41	AB11 ablation: LKH replaced with unicast key distribution — (a) rekey message count vs. vehicles registered per RSU zone $ \mathcal{V}_j $	153
5.42	AB11 ablation: LKH replaced with unicast key distribution — (b) rekey burst channel occupancy vs. vehicles registered per RSU zone $ \mathcal{V}_j $	154
6.1	Project Timeline	155

List of Tables

2.1	Comparative Summary of Related Works Against the Proposed Framework	22
3.1	Canonical mapping between attack scenarios and their primary rule-based detection signature.	40
3.2	Detection vs. Mitigation Placement of Framework Components . . .	47
3.3	A summary of the notations.	53
4.1	Ablation study design: eleven independent component isolation experiments. Metric abbreviations follow Section 4.2.	89
4.2	Metric-to-RQ mapping and evaluation scope.	94
4.3	Network and physical-layer settings.	101
4.4	Vehicle fleet composition.	102
4.5	Lightweight detection thresholds.	103
4.6	Attack-model parameters.	104
4.7	Grid search for the composite alert threshold ψ_{th} (Eq. 3.21). Selected value in bold.	106
4.8	Grid search for the MP-S3 KL-divergence threshold κ_{th} (Eq. 3.19), in nats. Selected value in bold.	106
4.9	Grid search for the position-drift threshold δ_{th} (Eq. 3.16). Selected value in bold.	106
4.10	Grid search for the scenario-specific kinematic speed bound s_{max} (Eqs. 3.11, 3.20), in km/h. Selected value in bold.	107
4.11	Grid search for the drift observation window and the Sybil density factor. Selected values in bold.	107
4.12	Full-mode GAT model selection by validation MCC (Eq. 4.1), mean $\pm$ std over ≥ 3 seeds. Selected configuration per scenario in bold.	109
4.13	Full-mode LSTM-AE model selection by validation reconstruction error (Eq. 3.46) under the FPR constraint, mean $\pm$ std over ≥ 3 seeds. Selected configuration per scenario in bold; values to be finalised on real data.	110
4.14	Selected per-scenario full-mode configuration carried into evaluation. GAT values finalised on real data; LSTM-AE values to be finalised.	110
4.15	Grid search for the LSTM-AE threshold sensitivity factor κ ($\theta_{\text{ae}} = \mu_{\varepsilon} + \kappa\sigma_{\varepsilon}$, Eq. 3.47). Selected value in bold.	111

4.16	Grid search for the fusion decision threshold Φ_{th} (Eq. 3.49). Selected value in bold.	111
4.17	Sensitivity of the FHE ring dimension N on cryptographic latency (Eqs. 3.52, 3.58). Selected value in bold; security level from CKKS parameter tables [1].	111
4.18	Analytical rekey cost $N_{\text{ring}} = n-1$ (Eq. 3.56) for the TRS ring size n ($n \geq 3f+1$, $f = 1$). The deployed value $n = 4$ is in bold; wall-clock COO_{trs} is flat across n and reported separately (Table 4.17).	112
4.19	Sensitivity of full-system detection performance to attack penetration rate ρ_a , at the default configuration (Table 4.30), urban scenario, 3-seed average.	112
4.20	Sensitivity of the GAT dropout rate (urban scenario). Selected value in bold.	113
4.21	Sensitivity of the GAT optimiser learning rate. Selected value in bold.	113
4.22	Sensitivity of the GAT weight decay. Selected value in bold.	113
4.23	Sensitivity of the LeakyReLU negative slope used in GAT attention computation (Eq. 3.29). Selected value in bold.	113
4.24	Run-control and reproducibility settings.	114
4.25	Per-scenario mobility and channel settings. Detection thresholds and attack-model parameters are identical across all scenarios.	115
4.26	Full-mode AI detection settings.	118
4.27	Full-mode training, calibration, and fusion pipeline settings (shared by GAT and LSTM-AE).	119
4.28	Post-quantum cryptographic pipeline settings.	122
4.29	Blockchain trust and revocation layer settings.	123
4.30	Default configuration held fixed across all experiments unless stated otherwise.	124
4.31	Composite Threat Level parameter mapping (Eq. 4.18). Total attacker fraction $\rho_a = 0.40$ is held fixed across all levels so that TL isolates attack sophistication, not attack quantity.	127

Acronyms

AI	Artificial Intelligence
BFT	Byzantine Fault Tolerance
BSM	Basic Safety Message
FHE	Fully Homomorphic Encryption
GAT	Graph Attention Network
HMAC	Hash-based Message Authentication Code
IPFS	InterPlanetary File System
ITS	Intelligent Transportation System
LSTM	Long Short-Term Memory
MitM	Man-in-the-Middle
NS-3	Network Simulator 3
PKI	Public Key Infrastructure
PQC	Post-Quantum Cryptography
RSU	Roadside Unit
SDN	Software-Defined Networking
SDVN	Software-Defined Vehicular Network
SUMO	Simulation of Urban Mobility
TRS	Threshold Ring Signature
V2I	Vehicle-to-Infrastructure
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
VANET	Vehicular Ad Hoc Network

Chapter 1

Introduction

1.1 Overview of SDVN

Vehicular Ad Hoc Networks (Vehicular Ad Hoc Network (VANET)) were originally designed to support direct vehicle to vehicle and vehicle to infrastructure communication in a fully distributed manner. However, the absence of centralized control in VANET makes global traffic management, scalability, and security enforcement difficult in highly dynamic environments [2].

Software Defined Networking (SDN) addresses these challenges by separating the control plane from the data plane and enabling centralized and programmable network control. By applying SDN principles to vehicular environments, Software-Defined Vehicular Networks (SDVN) were introduced to provide global network visibility, flexible control, and intelligent decision making capabilities [3].

In the context of SDVN, a centralized controller manages the network by harnessing a holistic and global understanding of vehicles, RSUs, and network communications. The system design makes it easy to manage traffic routing, congestion control, handover, and network optimization. To ensure intelligent decision-making, the SDVN heavily depends on learning approaches to evaluate vehicle mobility and path information. Data like vehicle position, speed, direction, and path history is constantly harvested and used by the controller to understand vehicle mobility and predict the future state of the network. These predictions help in improving the management of the network in highly dynamic environments [4].

1.2 Significance and Impact of Mobility Pattern and Trajectory Poisoning Attacks

Mobility pattern poisoning and trajectory poisoning attacks are serious threats to SDVN because they directly target the learning process of the controller. In these attacks, attackers inject false but realistic mobility data over time, causing the learning models to gradually produce incorrect results. Since the data follows

normal movement behavior, such attacks are difficult to detect using traditional security methods.

If these attacks succeed, the controller may make wrong traffic predictions, inefficient routing decisions, or unstable handover choices. This can lead to increased congestion, reduced network performance, and unreliable communication. In safety related applications, incorrect decisions may even result in dangerous situations. Therefore, detecting and mitigating mobility and trajectory poisoning attacks is essential for maintaining secure and reliable SDVN operation.

1.3 Problem Introduction

SDVN depend on centralized, learning based models to manage vehicle mobility, traffic conditions, and safety related decisions. These models continuously learn from mobility and trajectory data reported by vehicles and RSU to predict future network states. However, the correctness of these decisions depends on the integrity and reliability of the collected mobility data.

Recent studies shows that attackers can exploit this dependency by injecting falsified but realistic mobility information over time, gradually influencing the learning process without triggering conventional security alarms. Such attacks may originate from different entities within the SDVN architecture, including vehicles, roadside units, or even compromised controllers and can affect both data plane observations and control plane decisions.

1.4 Limitations of Existing Approaches

Although several solutions have been proposed for SDVN security, existing approaches exhibit ten critical limitations. First, most focus on traditional network attacks and do not address poisoning of learning models through falsified vehicle mobility and trajectory data [5, 6]. Second, cryptographic authentication schemes verify identity but not data truthfulness, leaving authenticated insiders free to inject false trajectories [5]. Third, most solutions assume a trustworthy SDVN controller, which is unrealistic when the controller itself may be compromised [7]. Fourth, existing detection thresholds are calibrated under static observation assumptions that collapse at high vehicle speeds, where the speed-dependent RSU contact duration τ_c (Eq. (3.2)) shrinks the observable beacon sampling window below the minimum needed to accumulate sufficient detection evidence [8]. Fifth, individual and pairwise vehicle analysis cannot detect coordinated multi-source attacks whose spatial inconsistency is only visible relationally [8]. Sixth, no reviewed work applies deep sequential learning to trajectory time-series, leaving gradual poisoning across beacon sequences undetected [9]. Seventh, all reviewed defenses are reactive, detecting attacks only after malicious data has already influenced control decisions [9, 10]. Eighth, every cryptographic scheme in the ve-

hicular security literature relies on classical hardness assumptions with no post-quantum resilience evaluation [11, 12]. Ninth, no framework analyses detection feasibility under resource-heterogeneous RSU deployments where legacy units cannot run deep learning inference — a gap this work identifies and motivates the deployment-feasible lightweight mode against, though a dedicated heterogeneous-RSU deployment evaluation remains outside the current experimental scope.

1.5 Problem Statement

In SDVN-enabled intelligent transportation systems, the high mobility of vehicles creates dynamically constrained RSU contact windows that adversaries exploit to inject mobility-consistent yet collectively falsified beacon sequences at rates that evade per-beacon plausibility checks, progressively corrupting the controller’s spatio-temporal learning models to induce safety-critical misrouting, phantom congestion, and collision risk escalation, a threat class that existing authentication-centric and generic intrusion detection approaches cannot detect or cryptographically mitigate, particularly when attacks originate from compromised controllers or coordinated multi-source adversaries.

1.6 Objectives and Scope

The main objectives of this project are:

- **Attack Analysis:** To study and model mobility pattern and trajectory poisoning attacks in SDVN, including attacks at both the data plane and control plane by malicious vehicles, compromised RSUs, and malicious controllers.
- **Mitigation Method Design:** To design a mitigation approach that combines blockchain and learning based techniques to protect vehicular mobility data, by analyzing spatial relationships using Graph Attention Networks and detecting abnormal mobility behavior over time using an LSTM temporal autoencoder.
- **Performance Evaluation:** To define an evaluation strategy for the effectiveness of the proposed mitigation framework under different poisoning attack scenarios using controlled NS-3 and SUMO simulation experiments across urban, rural, and highway mobility regimes.

The scope of this project is limited to the modeling and mitigation of mobility pattern and trajectory poisoning attacks in learning based SDVN. The evaluation of the proposed framework is planned to be conducted using simulation tools to analyze security performance and system behavior under controlled attack scenarios.

Real world deployment, hardware implementation, large scale field testing, and network attacks unrelated to mobility data poisoning are outside the scope of this project.

1.6.1 Research Questions

This work addresses the following research questions, each mapping directly to a contribution, its evaluation experiment, and its ablation variants.

RQ1. How do vehicle mobility parameters — specifically RSU contact duration, handover rate, and traffic density — quantitatively alter the detection feasibility window and the adversarial drift budget available to trajectory and mobility pattern poisoning attackers in SDVN, and do existing threshold-based defenses remain effective under high-mobility conditions? (Maps to: mobility amplification formulation; evaluated by detection feasibility condition Eq. (3.4) and drift budget Eq. (3.7) across speed regimes; primary empirical validation via Experiment 2 (Section 4.4.2, speed $v \in \{10, 60, 100, 140\}$ km/h) and Experiment 4 (Section 4.4.4, streak length component of TL); reported metrics: MCC (Eq. 4.1), TTD (Eq. 4.2), TPE (Eq. 4.8).)

RQ2. Does the proposed multi-source dual-plane threat model reveal attack vectors — specifically control-plane compromise by a malicious controller and coordinated multi-RSU Sybil injection — that single-source or data-plane-only threat models used in prior works fail to capture, and does the proposed framework maintain detection effectiveness across all four attacker classes simultaneously? (Maps to: threat model contribution; evaluated by per-attacker-class MCC (Eq. 4.1), TTD (Eq. 4.2), CDER (Eq. 4.4), TDEE (Eq. 4.7), and TPE (Eq. 4.8) across malicious vehicle, compromised RSU, MitM, and malicious controller scenarios; primary empirical validation via Experiment 5 (Section 4.4.5, per-attack variant table under default settings).)

RQ3. Can rule-based attack signature detection, operating exclusively at the RSU without deep learning inference, achieve practically sufficient detection performance against both trajectory and mobility pattern poisoning attacks, and where does it fail relative to the full mode? (Maps to: attack signature characterization and lightweight mode; evaluated by MCC (Eq. 4.1) and TTD (Eq. 4.2) of ablation variant AB5 (full AI layer removed, Section 4.1.2) across all attack variants and penetration rates; primary empirical validation via Experiment 1 (Section 4.4.1) and Experiment 2 (Section 4.4.2); PBPO (Eq. 4.9) confirms the 100 ms real-time constraint is satisfied by the lightweight-only path.)

- RQ4.** Does the integration of GAT spatial anomaly detection and LSTM temporal autoencoder scoring in the full operating mode significantly improve detection performance over the lightweight mode and over each component individually, across varying attacker types, penetration rates, and vehicle speed regimes? (Maps to: dual-mode detection framework; evaluated by MCC (Eq. 4.1) and TTD (Eq. 4.2) of ablation variants AB3 (GAT disabled), AB4 (LSTM-AE disabled), and AB5 (full AI layer removed) compared against the full mode across all scenarios (Section 4.1.2); primary system-level empirical validation via Experiment 1 (Section 4.4.1) across penetration rates and intensity levels, and Experiment 4 (Section 4.4.4) across threat levels.)
- RQ5.** Does the post-quantum TRS scheme structurally reduce the fraction of poisoned aggregates reaching the SDN controller independent of AI detection accuracy, and what are the associated computational and communication overheads at the RSU relative to classical signing? (Maps to: PQC-TRS and FHE mitigation pipeline; evaluated by PARR (Eq. 4.3) of ablation variant AB6 (TRS disabled) vs. full mode; RSU-side cryptographic overhead reported as COO (Eq. 4.10) compared against ECDSA baseline; communication overhead reported as BWO_{ratio} (Eq. 4.11) and BWO_{scale} (Eq. 4.12) against the LKH unicast baseline (ablation variant AB11); primary system-level scalability validation via Experiment 3 (Section 4.4.3, $N_v \in \{100, 200, 300, 400\}$)).
- RQ6.** Does the blockchain smart contract trust management layer — specifically the BFT-conditioned SC-Revoke contract — reduce the control decision error rate and sustained poisoning impact compared to frameworks relying on centralized controller-side revocation, and does it do so without incorrectly revoking or demoting honest entities or introducing unacceptable transaction latency for real-time ITS operation? (Maps to: blockchain smart contract contribution; evaluated by CDER (Eq. 4.4) and false revocation rate FRR_{revoke} (Eq. 4.5) and false demotion rate FRR_{demote} (Eq. 4.6) of ablation variant AB10 (blockchain removed) vs. full mode; on-chain transaction confirmation latency TCL_{confirm} (Eq. 4.13) and post-revocation reassignment latency TCL_{reassign} (Eq. 4.14) reported as framework-internal overhead measures; ablation variants AB8 (RSU lifecycle removed) and AB9 (multi-controller removed) isolate sub-components of the trust management layer; primary per-attack-class validation via Experiment 5 (Section 4.4.5).)

1.7 Contributions

1. **Multi-Source Dual-Plane Threat Model.** We propose a comprehensive threat model for mobility pattern and trajectory poisoning attacks in SDVN that considers four simultaneous attacker classes — malicious vehicles, compromised RSUs, MitM adversaries, and malicious or compromised controllers

— operating across both the data plane (beacon injection and interception) and the control plane (learning model corruption and control decision manipulation). Unlike prior works that assume a trusted controller [8, 9], confine attackers to the data plane [10], or treat poisoning as a single-source event, this threat model captures the full adversarial surface of learning-based SDVN and directly motivates the distributed trust and dual-plane mitigation design of the proposed framework.

2. **Mobility-Induced Threat Amplification Formulation.** We formally quantify how vehicle kinematics amplify poisoning attack effectiveness in SDVN by deriving closed-form expressions for speed-dependent RSU contact duration (Eq. (3.2)), beacon sampling window (Eq. (3.3)), handover-induced learning discontinuity (Eq. (3.5)), mobility-disguised drift rate (Eqs. (3.6)–(3.7)), and Sybil identity capacity (Eqs. (3.8)–(3.9)). This is the first formal characterization of mobility as an attack amplification vector in SDVN, showing that high-speed scenarios simultaneously reduce detection evidence and enlarge the adversarial action space.
3. **Attack Signature Characterization for Principled Detection.** We derive nine formally defined attack signatures (TP-S1 through MP-S4) that characterize the observable fingerprints of trajectory poisoning and mobility pattern poisoning in the BSM beacon stream (Eqs. (3.11)–(3.20)), forming a composite rule-based anomaly score (Eq. (3.21)). Unlike existing threshold-based methods [8] that apply isolated checks, our signature set is derived from the full kinematic feasibility space and includes cross-identity coordination tests absent from all reviewed works.
4. **Dual-Mode Detection Framework.** We design a lightweight operating mode executing entirely at the RSU using HMAC beacon integrity verification and rule-based signature detection (Algorithm 1), and a full operating mode executing GAT-based spatial anomaly detection over dynamic vehicle-RSU graphs (Eqs. (3.27)–(3.30)) and LSTM temporal autoencoder sequential anomaly scoring (Eqs. (3.44)–(3.47)) at the SDVN controller (Algorithm 5). Detection scores from both components are fused into a single decision variable (Eq. (3.48)). No prior SDVN security framework provides both a deployment-feasible lightweight mode and a high-assurance full mode evaluated comparatively.
5. **Post-Quantum FHE-then-TRS Mitigation Pipeline.** We design a post-quantum cryptographic mitigation pipeline in which each RSU first encrypts its local aggregate under post-quantum FHE (BFV/BGV/CKKS via OpenFHE, Eq. (3.51)) before any ring coordination, preventing compromised ring members from observing neighbouring RSU aggregate contents during coordination a privacy threat unique to multi-RSU threshold signing. Ring members then perform homomorphic addition across the received

ciphertexts (Eq. (3.52)), and a post-quantum Threshold Ring Signature (TRS) scheme requiring $t_{\text{sign}} = f+1$ collaborative RSU signing (Eqs. (3.54)–(3.56)) is applied to the combined encrypted aggregate, so that no plaintext crosses any RSU boundary at any point and any minority-compromised RSU subset (of size $< t_{\text{sign}}$) is structurally prevented from injecting a valid poisoned aggregate into the controller, independent of AI detection. The cloud performs further blind homomorphic computations across combined encrypted aggregates without decryption (Eq. (3.58)), following feasibility demonstrated in [13]. This is the first framework to identify and address cross-RSU aggregate leakage during threshold ring signing coordination using pre-coordination FHE in SDVN.

6. **Smart Contract-Based Byzantine Fault-Tolerant Trust Management.** We design two smart contracts SC-Trust for continuous trust score management across all three entity types (vehicles: Eq. (3.63); RSUs: Eq. (3.66); controllers: Eq. (3.64)), and SC-Revoke for RSU-direct BFT-conditioned identity revocation (Eq. (3.69)) that excludes the controller as an evidence source, preventing a compromised controller from suppressing legitimate revocations or fabricating false ones on a permissioned consortium blockchain with IPFS off-chain beacon storage, bridging real-time per-beacon detection and offline controller-side deep learning analysis. SC-Trust enforces a three-state lifecycle on each RSU trusted, probationary, and client driven by a peer quorum disagreement signal (Eq. (3.65)) that prevents demoted RSUs from inflating the BFT evidence count and maintains the $|\mathcal{R}_{\text{trusted}}(t)| \geq 3f+1$ safety floor at all times. When a controller’s trust score falls below the revocation threshold, SC-Revoke updates the on-chain trusted-controller set $\mathcal{C}_{\text{trusted}}(t)$ (Eq. (3.1)) and RSUs automatically reassign to any available trusted controller without manual failover. Unlike prior blockchain applications in vehicular networks [14, 15] that use blockchain only for data storage, our smart contracts encode the full detection-to-revocation state machine for vehicles, RSUs, and controllers with unified BFT safety guarantees parameterised by a single security parameter f .

1.8 Novelty Statement

This work advances SDVN security on two levels. At the problem level, we are the first to formally characterize mobility pattern and trajectory poisoning as a mobility-amplified, multi-source, dual-plane threat, deriving closed-form bounds that show how vehicle speed, handover rate, and traffic density simultaneously shrink the detection evidence window and enlarge the adversarial drift budget, and proposing a threat model that—unlike all reviewed works—treats malicious vehicles, compromised RSUs, MitM adversaries, and compromised controllers as concurrent poisoning sources across both data and control planes.

At the solution level, we are the first to propose a unified framework combining four independently novel contributions. First, an attack-signature-derived dual-mode detection framework pairing lightweight rule-based signature scoring at the RSU with a full-mode multi-task GAT—trained on $K = 7$ parallel attack-type classification heads sharing a common spatial encoder—and an LSTM temporal autoencoder at the controller, where the predicted attack type selects attack-conditioned fusion weights and signature reliability weights at inference, giving each detector component the discriminative emphasis appropriate for the identified attack class. Second, a pre-coordination FHE-blinded post-quantum threshold ring signature pipeline in which RSUs encrypt their local aggregates under FHE before ring coordination begins, eliminating cross-RSU aggregate leakage during share exchange, with TRS signing applied to the homomorphically combined ciphertext so that minority-compromised RSU subsets are structurally blocked from assembling a valid threshold signature and their aggregates never reach the controller, independent of AI detection outcome. Third, a three-state RSU trust lifecycle driven by a quorum-disagreement misbehavior signal evaluated against the currently trusted RSU set only, preventing demoted RSUs from continuing to inflate the BFT evidence count while their cryptographic keys remain valid. Fourth, a permissioned blockchain with smart contracts encoding the complete detection-to-revocation state machine under RSU-direct Byzantine fault-tolerant consensus, where controller compromise is identified by a directional conflict indicator that fires when the controller suppresses anomaly alerts independently raised by RSUs—penalizing alarm suppression, not disagreement—and revocation is triggered by RSU peer votes submitted directly to the blockchain without controller relay, with automatic RSU reassignment to the on-chain trusted-controller set $C_{\text{trusted}}(t)$, collectively eliminating the centralized trust bottleneck including controller-level compromise that every reviewed defense retains.

1.9 Implications of the Study

The results of the research have many implications for research and practice related to software-defined vehicular networks (SDVN). This research clearly articulates the importance of considering the patterns of mobility and trajectory poisoning as multi-source and time-evolving attacks, which brings into focus the issue of security that is currently unattended by the available solutions for the security of SDVN.

In practice, this proposed framework provides design insights for designing resilient SDVNs that do not rely on a trusted controller. The combination of blockchain trust, spatial learning, and temporal anomaly detection concepts can be used for future intelligent transportation systems to improve safety, reliability, and security from insider threats and controller-level attacks.

From a research perspective, this paper opens new research avenues for studying the issue of security threats in vehicular networks based on learning, specifically

concerning dynamic attacks. The proposed approach for the problem definition and solution can be applied in general SDN-based networks that make control decisions based on the data obtained by learning.

1.10 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a detailed review of existing literature related to SDVN security, mobility data validation, and poisoning attacks, and identifies the research gaps addressed in this work. Chapter 3 describes the proposed methodology, including the system model, threat model, attack scenarios, and the proposed mitigation framework. Chapter 4 outlines the evaluation plan used to assess the effectiveness of the proposed approach. Finally, Chapter 5 concludes the report and discusses future research directions.

Chapter 2

Literature Review

Software Defined Vehicular Networks (SDVNs) are viewed as an upcoming solution to the complex problems associated with the conventional vehicular network, especially those pertaining to the dynamic nature of the network and the efficient use of network resources. Although the benefits of the network are many, it poses serious risks, especially concerning the integrity of the data associated with the network's mobility. This chapter reviews the current state of the art to determine the shortcomings of the techniques employed.

2.1 Background

Software-Defined Vehicular Networks is the extension of the traditional SDN concept to mobile networks by splitting the control and data planes, which allows the centralized management of the network by means of SDN Controllers communicating with the vehicle and Road Side Units through the OpenFlow protocol. The proposed defense system combines the strengths of three different techniques, namely the Graph Attention Networks, which learn the spatial relationships among the vehicle network by assigning importance weights to the neighboring nodes through the use of self-attention mechanisms, which facilitate the identification of the coordinated attacks, the Temporal Autoencoders, which encode the trajectory data into the latent space to detect the sequential attacks through the analysis of the reconstruction error, and the Blockchain technology, which provides the distributed consensus and the immutable ledger to overcome the single point of failure of the centralized architecture. This combination of techniques can overcome the inherent limitations of the present state of the art of SDVN security solutions, which are based on the centralized trust management, the analysis of the vehicle data without considering the spatial patterns, and the rule-based validation, which is incapable of identifying the complex sequential poisoning attacks.

2.2 Previous Work

2.2.1 SDVN Architecture and Security Foundations

The three-layered SDVN architecture comprising infrastructure plane (vehicles and RSUs), control plane (SDN controllers), and application plane (network services) was comprehensively documented by [16]. Their survey showed how OpenFlow enables treating vehicles and RSUs as programmable switches, providing centralized control that dramatically improves resource management and handles network heterogeneity across DSRC, cellular, and WiFi technologies. Despite these architectural advantages, the authors made a critical observation: “security is still the major fear that is preventing the SDVN from becoming widely acceptable” [16].

The security vulnerabilities they identified are particularly relevant to this research. Beyond the well-known DDoS attacks on controllers and flow table overflow issues, they documented the complete absence of trustworthiness evaluation mechanisms for participating vehicles. More critically, their survey revealed three gaps that directly motivate the current work:

No AI or machine learning-based security mechanisms had been proposed for SDVN security [16], Graph-based network analysis for attack detection was entirely missing [16], No specific defenses existed against trajectory poisoning attacks [16].

The trajectory-prediction approach introduced by [17] demonstrates both the potential and the security risks of mobility-aware SDVN management. Their system extended OpenFlow to include position, velocity, and heading data, allowing the Status Manager to predict future vehicle locations and the Topology Manager to pre-install flow rules proactively. In Network Simulator 3 integrated with SUMO traffic simulation, this approach achieved 10-15% better packet delivery by switching between OLSR (for dense, slow traffic) and GPSR (for sparse, fast-moving scenarios). However, their architecture diagram included a “security module” that was never implemented, evaluated, or even described. The entire prediction mechanism operates on a dangerous assumption: that all vehicles honestly report their positions and velocities. This scenario clearly provides an attack vector: malicious vehicles may provide erroneous trajectory information to impact network dynamics; but in reference [17], there are no methods for identifying or mitigating this tampering. In survey article [6], 164 research papers on intelligent SDVN were surveyed, presenting novel ideas such as Digital Twin Networks for real-time observation and prediction. While they have thoroughly described ML-based applications for routing improvement and QoS improvement, all applications considered gave priority to performance improvement over security. They have accepted “security gaps in SDVN due to centralized architecture” in both abstract and discussion sections but failed to provide any solutions for this issue. In this survey article as well, there were no descriptions regarding blockchain-

based implementations, graph-based neural networks for pattern observation, or time-based models for trajectory validation.

2.2.2 Security Threats in SDVN

The literature provides the most detailed taxonomy of threats for SDN-based VANETs, which covers 19 types of attacks that include the application, control, and data layers. This paper is very significant in that it points out the exact problem that this research addresses. The authors claimed that "false position information" and "illusion attacks," which spread fake position information, are challenging and crucial issues in SDN-based VANETs. The authors explained that some vehicles that move slowly send fake information about their positions in order to create virtual congestions that affect the routing path of other vehicles. The authors did not propose any solution for this problem but just analyzed it in their paper [5].

Their results reveal a critical flaw in current cryptographic protections. Public Key Infrastructure and authentication through certificates are able to ensure a message is sent from a genuine vehicle – namely, they are able to verify the message's sender is authentic. But they are not able to ensure the message is genuine. An authentic vehicle with genuine credentials can transmit spurious GPS location information, spurious speeds, and entirely fabricated tracks, and these will withstand any and all cryptographic verification. Authentication therefore verifies authenticity, not truthfulness.

The difference pointed out in [5] between network topology and mobility topology is very important. They discussed topology poisoning attacks and referenced solutions like TopoGuard, but these protect network connectivity (which devices connect to which switches), not geographic movement patterns. A vehicle could maintain correct network attachments while systematically falsifying its location, speed, and trajectory. The authors acknowledged this gap but left it completely unaddressed.

2.2.3 Network Topology Defense

Literature exposed critical vulnerabilities in SDN topology management and demonstrated attacks against all eight major controllers (Floodlight, OpenDaylight, Beacon, POX, NOX, Ryu, Maestro, OpenIRIS) [10]. Their Host Location Hijacking attack spoofed host identifiers to falsify switch port attachments, while their Link Fabrication attack created illusions of non-existent switch connections using forged LLDP packets. Both succeeded because topology update mechanisms lacked authentication and integrity protection [10].

TopoGuard, their defense system implemented in 700 lines of Java within Floodlight, provides three key protections:

1. **Host migration verification:** Requires Port Down signals before accept-

ing movement (precondition) and uses ICMP probes with 1-second timeout to confirm hosts disappeared from previous locations (post condition)

2. **LLDP authentication:** Applies HMAC-SHA256 signatures to protect switch-to-switch discovery
3. **Port property management:** Tracks whether each port connects to switches or hosts, preventing hosts from participating in LLDP propagation

Their evaluation showed effective attack prevention with only 0.005-0.431ms overhead per operation [10].

However, TopoGuard’s scope differs fundamentally from mobility attack defense. In their system, host “location” means network attachment point: which switch and which port. Their Host Profile data structure contains MAC address, IP address, VLAN, switch DPID, and port number. Notice what’s missing: no GPS coordinates, no velocity vectors, no heading information, no trajectory history. TopoGuard cannot detect an adversary who injects false geographic positions showing a vehicle kilometers from its actual location, fabricates velocities claiming impossible speeds, or constructs fake trajectory sequences. The distinction is categorical: TopoGuard addresses topology structure poisoning (network connectivity) but not data content poisoning (falsified mobility information) [10].

Moreover, TopoGuard concentrates all trust in the SDN controller. All verification, HMAC signature generation, state machine management, and ICMP probe execution happen within the controller using controller-held secret keys. If the controller is compromised through software vulnerabilities, insider access, or physical attacks, the entire security system fails because there’s no distributed verification or redundancy. [10] acknowledged that powerful adversaries who completely mute host-generated traffic could evade detection, and that physical-layer verification lies beyond their scope.

2.2.4 Authentication and Intrusion Detection

Raja et al.[9] developed a two-level security architecture for SDVN combining RSU-based Group Authentication using Elliptic Curve Cryptography with a private Collaborative Intrusion Detection System. Their RGA scheme reduces per-message overhead by providing group identity-key pairs to authenticated vehicles, enabling efficient V2V communication. The collaborative IDS uses Alternating Direction Method of Multipliers for distributed learning across RSUs, with differential privacy adding noise to protect vehicle information and Paillier homomorphic encryption enabling computation on encrypted data. They achieved 96.81% detection accuracy on the NSL-KDD dataset [9].

The critical limitation lies in what they’re detecting. The paper explicitly acknowledges that “some rogue vehicles after authentication pose as legitimate VANET users and send messages to interrupt network communication” [9]—exactly the insider threat scenario where authenticated vehicles inject false data.

But their solution uses generic network intrusion detection, not mobility-specific validation. The NSL-KDD dataset contains TCP SYN floods, port scans, buffer overflows—computer network intrusion signatures, not vehicular mobility patterns. An IDS trained to recognize packet-level attacks won’t recognize when a vehicle reports a position on a valid road segment, a velocity within normal ranges, and a plausible acceleration, yet these individually valid values form an impossible trajectory sequence.

Furthermore, their privacy mechanisms don’t address data integrity. Differential privacy adds noise to protect training data but doesn’t verify whether the underlying data is genuine or poisoned—it just makes the poisoning private. Homomorphic encryption enables secure computation but doesn’t validate data authenticity: a vehicle can encrypt false trajectories, and the encrypted computation processes that false data without detection. The detection operates reactively, identifying attacks after they manifest as patterns in network traffic, with no real-time evaluation of incoming mobility data before network use [9].

2.3 Related Work

2.3.1 Mobility Pattern Analysis

This approach pioneered using mobility patterns themselves for misbehavior detection. Their system constructs Location Time Tables from periodic beacons containing position and timestamp information, giving each vehicle distributed local knowledge of surrounding vehicles’ claimed locations. The detection framework has four modules working sequentially [8]:

1. **Security verification:** Authenticates message origins using group signatures [8].
2. **Location verification:** Checks four conditions: positions must fall within road map boundaries (from Google Maps or OpenStreetMap), lie within transmission range (1km WAVE limit), not overlap with other vehicles’ positions (preventing Sybil attacks), and satisfy speed-of-light constraints[8].
3. **Movement analysis:** Compares current versus previous views to detect abnormal area differences indicating suspicious position changes[8].
4. **Plausibility detection:** Validates speeds against physical constraints (0-240 km/h range, road-specific limits)[8].

With an 80-car simulation, it was possible to differentiate between real and fake Post-Crash Notifications by observing their behavioral patterns together. If the cars were receiving real alerts of an accident, their trajectory plots showed joint behavior such as changing lanes, slowing down, and reacting together. However,

fake alerts showed no change in behavioral patterns because drivers do not act on information that is not visible to them [8].

The authors themselves admitted the use of "a simple and coarse grain method in classifying good and malicious vehicles" [8]. The admission of this fact is quite surprising as it reveals some limitations of the approach in question. The detection system is completely dependent on thresholds, which could be easily bypassed by a sophisticated malicious user. A malicious user knowing about a threshold of 0-240 km/h in terms of speed could report a speed of 110 km/h, which is still valid but could be a false report. If a malicious user is aware of road-map verification, they could feed false location information on valid roads. By knowing about constraints related to the speed of light, a malicious user could guarantee consistency in distance and time relationships. In these scenarios, the system could verify every value independently without realizing that all values represent a malicious path in combination. The approach examines vehicles individually (location verification and plausibility verification) or compared to a previous view in a pairwise fashion; however, it does not use a graph relational modeling approach. If multiple malicious vehicles coordinate to inject mutually consistent false data, where each vehicle's claims support the others' fabrications, individual checks might validate each separately without detecting the collectively impossible spatial configuration. The paper explicitly states that "vehicles need to wait until they get enough evidence in its LTT table" [8], introducing delays dangerous for split-second safety decisions.

Additionally, the system relies on centralized RSU trust for road map distribution and clock synchronization. A compromised RSU could distribute falsified maps, causing legitimate vehicles reporting correct positions to be flagged as misbehaving when their genuine coordinates fall outside the false map boundaries. Report misbehavior from Vehicle to RSUs at journey's end, delayed and potentially lossy if communication fails or malicious RSUs selectively ignore reports.

Aside from security threats in terms of mobility patterns in SDN-VANETs, other security concerns have been considered by researchers. Arif et al. [5] have presented a comprehensive classification of security threats in SDN-VANETs from DoS to GPS spoofing in different planes of a network. While they have considered false location data as a major security threat, their survey did not include measures to counter such threats. Likewise, Hong et al. [10] have presented topology poisoning attacks in SDN controllers in which attackers create network links or misrepresent host location data. They have developed TopoGuard to counter such attacks through cryptographic validation but considered "location" as a network attachment point instead of geographic location to preserve mobility data integrity.

A different approach is taken by Raja et al. [9] for collaborative intrusion detection in the SDVN, obtaining excellent performance through distributed machine learning while maintaining privacy. Nonetheless, their IDS is based on common network traffic patterns, but not vehicle mobility behavior, and is reactive, but

not proactive. Taken as a whole, these papers demonstrate sound bases for secure SDVN but indicate the need for ensuring the integrity of the mobility information itself, as cars may be verified as authentic but broadcast erroneous paths.

Yamamoto et al. [18] proposed a velocity vector field combined with a variance tensor field to model and predict manned-vehicle trajectories in open environments, enabling collision-risk estimation and anomaly detection when observed trajectories deviate from predicted occupancy regions. While this work introduces valuable trajectory prediction primitives applicable to moving vehicles, it operates in an isolated sensing context with no network adversarial model, no multi-source attacker consideration, and no connection to SDN-controlled vehicular infrastructure. It cannot detect coordinated multi-vehicle poisoning, and the variance tensor formulation assumes honest sensor inputs, rendering it ineffective when the trajectory data itself is the attack surface.

Ercan et al. [19] proposed a machine-learning-based misbehavior detection scheme for position falsification attacks in VANETs, deriving features such as position change and speed deviation from exchanged beacons and training supervised classifiers on the VeReMi dataset. While effective at distinguishing position falsification variants, the approach classifies beacons on individual per-message features without spatial-relational modeling across vehicles, does not address coordinated multi-source poisoning, operates as offline classification rather than a real-time SDVN control-plane defense, and assumes a trusted detection authority, leaving the centralized trust bottleneck of SDVN unaddressed.

2.3.2 Detection and Mitigation Techniques in Related Domains

2.3.2.1 Graph Attention Networks for Intrusion Detection

Frenken et al. [20] proposed KD-GAT, representing Controller Area Network (CAN) bus traffic as sliding-window graphs and training a multi-layer Graph Attention Network with jumping-knowledge aggregation, then distilling it into a student model only 6.32% the teacher’s parameter count, achieving 99.97% detection accuracy on the Car-Hacking dataset at resource-constrained deployment scales. Their subsequent work [21] further introduces a multi-stage pipeline using a Variational Graph Autoencoder (VGAE) for unsupervised anomaly pre-filtering followed by a knowledge-distilled GAT classifier, improving F1-score by up to 55% on highly imbalanced CAN intrusion datasets. These works validate the power of GAT architectures for detecting coordinated anomalies in graph-structured communication systems but are limited to in-vehicle CAN networks and do not generalize to dynamic, geographic SDVN vehicle-RSU graphs where node membership changes continuously due to mobility.

2.3.2.2 Data-Centric Machine Learning for Vehicular Misbehavior Detection

Sharma and Liu [22] proposed a data-centric misbehavior detection model for the Internet of Vehicles that engineers plausibility and consistency features from basic safety messages and evaluates multiple supervised classifiers on the VeReMi dataset, achieving strong position-falsification detection without relying on vehicle hardware trust. While effective as a data-centric detector, the model classifies each message largely in isolation, captures neither temporal trajectory drift across beacon sequences nor spatial coordination among vehicles, and provides no distributed trust or mitigation pathway, remaining a reactive centralized detector.

2.3.2.3 Blockchain-Based Trust and Distributed Consensus in Vehicular Networks

Yahiatene and Rachedi [15] combined SDN and blockchain for vehicular social network security through a hierarchical controller architecture, using a Distributed Miners Connected Dominating Set algorithm to dynamically select trusted miners and eliminate the SDN controller as a single point of failure. Sehar et al. [14] proposed VBCA, a consortium blockchain consensus replacing Proof-of-Work with a lightweight protocol reducing transaction latency for V2V/V2I data integrity. These works demonstrate blockchain’s viability in vehicular networks but do not integrate smart contract-based anomaly scoring or address mobility data poisoning at the learning model level.

2.3.2.4 Threshold Ring Signatures and Post-Quantum Cryptography for VANET

Ahmed et al. [12] combined a threshold ring signature (TRS) scheme requiring t -of- n cooperative signing without revealing individual identities with a blockchain incentive trust model to validate traffic events in VANETs, demonstrating resistance to collusion attacks. Zhong et al. [23] proposed identity-based broadcast encryption for V2I communication enabling a single ciphertext to serve a vehicle group, significantly reducing per-message overhead. Faleh et al. [11] proposed CSAS-V, a pairing-free certificateless Schnorr aggregate signature scheme for scalable VANET authentication. While these cryptographic advances address message authenticity and group privacy, none provide post-quantum security guarantees. Mamun et al. [13] provided the first experimental evaluation of lattice-based homomorphic encryption schemes (BFV, BGV, CKKS via OpenFHE) for privacy-preserving Infrastructure-to-Infrastructure communication in ITS, demonstrating feasibility under 128-bit post-quantum security with RSU-Cloud link benchmarks. Their work directly motivates the use of post-quantum fully homomorphic encryption for secure aggregate transmission in the proposed framework, though it addresses only privacy and provides no attack detection capability.

2.4 Gaps in Literature

2.4.1 Literature Gap Analysis

2.4.1.1 The Missing Link: Mobility Data Content Verification

After reviewing all this previous work, one gap stands out above all others: no existing mechanism specifically verifies the correctness of mobility data content. Multiple researchers acknowledge the problem: [5] call it a “hard and critical issue,” [10] demonstrate topology poisoning, [9] acknowledge “rogue vehicles after authentication”, but none provide comprehensive operational solutions for adversaries who systematically inject false GPS coordinates, fabricate velocities, or construct fictitious trajectories.

The problem manifests in several attack scenarios that current defenses cannot handle.

Current cryptographic schemes prove identity but not data truthfulness. A vehicle’s certificate confirms authorization to participate but provides no validation that claimed positions correspond to actual locations, that reported velocities match actual speeds, or that trajectory sequences represent actual paths rather than strategic fabrications [5].

2.4.1.2 Centralized Trust Creates Single Points of Failure

Every system examined relies on centralized trust. SDN controllers centralize all topology management, flow rule installation, and routing decisions [16]. If compromised, adversaries can poison topology databases, install malicious flow rules, suppress security alerts, and falsify statistics without remaining validation mechanisms.

RSU-based architectures show parallel vulnerabilities. [8] and [9] both trust RSUs for certificate distribution, road map provisioning, and clock synchronization. Compromised RSUs could distribute falsified maps, selectively ignore misbehavior reports, or inject false timestamps. [9]’s collaborative IDS assumes honest majority, but becomes vulnerable when attackers compromise sufficient RSUs to poison training data or vote for accepting attack patterns.

Blockchain technology offers distributed consensus eliminating single trusted entities through Byzantine fault tolerance. Multiple surveys mention blockchain as “future work” [5, 6, 16], but none provide architectural designs showing how to integrate blockchain consensus into SDVN mobility validation, protocol specifications for vehicle/RSU/controller participation, or performance evaluations demonstrating whether blockchain latency can meet real-time requirements.

2.4.1.3 Individual Analysis Misses Coordinated Attacks

Current approaches analyze vehicles in isolation or through pairwise comparisons, completely missing the rich relational structure in vehicular networks. Vehicle movements are fundamentally interdependent: following vehicles can't overtake without lane changes, parallel-lane vehicles show correlated speed adjustments, intersection approaches are constrained by signals and right-of-way, highway platoons exhibit characteristic spacing and synchronized patterns [8, 17].

Graph Attention Networks (GATs) provide a natural architecture for modeling these dependencies. GATs can learn which spatial configurations represent normal traffic: vehicles in adjacent lanes traveling at similar speeds, safe following distances determined by speed and conditions, platoon coordination patterns.

Attention mechanisms let the network automatically focus on most relevant neighbors without manual specification. Different scenarios require different relationships: on highways, leading/following vehicles matter most; at intersections, crossing paths become critical; in parking areas, nearby stationary vehicles are relevant.

Despite this natural fit and GAT's proven success in social network analysis, molecular property prediction, and traffic forecasting, no SDVN research applies GATs to mobility pattern analysis [6, 16]. This is a missed opportunity: these architectural advances suit a fundamentally graph-structured problem. The absence prevents detecting coordinated attacks where multiple adversaries inject mutually consistent but collectively implausible data.

2.4.1.4 No Temporal Deep Learning for Sequential Patterns

Vehicle trajectories are inherently sequential time-series data where temporal dependencies carry crucial legitimacy information. A single position provides minimal validation context beyond boundary checks. But a sequence over 10-30 seconds reveals movement patterns distinguishing genuine behavior from fabrications [8, 17].

TopoGuard concentrates all verification in the controller using controller-held HMAC keys and controller-managed state machines [10], such that controller compromise defeats all security simultaneously. RSU-based architectures show parallel vulnerabilities where compromised RSUs can distribute falsified maps or poison collaborative IDS training data [9], [8].

These architectures have proven success in financial fraud detection (anomalous transaction patterns), healthcare (unusual patient trajectories), and cyber security (network traffic sequences). Yet their application to vehicular trajectory validation remains unexplored in SDVN security literature [6, 16].

Existing prediction work uses classical models: linear extrapolation, Kalman filtering, traffic flow models [17]. These assume honest input and optimize performance (reduce overhead, enable proactive rules), not security. They predict likely future positions to improve efficiency, not detect whether historical posi-

tions are genuine or falsified. [9] used logistic regression on packet features, not deep sequential models for trajectory time-series.

2.4.1.5 Reactive Detection Allows Damage before Response

Existing systems identify attacks only after malicious data has been injected, propagated, and potentially influenced decisions [8–10]. Mobility pattern analysis requires accumulating “sufficient evidence in its LTT table” over multiple beacon intervals [8]. Intrusion detection matches behavior to learned signatures after attacks manifest in traffic patterns. Topology poisoning detection validates updates after receiving them, with Hong et al.’s ICMP probing adding 1 second timeout.

Proactive defense requires different architectures. Rather than waiting for evidence accumulation, proactive systems would continuously score incoming data’s anomaly level in real-time, immediately isolating suspicious data before operational impact. Rather than detecting only after damage manifests, they would learn normal patterns comprehensively and predict likely future states, flagging subtle deviations preemptively. Rather than reacting to known signatures from past attacks, they would adaptively learn from evolving threats through online learning, continuously updating models and adjusting thresholds dynamically.

No surveyed SDVN research demonstrates such proactive prevention where mobility data is evaluated and potentially quarantined at reception before any operational use [5, 16].

2.4.1.6 No Integration - Security Mechanisms Work in Isolation

Perhaps the most critical gap is the complete absence of comprehensive frameworks combining multiple complementary defenses [6, 16]. Each work addresses one isolated aspect:

- Cryptographic authentication of identities [9]
- Network intrusion detection through collaborative learning [9]
- Mobility pattern threshold checking [8]
- Network topology validation via LLDP verification [10]

No research synthesizes multiple layers into holistic systems where individual weaknesses are compensated by complementary strengths. [16] explicitly stated that existing solutions “focus on detection without mitigation” and lack “comprehensive framework combining multiple defense layers.” [6] acknowledged needing to integrate “blockchain, AI, and advanced analytics” but provided no architectural designs, protocol specifications, or implementation guidance.

A truly comprehensive defense would require the integration of distributed trust (removing points of failure via blockchain consensus), spatial pattern analysis (graph attention networks identifying attacks via relational anomalies), sequence analysis (deep learning methods identifying poisoning attacks), and real-time proactive scoring (blocking dubious data use before validation is completed). This integrated synthesis has yet to be demonstrated in any current work.

2.4.1.7 Detection Thresholds Calibrated Without Mobility Constraints

All threshold-based and signature-based detection works reviewed, including [8], calibrate detection parameters (speed bounds, position deviation limits, beacon evidence accumulation windows) under implicit assumptions of sufficient observation time per vehicle. None of these works analyse how vehicle speed constrains RSU contact duration or how handover frequency fragments trajectory sequences across RSU boundaries. Consequently, thresholds derived in these works are systematically calibrated against a static or low-mobility assumption that does not reflect real SDVN operating conditions, leaving the detection logic untested and likely inadequate in high-speed vehicular scenarios.

2.4.1.8 Cryptographic Protections Restricted to Classical Hardness Assumptions

Every cryptographic scheme reviewed for vehicular network security [9, 11, 12, 23] relies exclusively on classical hardness assumptions, namely elliptic curve discrete logarithm or integer factorisation problems. The vehicular security literature has not evaluated whether these protections remain viable under quantum adversaries, nor has any SDVN mobility data protection mechanism been assessed for post-quantum resilience. This is a recognised open problem in the broader cryptographic community yet has received no attention in the SDVN mobility security sub-field.

2.4.1.9 Absence of Deployment Feasibility Evaluation Across Heterogeneous RSU Infrastructure

All reviewed frameworks, including [8–10, 15], are evaluated under a single uniform deployment assumption where all RSUs are presumed to have equivalent computational resources. None of the reviewed works analyse how their proposed detection or mitigation mechanisms perform under resource-heterogeneous conditions where some RSUs may be computationally constrained legacy units incapable of running deep learning inference or executing complex cryptographic operations in real time. This omission means that the practical field deployability of every reviewed solution remains entirely undemonstrated.

2.4.2 Literature Comparison

Table 2.1: Comparative Summary of Related Works Against the Proposed Framework

Reference	Mobility Aware Model	Distributed Trust	Graph Analysis	Temporal Learning	Lightweight Mode	PQC	Attack Scope	Key Limitation
Ghaleb et al. [8]	Partial	No	No	No	Yes	No	Position falsification, Sybil	Threshold bypass; individual analysis only
Hong et al. [10]	No	No	No	No	No	No	SDN topology poisoning	Network attachment only; not geographic mobility data
Raja et al. [9]	No	No	No	No	No	No	Generic network intrusion	NSL-KDD dataset; not mobility-specific
Yamamoto et al. [18]	Yes	No	No	Partial	No	No	Single-vehicle trajectory anomaly	No network adversary model; no SDVN context
Ercan et al. [19]	Partial	No	No	No	No	No	Position falsification	Per-beacon ML features; individual analysis; no distributed trust
Sharma & Liu [22]	Partial	No	No	No	No	No	Position falsification (IoV)	Message-level classification; no temporal/spatial or trust model
Frenken et al. [20]	No	No	Yes	No	Yes	No	In-vehicle CAN bus intrusion	In-vehicle only; not SDVN vehicle-RSU graph
Ahmed et al. [12]	No	Yes	No	No	No	No	Traffic event validation	Classical TRS; no trajectory poisoning detection
Mamun et al. [13]	No	No	No	No	No	Yes	I2I communication privacy	Privacy only; no attack detection capability
Proposed	Yes	Yes	Yes	Yes	Yes	Yes	Trajectory poisoning, Mobility pattern poisoning (multi-source, dual-plane)	—

As shown in Table 2.1, the literature comparison reveals the following critical gaps addressed by the proposed framework:

Lack of Distributed Trust: Most of the existing literature requires a centralized entity (for instance, RSU or honest majorities). The proposed system is the only one that tackles the single point of failure problem using blockchain consensus, thus providing a distributed trust system.

No Graph-Based Analysis: The existing literature does not represent the spatial relationship between vehicles as a dynamic graph. The proposed Graph Attention Network (GAT) module reduces the fabrication of coordinated mobility pattern data through learning pattern relationships in the network of vehicles.

Lack of Temporal Learning: Current solutions depend on static thresholds [8] or primitive classifiers [9], which are not capable of identifying complex pattern anomalies. The proposed method uses a temporal autoencoder to learn normal trajectory dynamics to identify gradual poisoning.

Lack of Proactive Detection: The major part of the literature reviewed is based on a reactive approach, detecting attacks only after malicious data affects the network. The framework provides real-time scoring of anomalies to remove malicious mobility data before it affects SDN control decisions.

Lack of Comprehensive Integration: The current state-of-the-art solutions tend to focus on individual parts such as authentication mechanisms, topology safeguarding, or traditional intrusion prevention. The proposed strategy will comprehensively integrate blockchain concepts for trust mechanisms, GAT for graph-based reasoning tasks, and temporal autoencoders for validation.

2.5 Summary

This chapter reviewed SDVN architecture, threats, and countermeasures to establish how existing research addresses mobility data integrity. The picture that emerges is consistent: although [5] explicitly calls these "hard and critical issues,"

current defenses against trajectory and mobility pattern poisoning remain inadequate, for six interrelated reasons. First, authentication verifies identity but not data validity, so an authenticated insider can still inject fake trajectories undetected. Second, most solutions assume a trustworthy controller and RSU, with no blockchain-based or otherwise distributed alternative addressing this single point of failure. Third, vehicle-level analysis misses coordinated attacks that only become visible through graph-based, relational methods. Fourth, no reviewed work applies deep sequential learning to trajectory time-series, leaving attacks undetected where each individual sample looks legitimate but the sequence as a whole is not. Fifth, detection across the field is reactive rather than proactive. Sixth, existing methods are developed independently rather than integrated into a single framework. No prior SDVN work combines blockchain-based distributed consensus, Graph Attention Networks for spatial analysis, and temporal autoencoders for sequential analysis to address trajectory poisoning; combining the three is the gap this thesis targets.

Chapter 3

Methodology

3.1 Research Design

This research follows a modeling based methodology to design and analyze a learning based mitigation framework for mobility pattern and trajectory poisoning attacks in SDVN. The overall methodology consists of four main stages:

- System modelling: SDVN architecture, entity definitions, and BSM beacon communication model
- Attack modelling: trajectory poisoning, mobility pattern poisoning, and control-plane attack formalisation
- Mitigation framework design: dual-mode detection pipeline with post-quantum cryptographic protection
- Evaluation planning: simulation setup, metrics, and baseline comparison

The proposed system is modeled as consisting of vehicles, Roadside Units (RSUs), and a distributed set of SDVN controllers $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$, none of which is individually authoritative. Vehicles periodically generate mobility reports containing position, speed, direction, and time information. These reports are forwarded to the controller through RSUs and are used by the controller to learn mobility models for network management and decision making.

3.2 Threat Model

This work considers a realistic threat model for SDVN, with a focus on poisoning attacks that target learning-based mobility and trajectory management. Unlike traditional attacks that disrupt communication, these attacks aim to corrupt the data used by the SDVN controller for learning and decision making, while normal communication behavior stay unchanged.

The SDVN architecture is assumed to consist of vehicles, Roadside Units (RSUs), and a set of K registered controllers $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$, all of which may act as potential attack sources. RSUs are initially assigned to controllers based on geographic proximity or load-balancing through standard SDVN mechanisms. The blockchain continuously maintains trust scores for all registered controllers. When a controller is detected as malicious, RSUs previously assigned to it reassign to any currently trusted controller from $\mathcal{C}$ without disrupting network operation. Since the Cloud/ITS Server feeds aggregate results into controller decision pipelines, it is treated as part of the control plane threat boundary and is not assumed to be unconditionally trusted. Attacks may target both the data plane, where mobility and trajectory information is transmitted, and the control plane, where learning processes and control decisions are generated. It is also assumed that controllers rely on learned mobility and trajectory patterns for safety related and traffic management decisions.

The framework assumes the number of simultaneously compromised RSUs f satisfies $f \leq \lfloor (n-1)/3 \rfloor$, which is equivalent to the standard BFT condition $n \geq 3f+1$ and correctly tolerates f Byzantine faults for all valid n . The threshold parameters are set as $t_{\text{sign}} = f+1$ for TRS signing and $t_{\text{decrypt}} = f+2$ for FHE decryption. Setting $t_{\text{decrypt}} = f+2$ ensures that f RSUs and the Cloud, totalling $f+1$ shares, cannot reach the decryption threshold alone, preventing a coalition of f compromised RSUs plus a compromised Cloud from performing unilateral decryption. This maintains consistency across TRS signing, FHE decryption, and BFT revocation under a unified security parameterisation.

Under this threat model, attackers may be present at different components of the SDVN architecture. The following attacker types are considered:

- **Malicious Vehicles:** Vehicles with valid identities that intentionally send false mobility or trajectory data to influence the learning process.
- **Compromised Roadside Units (RSUs):** RSUs that intercept, alter, or generate false mobility data before forwarding it to the controller.
- **Man in the Middle (MitM) Adversaries:** Attackers that intercept and modify mobility messages exchanged between vehicles and RSUs.
- **Malicious or Compromised Controllers:** SDVN controllers that deliberately poison their internal learning models and issue incorrect control decisions.

Attackers are assumed to be capable of launching gradual and time series poisoning attacks, including identity impersonation and Sybil type behaviors. These attacks are designed to maintain realistic mobility patterns, allowing them to attack both data plane observations and control plane decisions without triggering sudden or obvious anomalies.

The threat model assumes that standard cryptographic mechanisms for authentication and secure communication are not broken by attackers. Physical layer attacks such as jamming and radio interference are not considered. The scope of this work is limited to data poisoning attacks that affect learning based mobility management, rather than Dos attack or availability attacks.

This threat model defines the assumptions used for the attack scenarios presented in the next section and directly guides the design of the proposed mitigation framework.

3.3 SDVN System Modeling

The SDVN is modeled as a system consisting of vehicles, RSUs, and a set of K registered controllers $\mathcal{C} = \{c_1, \dots, c_K\}$ whose trust scores are continuously maintained on the blockchain via SC-Trust. Each RSU r_j maintains a current controller assignment $c_{\text{assigned}}(r_j) \in \mathcal{C}$, initially determined by geographic proximity or load-balancing. The trusted controller set maintained by the blockchain at time t is shown in Equation 3.1:

$$\mathcal{C}_{\text{trusted}}(t) = \{c_k \in \mathcal{C} : \tau_{c_k}(t) > \tau_{\min}\} \quad (3.1)$$

As shown in Equation 3.1, any controller whose trust score falls below $\tau_{\min}$ is excluded from the trusted set and its flow rules are no longer accepted by RSUs. When $c_{\text{assigned}}(r_j)$ is revoked, RSU r_j queries the blockchain for $\mathcal{C}_{\text{trusted}}(t)$ and reassigns to any available controller in that set, maintaining uninterrupted network operation without manual intervention. Based on learned mobility information, each assigned controller generates control actions such as safety alerts, routing instructions, and traffic management decisions. These control messages are sent back to vehicles through RSUs. This system model captures the logical data flow and control interactions in an SDVN and forms the foundation for attack modeling and mitigation framework design.

3.4 Attack Modeling Based on Defined Scenarios

Based on the finalized problem sketches, multiple poisoning attack scenarios are systematically modeled in the simulation environment.

3.4.1 Trajectory Poisoning Attacks

Trajectory poisoning attacks aim to corrupt the temporal learning of vehicle movement paths by injecting falsified trajectory information over multiple time steps. These attacks affect how the SDVN controller learns and predicts vehicle trajectories, leading to incorrect safety and traffic management decisions.

Figure 3.1 shows a data plane trajectory poisoning attack which a compromised RSU change legitimate vehicle trajectory data before forwarding it to the SDVN controller.

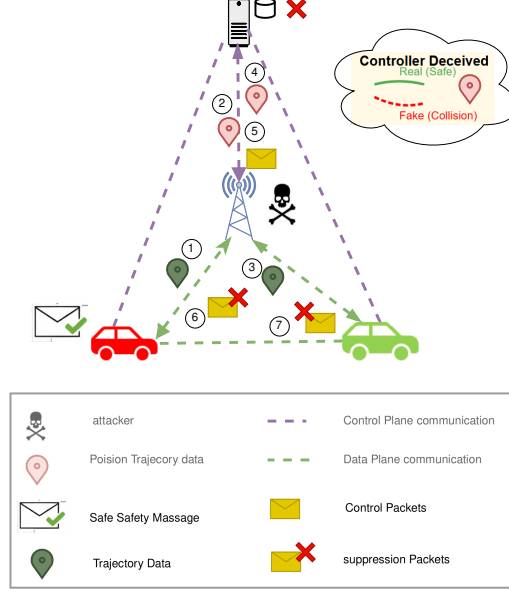


Figure 3.1: Malicious RSU based data-plane trajectory poisoning

- **Malicious RSU Trajectory Poisoning:** As shown in Figure 3.1, the attack begins when a legitimate vehicle sends correct trajectory information, such as its position and movement path, toward the RSU (Step 1). This trajectory data is first received by RSU as part of normal data plane communication.

Instead of forwarding the data honestly, the RSU is compromised and acts as the attacker. At Step 2, the malicious RSU modifies the received trajectory data by injecting falsified but realistic trajectory values. These poisoned trajectory packets still follow smooth movement patterns, making them difficult to detect and then the RSU forwards the poisoned trajectory data to the SDVN controller through the control plane communication channel. In the step 3 and 4, same transmission happen like another legitimate vehicle send the trajectory data to controller though the RSU.

At Step 5, Based on this corrupted input, the controller learns an incorrect trajectory model and is deceived into believing that the traffic situation is safe, even though a real collision risk exists. Using this wrong understanding, the controller generates incorrect control decisions and control packets (Step 5). As a result, safety critical warning messages that should be sent to vehicles are either suppressed or replaced with misleading control instructions (Step 7). Consequently, vehicles do not receive necessary safety alerts, which may lead to unsafe driving decisions and potential collisions, despite the vehicles behaving honestly throughout the process.

Figure 3.2 shows a data plane trajectory poisoning attack where a malicious vehicle

injects falsified but realistic trajectory information into the network.

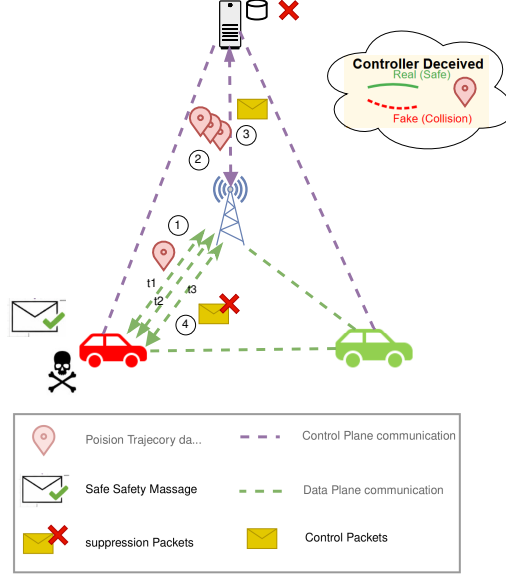


Figure 3.2: Malicious vehicle based data-plane trajectory poisoning

- **Malicious Vehicle Trajectory Poisoning:** As shown in Figure 3.2, the attack starts when a malicious vehicle acts as a legitimate participant in the network but intentionally injects falsified trajectory data. At Step 1, the compromised vehicle generates fake but smooth and realistic trajectory information instead of its true movement data. this poisoned trajectory data is transmitted to the RSU through normal data plane communication. Since the data follows realistic mobility patterns, the RSU does not detect any abnormal behavior and forwards the received information without modification.

At Step 2, the RSU sends the poisoned trajectory data to the SDVN controller via the control plane communication channel. Over multiple time steps, the controller continuously receives this falsified trajectory information and gradually learns an incorrect trajectory model for the malicious vehicle.

At the step 3. Based on this corrupted learning, the controller incorrectly assumes that the traffic situation is safe, and send the control signals even though a real collision risk exists . As a result, the controller generates improper control decisions and suppresses or delays safety critical warning messages that should be delivered to nearby vehicles (Step 4).

Consequently, surrounding vehicles do not receive timely safety alerts, leading to unsafe driving conditions and a higher risk of collision, despite normal network communication behavior being preserved throughout the attack.

Figure 3.3 illustrates a control plane trajectory poisoning attack in which the SDVN controller itself is malicious and poisons the learning process despite receiving correct data.

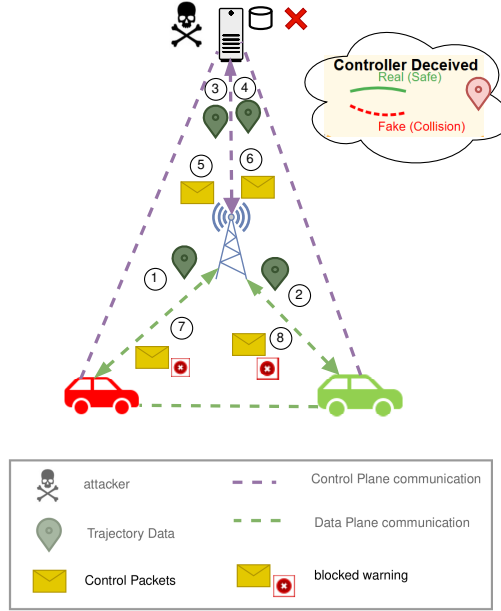


Figure 3.3: Control-plane trajectory poisoning

- **Control-Plane Trajectory Poisoning** As shown in Figure 3.3, this attack occurs entirely at the control plane while the data plane behaves correctly. At Step 1 and Step 2, vehicles honestly generate and transmit correct trajectory data to the RSU through normal data plane communication.

At Step 3 and 4, the RSU forwards this legitimate trajectory information to the SDVN controller using the control plane channel. Up to this point, no falsified data is injected, and the collected mobility information accurately reflects the real vehicle movements.

At Step 5 and 6, the attack begins inside the SDVN controller itself. Although it receives correct trajectory data, the controller is malicious or compromised and deliberately poisons its internal learning model. Instead of learning the real vehicle trajectories, the controller constructs a fake trajectory model that incorrectly indicates a safe traffic situation.

Based on this poisoned learning state, the controller generates incorrect control decisions at Step 5 and Step 6. These control packets are sent back to the RSU and then forwarded to vehicles (Step 7 and Step 8).

As a result, safety critical warning messages that should alert vehicles about a real collision risk are blocked or suppressed in the data plane. Even though

vehicles and RSUs behave honestly and communicate correctly, the malicious controller causes unsafe decisions, leading to potential collisions and traffic hazards.

3.4.2 Mobility Pattern Poisoning Attacks

Mobility pattern poisoning attacks try to corrupt the SDVN controller's global understanding of traffic flow and vehicle behavior by injecting falsified mobility data over time. Unlike trajectory poisoning, which focuses on individual vehicle paths, mobility pattern poisoning targets aggregated learning outcomes such as congestion estimation, traffic density prediction, and coordinated vehicle behavior.

Figure 3.4 illustrates a Sybil based mobility pattern poisoning attack launched through a compromised RSU.

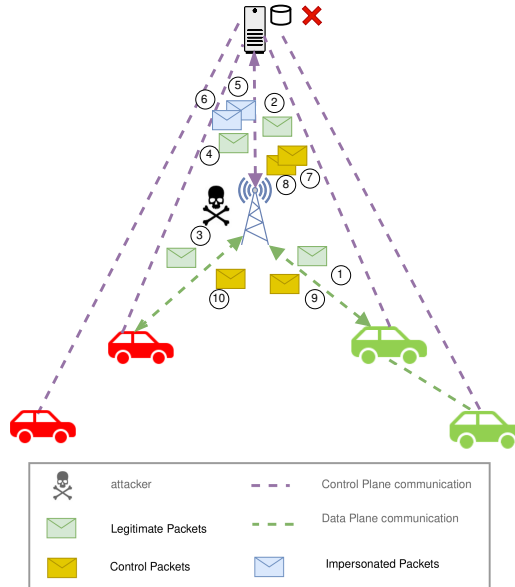


Figure 3.4: Sybil based attack via compromised RSU

- **Sybil Based Mobility Pattern Poisoning via Compromised RSU:**

As shown in Figure 3.4, the attack begins at Step 1 and 3 when legitimate vehicles transmit correct mobility data to the RSU through the data plane. These packets contain valid position, speed, and movement information and represent normal network behavior.

At step 2 and 4, legitimate packets are forwarded to the controller. Compromised rsu, impersonates multiple fake vehicle identities and then Instead of forwarding only the legitimate packets, the RSU generates additional forged mobility reports using different identities, as indicated at Step 5 and Step 6.

At Step 5 and 6 , impersonated mobility packets are forwarded to the SDVN controller through the control plane. Since the forged packets follow realistic mobility patterns and carry valid looking identities, the controller treats them as data coming from multiple independent vehicles.

As a result, the controller aggregates this poisoned mobility information into its global learning model. This leads to false traffic density estimation and incorrect global mobility patterns. Based on this corrupted model, the controller generates incorrect control decisions and sends false control signals back to the RSU at Steps 7 and 8.

Finally, at Steps 9 and 10, these incorrect control decisions reach the vehicles. This causes distorted traffic management actions and unsafe or inefficient network behavior, even though most vehicles in the network behave honestly.

Figure 3.5 illustrates a Sybil based mobility pattern poisoning attack via Malicious Vehicle Impersonation.

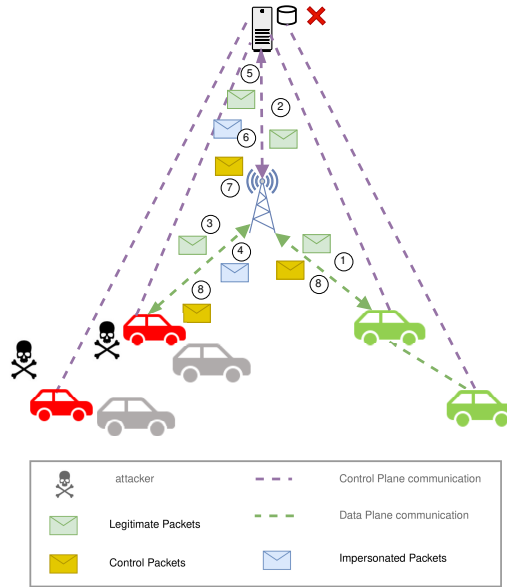


Figure 3.5: Sybil based attack via malicious vehicle impersonation

- **Sybil Based Mobility Pattern Poisoning via Malicious Vehicle Impersonation:** As shown in Figure 3.5, the attack starts at Step 1 and 3 when a malicious vehicle generates falsified mobility data while pretending to be a legitimate vehicle. The attacker forges multiple mobility packets using stolen or learned identities, even though these vehicles do not actually exist.

At Step 4, the malicious vehicle transmits impersonated mobility packets to the RSU through the data plane. These packets contain realistic position,

speed, and movement information, allowing them to bypass basic validity checks.

At Step 2, 5 and 6, the RSU forwards both the received mobility packets to the SDVN controller through the control plane without detecting the impersonation. Since the packets carry valid looking identities, the controller assumes that the data is coming from multiple independent vehicles.

The controller aggregates this poisoned mobility information into its global mobility learning model. As a result, false traffic density and incorrect mobility patterns are learned

Based on the corrupted learning model, the controller generates incorrect control decisions and sends control packets back to the RSU at Step 7. These control packets are then forwarded to vehicles at Step 8.

Finally, the vehicles receive incorrect traffic management and control decisions, leading to distorted mobility predictions and unsafe or inefficient network behavior, despite most vehicles in the network behaving honestly.

Figure 3.6 illustrates a Data plane MitM based mobility pattern poisoning attack

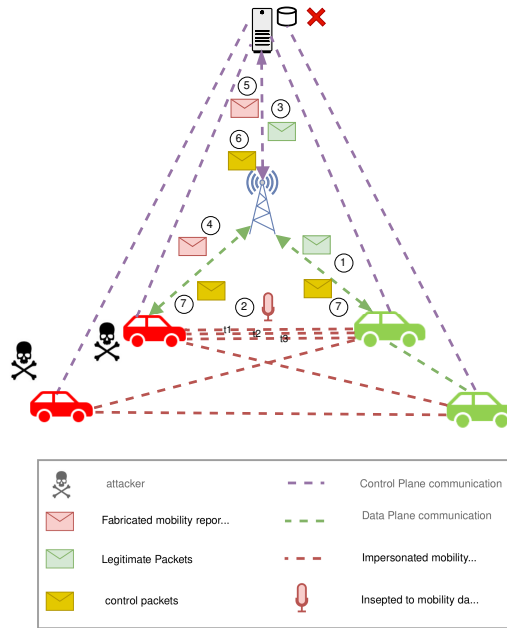


Figure 3.6: Data plane MitM based mobility pattern poisoning

- **Data Plane Mobility Pattern Poisoning via Man in the Middle Attack :** As shown in Figure 3.6, the attack begins at Step 1 when legitimate vehicles transmit correct mobility data, such as position and speed, to the

RSU through the data plane. This represents normal communication in the SDVN.

At Step 2, a Man in the Middle (MitM) attacker positioned in the data plane intercepts these mobility packets before they reach the RSU. The attacker observes the legitimate messages and learns valid vehicle identities and mobility formats. The attacker fabricates new mobility reports by modifying attributes such as location, speed, and acceleration, while keeping the values realistic to avoid detection. These forged mobility packets are generated over multiple time steps to mimic normal driving behavior. At Step 4, the poisoned mobility packets are forwarded to the RSU as if they were sent by legitimate vehicles. Since the packets follow valid formats and realistic patterns, the RSU does not detect any anomaly.

At Step 3 and Step 5, the RSU forwards both legitimate and poisoned mobility data to the SDVN controller through the control plane. The controller treats all received reports as trustworthy inputs from different vehicles.

At the step 6 The controller aggregates this poisoned mobility information into its global learning model. Over time, the accumulated false data corrupts the learned mobility patterns, and send back to the RSU.

At the step 7, the vehicle received incorrect control decisions based on the poisoned model, which may lead to unsafe, inefficient, or misleading traffic management actions, even though most vehicles in the network behave honestly.

Figure 3.7 shows a control plane mobility pattern poisoning attack in which the SDVN controller itself is malicious and corrupts the learned global mobility model despite receiving correct mobility data.

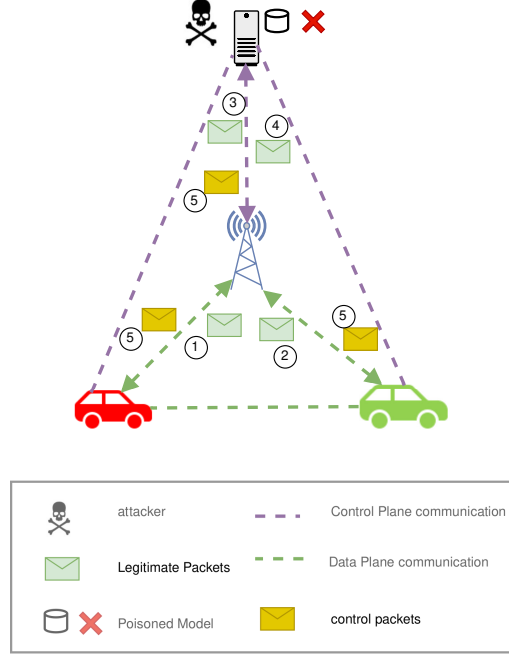


Figure 3.7: Control plane mobility pattern poisoning

- **Control Plane Mobility Pattern Poisoning:** As shown in Figure 3.7, the attack begins at Step 1 and Step 2 when vehicles transmit legitimate mobility data, such as position and speed, to the RSU through the data plane. This communication represents normal and honest vehicle behavior. At Step 3 and Step 4, the RSU forwards the collected mobility information to the SDVN controller via the control plane. At this stage, both the vehicles and the RSU operate correctly and do not inject any false data.

At Step 5, the SDVN controller, which is malicious or compromised, intentionally poisons its internal learning model despite receiving correct mobility inputs. Instead of learning the true global mobility patterns, the controller constructs a corrupted mobility model that misrepresents traffic conditions and vehicle behavior.

Based on the poisoned model, the controller generates incorrect control packets, as indicated at Step 5, and sends these faulty control decisions back to the RSU through the control plane. Finally, the RSU forwards the incorrect control packets to vehicles at Step 6 in the data plane. As a result, safety warnings may be suppressed, collision risks may be misinterpreted as safe situations, and traffic control actions become incorrect, even though all data plane mobility observations were accurate.

3.4.3 Mobility-Induced Threat Amplification in SDVN

Unlike static networks, SDVN attack surfaces are fundamentally shaped by vehicle kinematics. High vehicular mobility constrains RSU contact windows, accelerates topology changes, and reduces the number of observable beacons per vehicle–RSU association, all of which directly amplify the effectiveness of poisoning attacks. We formalize these mobility-induced amplification effects as follows.

3.4.3.1 RSU Contact Duration and Beacon Sampling Window

Let vehicle v_i travel at speed s_i along a trajectory with perpendicular distance $d_\perp$ from RSU r_j having coverage radius R_j . The contact duration τ_c is given in Equation 3.2.

This equation quantifies the available time window within which an RSU can observe and collect beacon evidence from a passing vehicle, establishing the upper bound on how much trajectory data any single RSU can accumulate for anomaly detection before the vehicle exits coverage.

$$\tau_c(v_i, r_j) = \frac{2\sqrt{R_j^2 - d_\perp^2}}{s_i}, \quad d_\perp \leq R_j \quad (3.2)$$

As shown in Equation 3.2, τ_c decreases as vehicle speed s_i increases or as the vehicle passes farther from the RSU centre, directly reducing the observation window available for attack detection.

Given a beacon broadcast interval T_b (typically 100 ms per IEEE 802.11p), N_b is the number of beacon messages the RSU r_j can observe from vehicle v_i during a single contact (one pass through coverage). The number of observable beacons N_b per RSU contact is given in Equation 3.3.

This equation translates the contact duration into a discrete count of observable beacon messages, directly determining whether the temporal autoencoder receives sufficient sequence length to achieve reliable detection power during a single RSU contact.

$$N_b(v_i, r_j) = \left\lfloor \frac{\tau_c(v_i, r_j)}{T_b} \right\rfloor \quad (3.3)$$

As shown in Equation 3.3, as s_i increases, τ_c decreases and N_b shrinks, directly reducing the evidence available for sequential anomaly detection. Let $N_{\min}$ be the minimum beacon sequence length required for a temporal detector to achieve target detection power.

The detection feasibility condition is given in Equation 3.4. This condition formally

defines when sequential anomaly detection is feasible at a given RSU: it identifies the speed threshold beyond which the lightweight and full mode detectors must defer to cross-RSU offline analysis, motivating the dual-mode architecture design.

$$N_b(v_i, r_j) \geq N_{\min} \quad (3.4)$$

As shown in Equation 3.4, when this condition is violated due to high speed, the temporal autoencoder receives insufficient sequence length per RSU contact, creating an evasion window exploitable by adversaries.

3.4.3.2 Handover-Induced Learning Discontinuity

The handover rate $\lambda_h(s_i)$ of vehicle v_i across RSUs is given in Equation 3.5. This equation models how frequently a vehicle transitions between RSU coverage zones, quantifying the degree to which trajectory sequences are fragmented across RSU records and thus unavailable to any single RSU's sequential detector which is a fragmentation effect that attackers deliberately exploit to evade cross-beacon correlation.

$$\lambda_h(s_i) = \frac{s_i}{\mathbb{E}[2\sqrt{R^2 - d_{\perp}^2}]} \quad (3.5)$$

As shown in Equation 3.5, high λ_h means vehicle trajectory sequences are split across multiple RSU records, disrupting the temporal continuity required for sequential poisoning detection. A malicious vehicle exploiting this effect injects falsified beacons only within a single RSU contact window $[t_0, t_0 + \tau_c]$ and remains honest otherwise, evading cross-RSU correlation where $\mathbb{E}[\cdot]$ is the expectation over the perpendicular distance distribution.

3.4.3.3 Mobility-Disguised Poisoning Drift Rate

In trajectory poisoning, an adversary injects a false position offset $\delta(t)$ that grows gradually over time. To avoid triggering threshold-based detectors, the attacker must ensure the per-step drift constraint given in Equation 3.6.

This equation characterizes the per-beacon falsification budget available to a trajectory poisoning attacker who wishes to remain undetected by threshold-based checks, establishing why gradual poisoning is structurally harder to detect than abrupt position jumps and motivating the cumulative drift score in TP-S5.

$$\|\delta(t) - \delta(t - T_b)\| \leq \epsilon_{\max} \quad (3.6)$$

As shown in Equation 3.6, the attacker is constrained to inject only small incremental offsets per beacon, forcing a slow gradual drift that mimics legitimate movement to evade threshold-based detectors where $\epsilon_{\max}$ is the maximum allowable positional deviation per beacon interval inferred from speed plausibility checks.

The cumulative falsified displacement after k beacons is given in Equation 3.7. This equation quantifies the total falsified displacement an attacker can accumulate over an entire RSU contact window while staying within per-beacon kinematic bounds, revealing that the attack budget grows with vehicle speed and directly motivating the cumulative drift score detector TP-S5 rather than per-beacon threshold checks alone.

$$\|\delta(t_0 + kT_b)\| \leq k \cdot \epsilon_{\max} \quad (3.7)$$

As shown in Equation 3.7, the attack budget per RSU contact is therefore bounded by $N_b \cdot \epsilon_{\max}$. At higher vehicle speeds, N_b decreases (Equation 3.3), but $\epsilon_{\max}$ increases because speed plausibility thresholds widen at high velocity, giving the attacker a larger per-beacon drift allowance. This creates a speed-proportional attack amplification effect unique to vehicular environments.

3.4.3.4 Sybil Identity Capacity Under Mobility Constraints

A compromised RSU performing Sybil-based mobility pattern poisoning (Section 3.4.2) can sustain at most K_{sybil} ghost identities whose beacons collectively remain spatially consistent. Given the RSU coverage area $\mathcal{A}_j = \pi R_j^2$ and a minimum inter-vehicle spacing $d_{\min}$ enforced by spatial plausibility checks, the Sybil identity capacity is bounded as given in Equation 3.8.

$$K_{\text{sybil}} \leq \left\lfloor \frac{\mathcal{A}_j}{\pi(d_{\min}/2)^2} \right\rfloor \quad (3.8)$$

As shown in Equation 3.8, the maximum number of ghost identities is directly bounded by the RSU coverage area and the minimum spacing constraint, meaning a larger coverage area or looser spacing allows more Sybil identities to coexist undetected. Since $\mathcal{A}_j$ is fixed but the number of legitimate vehicles in $\mathcal{A}_j$ varies with traffic density ρ_v (vehicles per m^2), the fraction of Sybil identities relative to total reported identities is given in Equation 3.9.

$$f_{\text{sybil}} = \frac{K_{\text{sybil}}}{\rho_v \cdot \mathcal{A}_j + K_{\text{sybil}}} \quad (3.9)$$

As shown in Equation 3.9, at low traffic density (e.g., highway scenarios with high s_i), $\rho_v \cdot \mathcal{A}_j$ is small, so f_{sybil} approaches 1, meaning a compromised RSU can dominate the controller's view of local traffic. This quantifies why mobility conditions that reduce traffic density directly amplify the Sybil attack's impact on the SDVN controller's learning model.

Figure 3.8 presents the combined mobility-induced threat amplification effects, illustrating how RSU contact duration and the adversarial drift budget vary with vehicle speed across urban, rural, and highway regimes.

FIGURE 4 — Mobility-Induced Threat Amplification

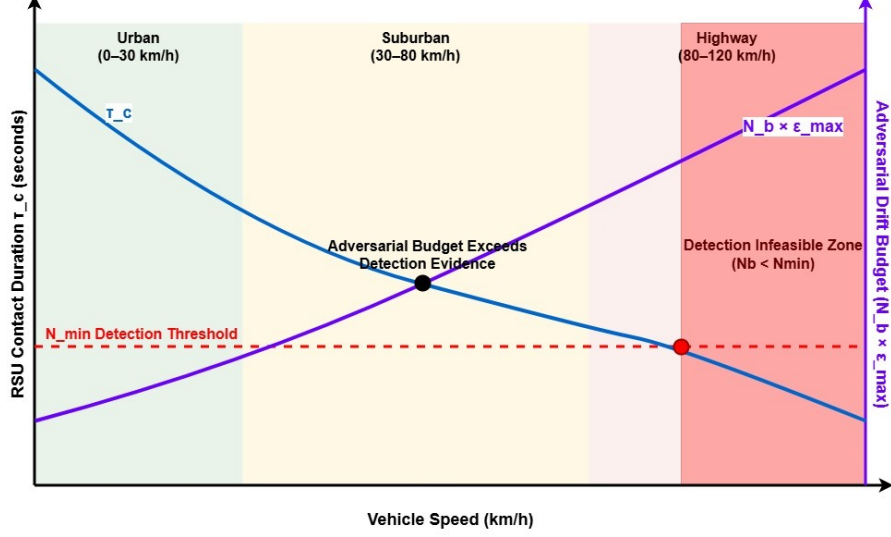


Figure 3.8: Mobility-Induced Threat Amplification.

As shown in Figure 3.8, as vehicle speed increases from urban to highway regimes, the RSU contact duration τ_c decreases while the adversarial drift budget $N_b \times \epsilon_{\max}$ increases, with their crossover point marking the Detection Infeasible Zone where $N_b < N_{\min}$ and sequential anomaly detection becomes unreliable.

3.4.4 Attack Signature Characterization

A Basic Safety Message (BSM) beacon broadcast by vehicle v_i at time t is defined as the tuple given in Equation 3.10. This tuple formally defines the observable input to all detection procedures, establishing which kinematic fields are available at the RSU for signature evaluation and making explicit that the detection framework operates solely on standard beacon content without requiring additional sensors or infrastructure.

$$\mathbf{b}_i(t) = (p_i(t), s_i(t), \theta_i(t), a_i(t), t, \text{ID}_i) \quad (3.10)$$

As shown in Equation 3.10, each BSM encodes the full kinematic state of vehicle v_i , forming the observable beacon stream $\{\mathbf{b}_i(t)\}$ from which the detectable signatures of each attack class are characterised where $p_i(t) \in \mathbb{R}^2$ is the GPS position, $s_i(t)$ is speed, $\theta_i(t)$ is heading angle, $a_i(t)$ is acceleration, and ID_i is the pseudonymous vehicle identifier.

Table 3.1 fixes the canonical naming used for the remainder of this report: each of the nine rule-based signatures is referred to by its TP-S/MP-S identifier only,

and each of the seven evaluated attack scenarios (as used in Experiment 5, Section 5.1, and throughout the Results chapter) is referred to by its scenario number (a1–a7) only. The two identifier schemes index different things — a signature is a detection rule, an attack scenario is a simulated adversary configuration — and are kept deliberately distinct rather than used interchangeably, since a single attack scenario may trigger several signatures simultaneously and a single signature may fire under several scenarios.

Table 3.1: Canonical mapping between attack scenarios and their primary rule-based detection signature.

Scenario	Primary signature	Plane	Compromised entity
a1	TP-S1	Data	Malicious vehicle
a2	TP-S2	Data	Compromised RSU
a3	MP-S1	Data	Compromised RSU (Sybil)
a4	MP-S2	Data	Vehicle (impersonation)
a5	TP-S3	Control	Malicious controller
a6	MP-S3	Data	MitM adversary
a7	MP-S4	Control	Malicious controller

TP-S4 and TP-S5 (cumulative drift residual and its threshold, Table 3.3) are supplementary checks within TP-DETECT that refine detection across all trajectory-poisoning scenarios rather than identifying a single scenario of their own, so they do not appear in Table 3.1.

3.4.4.1 Trajectory Poisoning Signatures

TP-S1 (Kinematic Position Feasibility). A reported position change must satisfy the speed-of-light constraint over beacon interval T_b . The kinematic position feasibility violation is defined in Equation 3.11.

This signature detects the most aggressive form of trajectory poisoning — a single-beacon position jump that violates physical speed limits — serving as the first and fastest gate in TP-DETECT that eliminates obviously falsified beacons before the more computationally expensive cumulative checks are applied.

$$\|p_i(t) - p_i(t - T_b)\| > s_{\max} \cdot T_b \quad (3.11)$$

As shown in Equation 3.11, a position displacement exceeding $s_{\max} \cdot T_b$ in one beacon interval flags a kinematic feasibility violation where $s_{\max}$ is the road-class maximum speed.

TP-S2 (Heading Rate Deviation). A realistic vehicle cannot change heading

faster than a maximum angular velocity $\omega_{\max}$ (rad/s). The heading rate deviation signature is given in Equation 3.12.

This signature detects unrealistic heading changes that cannot be produced by a road-constrained vehicle, catching trajectory falsifications where an attacker maintains a plausible position sequence but fabricates direction information to mislead path prediction models.

$$\Delta\theta_i(t) = |\theta_i(t) - \theta_i(t - T_b)| > \omega_{\max} \cdot T_b \quad (3.12)$$

As shown in Equation 3.12, a heading change exceeding $\omega_{\max} \cdot T_b$ over one beacon interval indicates an unrealistic turn rate inconsistent with vehicle dynamics.

TP-S3 (Kinematic Bound Violation). Vehicle acceleration and speed are physically bounded by road friction, vehicle dynamics, and the road-class speed limit. The kinematic bound violation signature is given in Equation 3.13.

This signature detects physically impossible acceleration or speed values, providing a complementary kinematic check to TP-S1. It catches falsifications where position increments appear plausible but the implied speed change violates vehicle dynamic limits, and—critically—attacks that report a constant yet physically impossible speed: a steady extreme speed produces zero implied acceleration (evading the acceleration term) on a plausible position (evading TP-S1), so the reported speed magnitude is the only direct tell.

$$|a_i(t)| > a_{\max} \quad \vee \quad s_i(t) > s_{\max} \quad (3.13)$$

As shown in Equation 3.13, any reported acceleration exceeding $a_{\max}$ or reported speed exceeding $s_{\max}$ is physically impossible and flags a falsified beacon, where $a_{\max}$ is the maximum achievable acceleration and $s_{\max}$ is the road-class maximum speed (Eq. 3.11). Because $s_{\max}$ is road-class specific, honest urban and rural traffic remains well below it, while for high-speed (highway) road classes $s_{\max}$ is set to that class’s speed limit so that legitimate high-speed vehicles are not flagged.

TP-S4 (Dead-Reckoning Residual). Using the previous beacon state, a predicted position is computed as given in Equation 3.14.

This prediction step reconstructs the expected vehicle position from its previously reported kinematic state, providing a ground-truth reference derived entirely from the vehicle’s own prior beacons against which the current reported position is compared to detect injected displacement.

$$\hat{p}_i(t) = p_i(t - T_b) + s_i(t - T_b) \cdot T_b \cdot \begin{bmatrix} \cos \theta_i(t - T_b) \\ \sin \theta_i(t - T_b) \end{bmatrix} \quad (3.14)$$

As shown in Equation 3.14, $\hat{p}_i(t)$ estimates where the vehicle should be based solely on its previously reported speed and heading.

The dead-reckoning residual $r_i(t)$ is given in Equation 3.15. This residual quantifies the discrepancy between what the vehicle reported and where its own prior kinematics predict it should be, providing a per-beacon anomaly signal that catches falsifications individually plausible by road-map checks but inconsistent with the vehicle’s own movement history.

$$r_i(t) = \|p_i(t) - \hat{p}_i(t)\| \quad (3.15)$$

As shown in Equation 3.15, $r_i(t)$ measures the deviation between the reported position and the kinematically predicted position, with a large residual indicating injected false displacement.

TP-S5 (Cumulative Drift Score). The cumulative drift score $D_i(k)$ for detecting gradual trajectory poisoning is given in Equation 3.16.

This score aggregates dead-reckoning residuals over a sliding beacon window to detect gradual trajectory poisoning that evades per-beacon checks, directly addressing the slow-drift attack strategy formalized in Equation 3.7 where individual beacons appear kinematically valid but their sequence reveals systematic falsification.

$$D_i(k) = \frac{1}{k} \sum_{j=1}^k r_i(t - (k - j) \cdot T_b) \quad (3.16)$$

As shown in Equation 3.16, $D_i(k)$ averages the dead-reckoning residuals over the last k beacons. A drift alert is issued when $D_i(k) > \delta_{\text{th}}$, where δ_{th} is a road-class calibrated threshold.

3.4.4.2 Mobility Pattern Poisoning Signatures

MP-S1 (Identity Density Anomaly). The number of distinct vehicle IDs reported within RSU r_j 's coverage must not exceed the Sybil identity capacity bound derived in Equation 3.8. The identity density anomaly check is given in Equation 3.17.

This check uses the Sybil capacity bound from Equation 3.8 as a data-driven threshold to flag RSU coverage zones where the number of reported vehicle identities exceeds what is physically possible, enabling Sybil detection without requiring knowledge of which specific identities are forged.

$$|\{\text{ID}_i : p_i(t) \in \mathcal{A}_j\}| > K_{\text{sybil}} + \rho_v \cdot \mathcal{A}_j \quad (3.17)$$

As shown in Equation 3.17, when the observed number of distinct IDs exceeds the sum of the Sybil capacity bound and the expected legitimate vehicle count, an identity density anomaly is flagged where $\mathcal{A}_j$ denotes the geographic coverage area of RSU r_j and ρ_v is the expected vehicle spatial density.

MP-S2 (Synchronized Beacon Timing). Sybil identities generated by a single source exhibit suspiciously correlated transmission timestamps. For identity pair $(\text{ID}_a, \text{ID}_b)$, the synchronized beacon timing signature is given in Equation 3.18.

This signature exploits the implementation artifact that Sybil identities generated by a single source share the same hardware clock, producing statistically improbable beacon timing correlations that expose co-sourced identity pairs even when their reported positions and speeds are individually plausible.

$$\frac{1}{K} \sum_{k=1}^K \mathbf{1} \left[|t_a^{(k)} - t_b^{(k)}| < \tau_{\text{sync}} \right] > \rho_{\text{sync}} \quad (3.18)$$

As shown in Equation 3.18, synchronization is flagged when the fraction of beacon pairs transmitted within τ_{sync} of each other exceeds the co-occurrence threshold ρ_{sync} where τ_{sync} is a sub-millisecond timing resolution threshold and $\rho_{\text{sync}} \in (0, 1)$ is a co-occurrence rate threshold.

Implementation note. An early implementation of MP-S2 contained a dead-code path in which the synchronised-timing check was constructed but never invoked from the combined-attack evaluation loop, so MP-S2's contribution to $\psi_i(t)$ was silently zero whenever multiple attack types were active simultaneously. This has been identified and corrected; all combined-attack results reported in this document reflect the corrected implementation.

MP-S3 (Regional Speed Distribution Shift). Mobility pattern poisoning

shifts the observed regional speed distribution $\mathcal{P}_t$ away from the historical baseline $\mathcal{P}_{\text{hist}}$. The regional speed distribution shift signature is given in Equation 3.19.

This signature detects MitM and mobility pattern poisoning attacks by measuring how much the current regional speed distribution has shifted from historical norms, catching large-scale traffic manipulation that affects the aggregate learned by the controller even when individual beacon modifications are individually subtle.

$$D_{\text{KL}}(\mathcal{P}_t \parallel \mathcal{P}_{\text{hist}}) = \sum_s \mathcal{P}_t(s) \log \frac{\mathcal{P}_t(s)}{\mathcal{P}_{\text{hist}}(s)} > \kappa_{\text{th}} \quad (3.19)$$

As shown in Equation 3.19, when the KL-divergence between the current and historical speed distributions exceeds κ_{th} , a regional mobility pattern shift is detected where κ_{th} is a calibrated KL-divergence threshold.

MP-S4 (Ghost Vehicle Transit Impossibility). A vehicle identity ID_i appearing at non-overlapping RSUs r_j and r_k within a transit time that violates kinematics is classified as a ghost identity. The ghost vehicle transit impossibility signature is given in Equation 3.20.

This signature detects ghost identities that appear at geographically separated RSUs within a transit time that violates physical speed limits, catching Sybil attacks that a single RSU cannot detect in isolation because each individual appearance is locally plausible.

$$\frac{d(r_j, r_k)}{|t_j - t_k|} > s_{\text{max}} \implies \text{ID}_i \text{ is a ghost identity} \quad (3.20)$$

As shown in Equation 3.20, if the implied travel speed between two RSU appearances exceeds s_{max} , the identity is physically impossible and is flagged as a ghost vehicle where $d(r_j, r_k)$ is the Euclidean distance between the two RSU positions.

3.4.4.3 Composite Rule-Based Anomaly Score

Each signature test produces a binary flag weighted by its per-attack-type reliability $w_k^{(\hat{k}_i)}$, learned in Phase 2 alongside the fusion weights. In the lightweight mode, a single calibrated weight vector w_k is used since the attack type is unknown. In the full mode, the predicted attack type $\hat{k}_i$ (Eq. 3.32) selects the weight vector most appropriate for that attack class, up-weighting the signatures that are discriminative for class $\hat{k}_i$ and suppressing signatures that do not fire for it. The attack-type-conditioned composite rule-based anomaly score for vehicle v_i at time

t is given in Equation 3.21.

$$\psi_i(t) = \frac{\sum_{s=1}^{|\mathcal{S}|} w_s^{(\hat{k}_i)} \cdot \mathbf{1}[\text{signature } s \text{ violated}]}{\sum_{s=1}^{|\mathcal{S}|} w_s^{(\hat{k}_i)}} \quad (3.21)$$

As shown in Equation 3.21, $\psi_i(t)$ is a reliability-weighted fraction of violated signatures, where the weight vector $\{w_s^{(\hat{k}_i)}\}$ is selected by the predicted attack type $\hat{k}_i$. In the lightweight operating mode, $\hat{k}_i$ is unavailable and a single calibrated weight vector $\{w_s\}$ (learned from aggregated Phase 2 data) is used instead. In the lightweight mode, $\psi_i(t) > \psi_{\text{th}}$ directly triggers an isolation response without AI inference. In the lightweight mode, $\psi_i(t) > \psi_{\text{th}}$ directly triggers an isolation response without AI inference where $\mathcal{S} = \{\text{TP-S1}, \dots, \text{MP-S4}\}$ is the full signature set, ψ_{th} is the composite anomaly score threshold, and δ_{th} is the drift alert threshold. As shown in Equation 3.21, $\psi_i(t)$ is a reliability-weighted fraction of violated signatures, ranging from 0 to 1.

The four detection procedures that formalise these signature checks are presented within the mitigation framework in Section 3.5.3.

3.5 Proposed Mitigation Framework

3.5.1 Cryptographic Mitigation Scope Analysis

Before formulating the mitigation framework, we determine whether cryptographic mechanisms alone can fully mitigate each attack variant, as this dictates their placement in the detection-mitigation pipeline.

Standard authentication schemes (PKI, certificate-based) verify identity but not data truthfulness [5]. A vehicle that possesses valid credentials can transmit false GPS coordinates, speeds, and trajectories that pass all cryptographic verifications. Therefore, cryptography alone cannot mitigate trajectory poisoning or mobility pattern poisoning originating from authenticated malicious vehicles or compromised RSUs.

However, a post-quantum threshold ring signature (TRS) scheme requiring collaborative signing by at least t -of- n neighbouring RSUs [12] structurally prevents any minority-compromised RSU subset (of size $< t$) from generating a valid aggregate signature. This mitigation acts before detection: a poisoned aggregate that fails threshold verification is rejected at the aggregation step without requiring AI-based anomaly scoring. Accordingly, TRS is placed **pre-detection** in the pipeline. TRS verification binds each submission to an authorised ring identity (Eq. (3.56)); it attests who signed, not the numerical validity of the aggregate they contributed. Insider poisoning by an authorised ring member, in which a field of

$\mathbf{A}_j(t)$ is perturbed while remaining inside the valid SUMO-derived envelope, is therefore not rejected by TRS and is not caught by the pre-encryption self-check of Algorithm 6, since that check is administered by the contributing RSU itself and a malicious member may bypass it. We instead treat this case as a bounded residual. Its impact is contained by three independent mechanisms. First, static plausibility envelopes applied both before encryption and after threshold decryption cap the magnitude of any single contribution, so an in-bounds perturbation cannot exceed the design-time field range. Second, because $n \geq 3f + 1$ bounds the number of compromised rings at $\leq f$, the global aggregate $\bar{X}_{\text{global}}$ is a mean over M rings dominated by the $\geq 2f + 1$ honest contributions, so a single poisoned ring shifts the mean only within a bounded residual rather than controlling it. Third, across successive windows the SC-Trust score of a persistently anomalous RSU decays (Eq. (3.63)) and triggers $2f + 1$ BFT revocation (Eq. (3.69)), removing the source. The unavoidable limitation is that a single in-window, in-bounds poisoned contribution is bounded in magnitude but not per-RSU attributable from the blinded aggregate alone; it is mitigated temporally rather than eliminated instantaneously.

Post-quantum fully homomorphic encryption (FHE) via BFV, BGV, or CKKS schemes [13] addresses a precise privacy threat inherent to the TRS ring coordination design: when t -of- n neighboring RSUs collaborate to collectively sign an aggregate, each RSU in the signing ring is exposed to the aggregate contents of every other participating RSU, which it has no legitimate reason to observe. A compromised RSU in the ring therefore learns traffic patterns from neighboring zones without detection. FHE eliminates this leakage by requiring each RSU to encrypt its own local aggregate before sharing it with the signing ring. Ring RSUs perform homomorphic addition across ciphertexts, producing a combined encrypted aggregate on which TRS is applied. Since computation proceeds entirely on ciphertext, no RSU in the signing ring ever observes another RSU’s plaintext aggregate. FHE is therefore placed **pre-TRS coordination** and is irreplaceable by simpler encryption in this role, as the ring must compute on ciphertexts without decryption.

For the lightweight pipeline, HMAC-based beacon tagging provides a computationally cheap integrity anchor at RSU ingestion, preventing undetected in-transit modification (MitM scenarios) without requiring AI inference.

The AI components (GAT and Temporal Autoencoder) constitute purely **detection logic**, operating on received beacon sequences to produce anomaly scores. The blockchain layer serves **both** detection (through smart contract-based consensus scoring) and mitigation (through on-chain identity revocation and trust enforcement). Table 3.2 summarises these placements.

Table 3.2: Detection vs. Mitigation Placement of Framework Components

Component	Role	Placement	Mode
Rule-based signatures	Detection	Pre-model	Lightweight & Full
HMAC beacon tagging	Mitigation	Pre-detection	Lightweight
GAT	Detection	Post-ingestion	Full only
Temporal Autoencoder	Detection	Post-ingestion	Full only
TRS (post-quantum)	Mitigation	Pre-detection	Full only
FHE (post-quantum)	Mitigation	Pre-TRS coordination	Full only
Blockchain (ledger)	Both	RSU-direct, continuous	Lightweight & Full
Smart contracts	Both	RSU-triggered, event-driven	Lightweight & Full

As shown in Table 3.2, the framework components are distributed across pre-detection, post-ingestion, and continuous placement stages, with cryptographic components (HMAC, TRS, FHE) handling mitigation and AI components (GAT, Temporal Autoencoder) handling detection, while the blockchain and smart contracts serve both roles across both lightweight and full operational modes.

3.5.2 Group Key Management via Logical Key Hierarchy

The framework requires two classes of keys: vehicle session keys $\{K_i\}$ for HMAC beacon integrity (Equation 3.42), and RSU ring key material $\{sk_j\}$ for TRS partial signing (Equation 3.54). We adopt a Logical Key Hierarchy (LKH) to manage both classes efficiently under vehicle join, leave, and RSU membership changes, following the group key management principle underlying broadcast encryption for V2I [23].

3.5.2.1 LKH Tree Structure

The LKH is organised as a balanced binary key tree $\mathcal{T}$ of depth d , where leaf nodes represent individual entities (vehicles or RSUs) and internal nodes hold group encryption keys. Let $\mathcal{K}_u$ denote the key assigned to node $u \in \mathcal{T}$. The set of keys held by each leaf entity e_i along its root path is given in Equation 3.22.

$$\mathcal{K}_{e_i} = \{\mathcal{K}_u : u \in \text{path}(u_i \rightarrow \text{root})\} \quad (3.22)$$

As shown in Equation 3.22, each entity holds exactly the keys along its path to the root, meaning that rekeying only requires updating nodes on one root path rather than all members. The root key $\mathcal{K}_{\text{root}}$ serves as the current group key shared among all active members. A broadcast message encrypted under $\mathcal{K}_{\text{root}}$ is decryptable by all legitimate members and no revoked entity.

The FHE key shares $\{sk_j^{\text{share}}\}_{j=1}^n$ produced during threshold setup (Eq. (3.60)) are distributed to RSU ring members via the same authenticated LKH ring key synchronisation channel, eliminating the need for a separate key distribution protocol for FHE share delivery.

3.5.2.2 Vehicle Session Key Establishment

When vehicle v_i registers with RSU r_j , it is assigned a leaf node u_i in a local LKH subtree maintained per RSU coverage zone. The RSU derives v_i 's session key as a keyed hash of the leaf key and a session nonce η_i . The vehicle session key derivation during RSU registration is given in Equation 3.23.

This derivation produces a per-session HMAC key unique to each vehicle registration, ensuring that a session key compromised at one RSU contact cannot be reused to forge authenticated beacons at subsequent RSU contacts after the vehicle moves on.

$$K_i = \text{KDF}(\mathcal{K}_{u_i}, \eta_i, \text{ID}_i) \quad (3.23)$$

As shown in Equation 3.23, the session key K_i binds the entity's leaf key, a fresh nonce, and its identity, ensuring per-session uniqueness where $\text{KDF}(\cdot)$ is a key derivation function (e.g., HKDF-SHA256). This key is used directly in Equation 3.42 for per-beacon HMAC tagging.

3.5.2.3 Key Update on Vehicle Leave or Revocation

When vehicle v_i leaves the RSU coverage zone or is revoked via SC-Revoke (Equation 3.69), the RSU must update all keys on v_i 's root path to preserve forward secrecy. The number of re-encrypted key messages required upon vehicle leave or revocation is given in Equation 3.24.

This logarithmic cost expression justifies the LKH design choice over individual unicast rekeying: as vehicle density increases, the LKH overhead grows only logarithmically while unicast rekeying grows linearly, making LKH the only scalable option for high-density urban deployments.

$$N_{\text{rekey}} = \log_2 |\mathcal{V}_j| \quad (3.24)$$

where $|\mathcal{V}_j|$ is the current number of vehicles in the RSU zone. As shown in Equation 3.24, the logarithmic rekey cost derived from the LKH tree depth provides a significant efficiency advantage over individual unicast rekeying, which costs $O(|\mathcal{V}_j|)$ messages [23].

3.5.2.4 RSU Ring Key Generation via Distributed Key Generation (DKG)

To eliminate the controller as a single point of failure on the key plane, TRS signing keys and FHE key shares are generated via a Distributed Key Generation (DKG) protocol executed directly among the RSU ring members and the Cloud, with no single dealer and no controller involvement. Each participant contributes a random secret polynomial, broadcasts a commitment to the polynomial coefficients, exchanges encrypted share evaluations, and verifies received shares against commitments. The blockchain serves as the authenticated public bulletin board for commitment publication, anchoring key generation in the same trust layer as identity registration.

The DKG protocol producing individual signing key pairs per RSU — one secret key and one public key per participant — without any central authority or single dealer is shown in Equation 3.25:

$$(\{sk_j, pk_j\}_{j=1}^n) \leftarrow \text{DKG}(\mathcal{R}, n, t_{\text{sign}}) \quad (3.25)$$

As shown in Equation 3.25, each RSU receives its own individual key pair (sk_j, pk_j) . The per-RSU public keys $\{pk_j\}_{j=1}^n$ are used directly in TRS verification via Eq. (3.56), maintaining full consistency with the verification step. No RSU and no controller observes any other RSU's secret key at any point during or after key generation.

The distributed FHE key generation producing shares for all participants without any single dealer is shown in Equation 3.26:

$$(pk_{\text{FHE}}, \{sk_j^{\text{share}}\}_{j=1}^n, sk_{\text{cloud}}^{\text{share}}, evk) \leftarrow \text{FHE-DKG}(\mathcal{R} \cup \{\text{Cloud}\}, n+1, t_{\text{decrypt}}) \quad (3.26)$$

As shown in Equation 3.26, no controller participates in TRS or FHE key generation at any stage. Vehicle session keys K_i continue to be derived locally at each RSU via Eq. (3.23), as the vehicle-RSU session key relationship is independent of ring-level key generation. The FHE key shares $\{sk_j^{\text{share}}\}_{j=1}^n$ produced during threshold setup are distributed to RSU ring members via the same authenticated LKH ring key synchronisation channel, eliminating the need for a separate key distribution protocol for FHE share delivery.

3.5.3 Dual-Mode Detection and Mitigation Pipeline

The SDVN controller contributes full-mode GAT and temporal autoencoder anomaly scores to the blockchain as one untrusted peer submission, subject to the same BFT verification as RSU submissions (Equation 3.68). Since the threat model includes a compromised controller, no blockchain decision trust score update, revocation, or flow rule enforcement relies exclusively on controller-supplied evidence. RSUs independently submit beacon-level anomaly evidence directly to the blockchain (Equation 3.67), and revocation is triggered by RSU peer consensus alone (Equation 3.69).

The proposed framework operates in two modes. The lightweight mode executes entirely at the RSU using rule-based signature detection and HMAC-based integrity protection, requiring no AI inference. The full mode augments this with GAT-based spatial detection and temporal autoencoder anomaly scoring at the SDVN controller, together with post-quantum TRS and FHE cryptographic mitigation. Both modes share the blockchain trust layer and smart contract arbitration.

Figure 3.9 presents the dual-mode system architecture of the proposed MPTD-PQS framework.

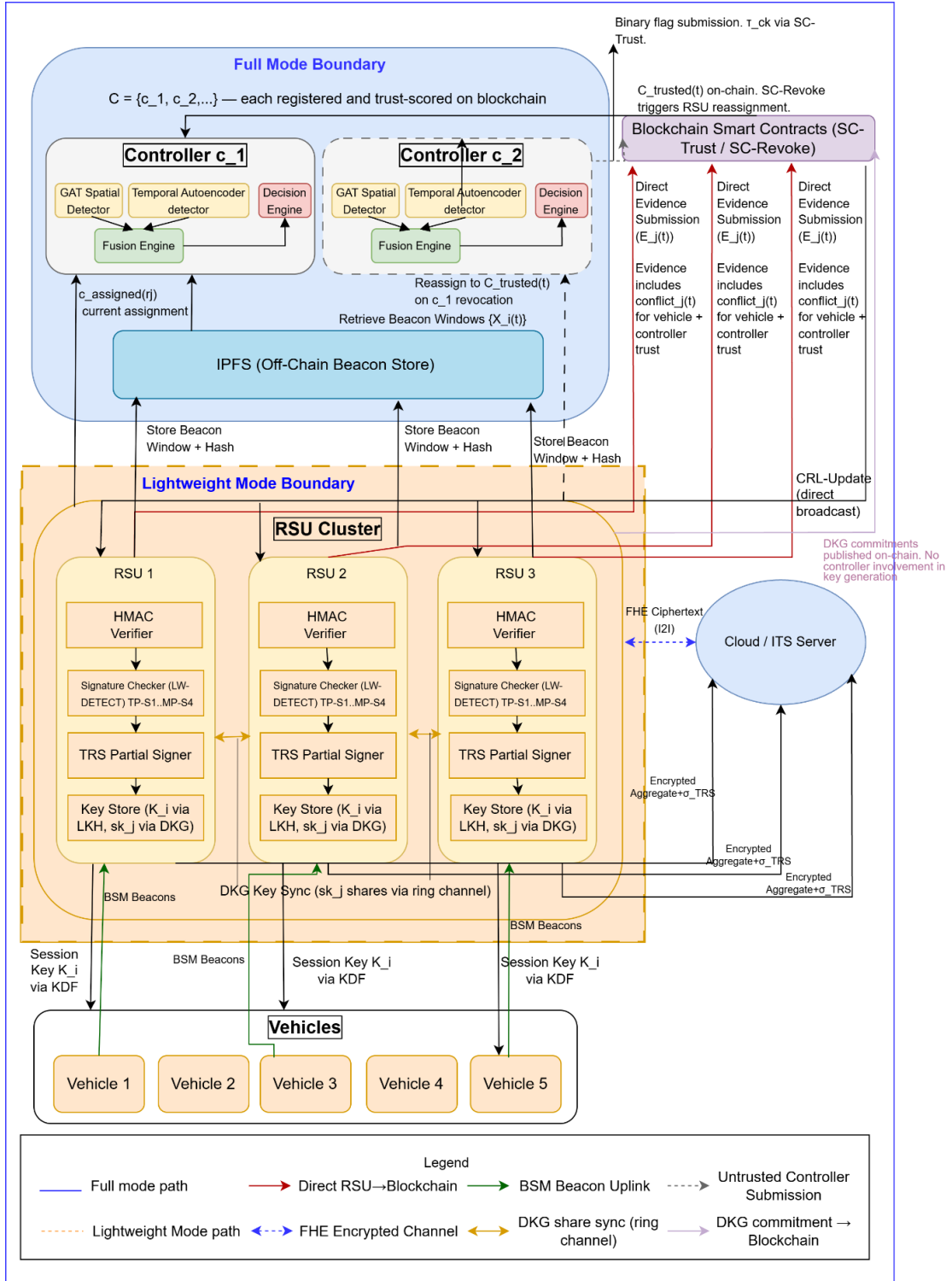


Figure 3.9: Dual-mode system architecture of the proposed MPTD-PQS framework.

As shown in Figure 3.9, the architecture is organised into three horizontal layers

that reflect the actual deployment boundaries of the framework. At the bottom, the vehicle layer contains five representative vehicles that broadcast BSM beacons over the V2I channel and establish per-vehicle session keys K_i through a KDF exchange with their serving RSU. The middle layer, enclosed by the lightweight mode boundary (orange dashed border), contains the RSU cluster comprising RSU 1 through RSU 3. Each RSU executes an identical four-stage pipeline: the HMAC Verifier (Equation 3.42) checks beacon integrity first, the Signature Checker evaluates all nine rule-based signatures TP-S1 through MP-S4 via the **LW-DETECT** procedure, the TRS Partial Signer (Equation 3.54) accumulates partial threshold signatures from peer RSUs over the LKH ring key synchronisation channel, and the LKH Key Store maintains the per-vehicle session keys K_i and the RSU signing key sk_j . When an anomaly is detected, each RSU submits a direct evidence tuple $\mathcal{E}_j(t)$ to the Blockchain Smart Contracts (SC-Trust / SC-Revoke) via the red direct-submission channel; the blockchain issues a CRL update back to the RSU cluster by direct broadcast without routing through the controller. Beacon windows are stored off-chain in the IPFS Off-Chain Beacon Store together with their cryptographic hashes, decoupling real-time RSU processing from controller-side deep learning analysis. The top layer, enclosed by the full mode boundary (blue solid border), contains the SDVN Controller, which retrieves beacon windows $\{\mathbf{X}_i(t)\}$ from IPFS, computes GAT spatial anomaly scores and temporal autoencoder reconstruction errors, and fuses them via the Fusion Engine (Equation 3.48) before the Decision Engine submits a single untrusted peer evidence entry to the blockchain indicated by the grey dashed arrow labelled Untrusted Peer Submission not required for revocation. On the right, FHE-encrypted, TRS-verified aggregates are forwarded to the Cloud/ITS Server over the encrypted channel only after the threshold signature σ_{TRS} has been assembled across the RSU cluster, ensuring privacy and structural integrity guarantees are enforced before any data leaves the RSU boundary.

Before detailing the individual detection, cryptographic, and trust-management mechanisms of the proposed framework, this section consolidates the mathematical notation used throughout the remainder of the chapter. Table 3.3 presents a comprehensive summary of every symbol introduced in the design, organised so that each notation can be quickly located alongside the equation in which it is first defined. Grouping the symbols in this manner allows the subsequent formulations—which span the lightweight rule-based signatures, the deep-learning detection pipeline, and the post-quantum cryptographic primitives—to be read without repeatedly restating the meaning of each variable.

Table 3.3: A summary of the notations.

Notation	Description
$v_i, r_j, c_k; \mathcal{R}, \mathcal{C}; \mathcal{R}_{\text{trusted}}(t), \mathcal{C}_{\text{trusted}}(t); c_{\text{assigned}}(r_j); \mathcal{V}_j(t)$	Vehicle i , RSU j , controller k ; set of RSUs and set of K registered controllers $\mathcal{C} = \{c_1, \dots, c_K\}$; trusted-RSU endorser set and trusted-controller set at time t ; RSU r_j 's currently assigned controller; vehicles within RSU r_j 's coverage zone
$f, n; t_{\text{sign}}, t_{\text{decrypt}}; \mathcal{S}, \mathcal{D}; M; T_w, T_{\text{rev}}$	Tolerated Byzantine faults and number of RSUs ($n \geq 3f+1$); TRS signing threshold ($f+1$) and FHE decryption threshold ($f+2$); TRS signing subset and decryption coalition; number of ring aggregates; revocation temporal window and required low-trust epochs
$\mathcal{R}_i^{\text{obs}}(t); \mathcal{R}_{c_k}^{\text{obs}}(t)$	Trusted RSUs that observed vehicle v_i in epoch t : $\{r_j \in \mathcal{R}_{\text{trusted}}(t) : v_i \in \mathcal{V}_j(t)\}$; trusted RSUs assigned to controller c_k in epoch t : $\{r_j \in \mathcal{R}_{\text{trusted}}(t) : c_{\text{assigned}}(r_j) = c_k\}$. Normalising denominators in Eqs. (3.63)–(3.64); trust is unchanged when the observing set is empty.
$\mathbf{b}_i(t); p_i(t), s_i(t), \theta_i(t), a_i(t); \text{ID}_i; T_b$	BSM beacon tuple of vehicle i (Eq. 3.10); GPS position, speed, heading angle, and acceleration; pseudonymous vehicle identifier; beacon broadcast interval (100 ms)
$\tau_c, N_b, N_{\min}, \lambda_h; R_j, d_{\perp}, \mathcal{A}_j; \rho_v, d_{\min}; K_{\text{sybil}}, f_{\text{sybil}}; \delta(t), \epsilon_{\max}$	RSU contact duration, observable-beacon count, minimum detectable sequence length, handover rate (Eq. 3.2–3.5); RSU coverage radius, perpendicular distance, coverage area; traffic density and minimum inter-vehicle spacing; Sybil identity capacity and Sybil fraction (Eq. 3.8–3.9); injected position-offset drift and per-beacon drift bound
$s_{\max}, \omega_{\max}, a_{\max}; \hat{p}_i(t), r_i(t), D_i(k), \delta_{\text{th}}; D_{\text{KL}}, \mathcal{P}_t, \mathcal{P}_{\text{hist}}, \kappa_{\text{th}}; \tau_{\text{sync}}, \rho_{\text{sync}}$	Road-class maximum speed, heading rate, and acceleration bounds (TP-S1–S3); dead-reckoned position, residual, cumulative drift score and its threshold (TP-S4–S5); KL-divergence of current vs. historical speed distribution and its threshold (MP-S3); beacon-timing and co-occurrence thresholds (MP-S2)
$\psi_i(t); \bar{\psi}_i(t); \psi_{\text{th}}; w_s^{(k)}, w_s; \theta_{\text{conf}}^{(k)}; \bar{\lambda}_j$	Composite rule-based anomaly score per beacon (Eq. (3.21)); its per-detection-window mean $\bar{\psi}_i(t) = \frac{1}{L} \sum_{l=1}^L \psi_i(t_l)$, normalised as $\min(\bar{\psi}_i/\psi_{\text{th}}, 1)$ in the full-mode fusion (Eq. (3.48)); its rule-based alert threshold; attack-type-conditioned signature reliability weight for signature s under class k (full mode) and aggregate calibrated weight (lightweight mode); per-attack-class GAT confidence gate threshold; fallback fusion weight (average over all K sets)
$K_i, \nu_i, \Delta_{\text{HMAC}}; \text{MAC}_i(t)$	Vehicle session key, monotonic anti-replay nonce, freshness window; HMAC beacon-integrity tag (Eq. 3.38)
$\mathcal{T}, \mathcal{K}_u, \mathcal{K}_{\text{root}}; \eta_i, N_{\text{rekey}}; sk_j, pk_j$	LKH key tree, per-node key and root (group) key; session nonce and logarithmic rekey cost (Eq. 3.22–3.24); RSU TRS signing/verification key pair from DKG (Eq. 3.25)

continued on next page

Table 3.3 – continued from previous page

Notation	Description
$\mathcal{G}(t), \mathcal{V}(t), \mathcal{E}(t);$ $R_{\max}, \phi_{\max}, \mathbf{v}_i;$ $\tilde{x}_i^{(f)}, \alpha_{ij}^{(h)}; \mathbf{W}_h, \mathbf{a}_h, H;$ $\mathbf{x}'_i, \mathcal{S}_i(t)$	Dynamic vehicle-RSU graph with nodes and mobility-aware edges; max. range, heading-divergence bound and velocity vector (Eq. 3.27); z-score-normalised feature and head- h attention coefficient; head weight matrix, attention vector, number of heads; node embedding and GAT spatial anomaly score (Eq. 3.39)
$y_i^{(k)}, \hat{y}_i^{(k)}; K; \hat{k}_i;$ $\mathbf{w}_{\text{cls}}^{(k)}, b_{\text{cls}}^{(k)}; w_k^+, w_k^-;$ $\mathcal{V}_{\text{train}}, \mathcal{V}_k^+, \mathcal{V}_k^-; \mathcal{L}_{\text{GAT}}$	Ground-truth and predicted binary node label for attack class k ($y_i^{(k)}=1$ attacking with class k , $y_i^{(k)}=0$ normal); number of attack classes ($K=7$); predicted attack type (Eq. (3.32)); per-class GAT classification head parameters, discarded after training (Eq. (3.31)); per-class inverse-frequency class weights (Eq. (3.34)); labeled training node set and its per-class positive and negative subsets; multi-task GAT supervised cross-entropy training loss (Eq. (3.34))
$\mathbf{X}_i(t), L, d;$ $\mathbf{f}_l, \mathbf{u}_l, \mathbf{g}_l, \mathbf{o}_l, \mathbf{c}_l, \mathbf{h}_l;$ $\hat{\mathbf{X}}_i(t), \mathcal{L}_{\text{total}}, \beta;$ $\varepsilon_i(t), \theta_{ae}, \theta_S, \kappa, \alpha_{\text{freeze}}$	Beacon-window input, window length and feature dimension; LSTM forget/input/candidate/output gates, cell and hidden states (Eq. 3.31–3.36); reconstruction, mobility-constrained loss and kinematic-penalty weight (Eq. 3.37); reconstruction error, offline MAD-based detection threshold, GAT spatial normalisation threshold (Eq. 3.48) scale factor and manifold-freeze threshold (Eq. 3.42–3.43)
$\Phi_i(t), \lambda_j^{(k)}, \Phi_{\text{th}}; \theta_S;$ $\text{flag}_i(t); \text{flag}_i^{\text{GAT}}$	Fused anomaly score in $[0, 1]$; attack-conditioned fusion weight for component j under attack class k ($\sum_j \lambda_j^{(k)}=1$, $\lambda_j^{(k)} \geq 0.05$; K sets learned in Phase 2, selected at inference by $\hat{k}_i$, Eq. (3.48)–(3.49)); GAT spatial score normalisation threshold (Phase 1b, locked); binary detection flag (Eq. (3.49)); GAT multi-task binary detection flag (Eq. (3.33))
$\mathbf{A}_j(t), c_j,$ $\text{Enc}(\mathbf{A}_{\text{ring}});$ $pk_{\text{FHE}}, sk_j^{\text{share}},$ $sk_{\text{cloud}}^{\text{share}}, evk$	RSU local plaintext aggregate, its FHE ciphertext, and homomorphic ring aggregate (Eq. 3.46–3.48); FHE public key, per-RSU and Cloud threshold key shares, and evaluation key (Eq. 3.26, 3.56)
$m, \nu_S, \text{ID}_S, h(\mathcal{S});$ $\sigma_j, \sigma_{\text{TRS}}, \Delta_{\text{TRS}}$	Signed TRS message and its ring nonce, ring identity and member-set hash (Eq. 3.49); per-RSU partial signature, aggregated threshold ring signature, and verification freshness window (Eq. 3.50–3.53)
$\bar{X}_{\text{global}}, pd_j, pd_{\text{cloud}};$ $L^{\text{SUMO}}, U^{\text{SUMO}}$	Decrypted global aggregate, per-RSU and Cloud partial decryptions (Eq. 3.54–3.58); offline SUMO-derived lower/upper plausibility-envelope bounds
$\tau_i(t), \tau_{c_k}(t), \alpha,$ $\tau_{\min}, \tau_{\text{warn}};$ $\mathcal{E}_j(t), \mathcal{E}_c(t),$ $\sigma_j^{\text{sub}}, \sigma_c^{\text{sub}};$ $\text{conflict}_j(t), \text{flag}_c;$ $n_{\text{endorse}}, \tau_{\text{init}}$	Vehicle and controller trust scores, EMA decay factor, revocation and probationary thresholds (Eq. 3.59–3.60); RSU and controller evidence tuples with their signatures (Eq. 3.61–3.62); directional controller-RSU conflict indicator and controller-compromise flag (Eq. 3.66–3.67); registration endorsement count and initial trust (Alg. 8)

As summarised in Table 3.3, the notations are grouped by their functional role within the framework. The first group defines the core network entities—the vehi-

cles, RSUs, and the set of registered controllers together with the trusted-controller set $\mathcal{C}_{\text{trusted}}(t)$ and the per-RSU controller assignment. The next group captures the Byzantine fault-tolerance and threshold parameters that govern the signing and decryption coalitions, followed by the beacon and kinematic quantities that constitute each vehicle’s reported state. Subsequent groups cover the nine rule-based trajectory- and mobility-poisoning signatures and their composite score $\psi_i(t)$, the HMAC and LKH key-management variables, and the deep-learning components comprising the GAT spatial detector, the LSTM temporal autoencoder, and their fused anomaly score $\Phi_i(t)$. The final groups list the post-quantum threshold cryptographic primitives—spanning the per-RSU FHE encryption, the homomorphic ring aggregate, the threshold ring signature σ_{TRS} , and the threshold decryption coalition—together with the blockchain trust-management variables that drive the SC-Trust and SC-Revoke decisions. Each symbol is annotated with the equation in which it first appears, so that the table serves as a single reference point for the detailed formulations presented in the following subsections.

Figure 3.10 presents the UML sequence diagram for the lightweight operating mode, tracing the per-beacon message flow across the vehicle, RSU, SC-Trust smart contract, and controller entities during a single beacon epoch.

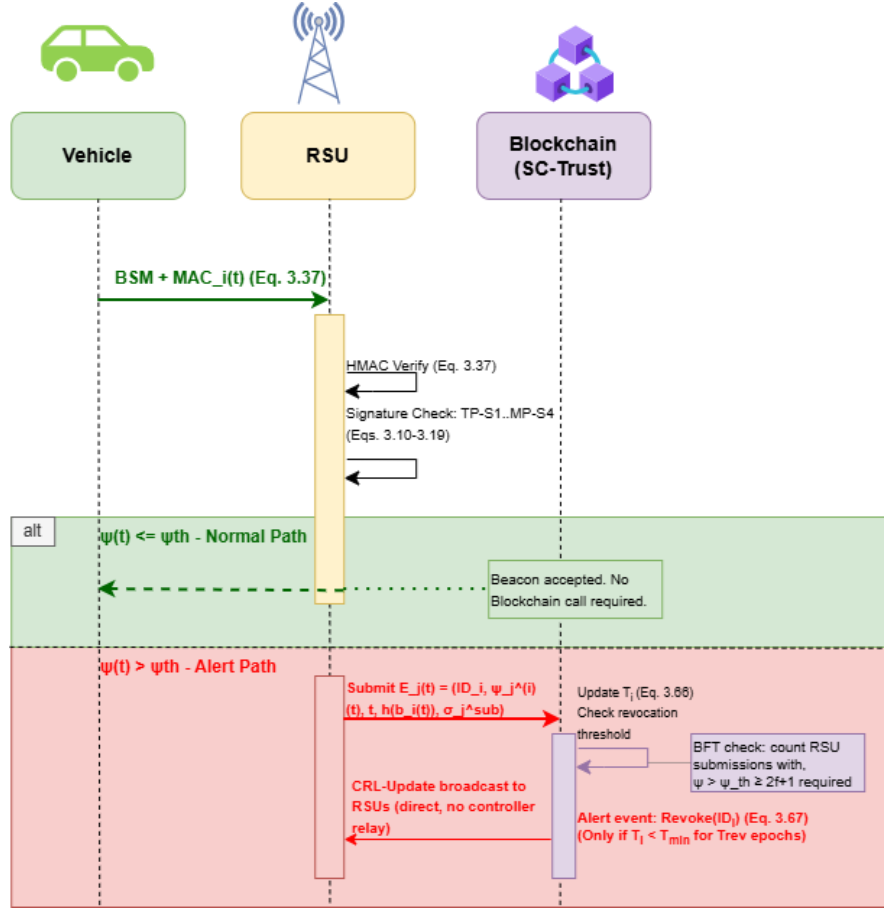


Figure 3.10: UML sequence diagram for lightweight real-time detection across one beacon epoch.

As shown in Figure 3.10, the sequence begins with the vehicle broadcasting $\mathbf{b}_i(t) \parallel \text{MAC}_i(t)$, where the HMAC tag is computed according to Equation 3.42 and binds the beacon payload, timestamp, and vehicle identity under session key K_i . The RSU first verifies this tag; any failure causes immediate rejection before the rule-based signatures are evaluated, forming a fail-fast integrity gate. Beacons that pass verification are then scored against the nine rule-based signatures TP-S1 through MP-S4 (Equations 3.11–3.20), yielding the composite anomaly score $\psi_i(t)$ from Equation 3.21. The diagram’s alt fragment captures the two outcomes of this check. On the normal path ($\psi_i(t) \leq \psi_{th}$), the beacon is accepted locally at the RSU with no blockchain interaction and no controller involvement, keeping latency well within the 100 ms beacon interval T_b . On the alert path ($\psi_i(t) > \psi_{th}$), the RSU submits a signed evidence tuple $\mathcal{E}_j(t)$ directly to **SC-Trust**, which updates the vehicle’s cumulative trust score T_i and evaluates the revocation condition: revocation is issued only if $T_i < T_{min}$ has persisted for T_{rev} epochs and at least $2f + 1$ RSUs have submitted corroborating evidence, satisfying the BFT endorsement condition in Equation 3.69. When both conditions hold, **SC-Trust** emits

Revoke(ID_{*i*}) and broadcasts a CRL update directly to all RSUs, bypassing the controller to ensure revocation propagates even under controller compromise.

3.5.3.1 GAT Architecture Design for Vehicle–RSU Mobility Graphs

Graph Construction. The vehicle–RSU communication graph $\mathcal{G}(t)$ is constructed with a mobility-aware edge definition. An edge $(i, j) \in \mathcal{E}(t)$ is instantiated between nodes v_i and v_j (or v_i and r_j) only when two conditions are jointly satisfied. The mobility-aware edge definition for the vehicle–RSU communication graph is given in Equation 3.27.

This edge definition restricts the GAT’s attention computation to kinematically relevant vehicle pairs those within communication range and traveling in compatible directions preventing the spatial anomaly detector from being confused by oncoming or crossing traffic whose mobility patterns are legitimately dissimilar.

$$(i, j) \in \mathcal{E}(t) \iff \|p_i(t) - p_j(t)\| \leq R_{\max} \wedge \cos^{-1}\left(\frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\| \|\mathbf{v}_j\|}\right) \leq \phi_{\max} \quad (3.27)$$

As shown in Equation 3.27, this edge definition excludes kinematically irrelevant neighbours (e.g., oncoming vehicles on opposite lanes) from the attention computation, reducing noise in the spatial anomaly score where $R_{\max}$ is the maximum V2V/V2I communication range, $\mathbf{v}_i = s_i[\cos \theta_i, \sin \theta_i]^\top$ is the velocity vector of v_i , and $\phi_{\max}$ is a maximum heading divergence threshold.

Node Feature Normalization. BSM fields have heterogeneous scales (GPS coordinates in degrees, speed in m/s, acceleration in m/s²). To prevent scale dominance in the attention computation, the rolling z-score normalization for node features is given in Equation 3.28.

This normalization ensures that GPS coordinates, speed, and acceleration contribute equally to the attention computation regardless of their raw scale differences, preventing larger-magnitude features from dominating the spatial anomaly score and obscuring subtle mobility pattern deviations.

$$\tilde{x}_i^{(f)}(t) = \frac{x_i^{(f)}(t) - \mu_f(t)}{\sigma_f(t) + \epsilon} \quad (3.28)$$

As shown in Equation 3.28, each feature is normalised relative to the current population of vehicles in the RSU zone, making the representation robust to varying traffic densities where $\mu_f(t)$ and $\sigma_f(t)$ are the rolling mean and standard deviation of feature f across all vehicles in $\mathcal{V}_j(t)$, and $\epsilon = 10^{-8}$ prevents division by zero.

Multi-Head Attention. To capture multiple relational aspects of vehicle mobility simultaneously (e.g., speed correlation, heading alignment, proximity), the

normalized attention coefficient computation for the h -th head is given in Equation 3.29.

This attention coefficient determines how much each neighbouring vehicle influences a node's spatial embedding, enabling the GAT to automatically learn which spatial relationships such as platoon spacing or intersection right-of-way are most diagnostic of normal versus poisoned mobility patterns without manual feature engineering.

$$\alpha_{ij}^{(h)} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}_h^\top [\mathbf{W}_h \tilde{\mathbf{x}}_i \| \mathbf{W}_h \tilde{\mathbf{x}}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}_h^\top [\mathbf{W}_h \tilde{\mathbf{x}}_i \| \mathbf{W}_h \tilde{\mathbf{x}}_k]))} \quad (3.29)$$

As shown in Equation 3.29, this formula computes how much attention node i should pay to its neighbour node j during the h -th attention head essentially a relevance score between two vehicles (or a vehicle and an RSU) in the mobility graph.

The multi-head node embedding is given in Equation 3.30 where $\mathbf{W}_h$ is the learnable weight matrix for head h , $\mathbf{a}_h$ is the attention weight vector, and $\mathcal{N}(i)$ is the neighbourhood of node i defined by Equation 3.27. As shown in Equation 3.29, the softmax-normalised attention coefficient $\alpha_{ij}^{(h)}$ measures the relative importance of neighbour j to node i for head h .

$$\mathbf{x}'_i = \left\|_{h=1}^H \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(h)} \mathbf{W}_h \tilde{\mathbf{x}}_j \right) \right. \quad (3.30)$$

As shown in Equation 3.30, the embedding $\mathbf{x}'_i$ aggregates attention-weighted neighbour features from all heads, capturing diverse spatial relationships simultaneously. The final layer uses averaging instead of concatenation to produce a fixed-size output regardless of H where $\|$ denotes concatenation across all H heads.

GAT Training Objective.

The attention weights $\mathbf{W}_h$ and $\mathbf{a}_h$ are trained under K parallel supervised binary cross-entropy tasks, one per attack class, using ground-truth per-attack-type labels obtained directly from the NS-3 attack configuration. For each vehicle v_i in the labeled training node set $\mathcal{V}_{\text{train}}$, the ground-truth label for attack class k is $y_i^{(k)} \in \{0, 1\}$, where $y_i^{(k)} = 1$ denotes that vehicle v_i is executing an attack of type k at the current timestep; $K = 7$ matches the seven attack variants defined in the threat model (Section 3.2). A separate linear classification head maps the shared multi-head embedding $\mathbf{x}'_i$ to a per-attack detection probability. Multi-task training allows the shared encoder to benefit from all K tasks simultaneously, providing each attack-specific head with the relational spatial features it needs while

enriching the shared representation through cross-attack regularisation. The per-attack-type detection probability produced by classification head k is given in Equation 3.31.

This formulation gives the proposed method the same discriminative information per attack class as a specialist baseline trained on that class, while additionally benefiting from the shared encoder trained across all classes simultaneously — an advantage unavailable to per-attack specialist architectures.

This prediction converts the learned spatial embedding into a node-level maliciousness score, enabling the cross-entropy loss to directly supervise the attention weights to separate normal and anomalous spatial graph configurations.

$$\hat{y}_i^{(k)} = \sigma\left(\mathbf{w}_{\text{cls}}^{(k)\top} \mathbf{x}'_i + b_{\text{cls}}^{(k)}\right), \quad k = 1, \dots, K \quad (3.31)$$

As shown in Equation 3.31, each head produces an independent sigmoid probability $\hat{y}_i^{(k)}$ from a linear projection of the shared embedding $\mathbf{x}'_i$, where $\mathbf{w}_{\text{cls}}^{(k)}$ and $b_{\text{cls}}^{(k)}$ are the weight vector and bias of the k -th head. At inference, the predicted attack type is given in Equation 3.32:

$$\hat{k}_i = \arg \max_{k \in \{1, \dots, K\}} \hat{y}_i^{(k)} \quad (3.32)$$

As shown in Equation 3.32, $\hat{k}_i$ identifies the most likely attack class for node v_i , which is used to select the attack-conditioned fusion weights in Equation 3.48. The binary detection flag from the GAT is given in Equation 3.33:

$$\text{flag}_i^{\text{GAT}} = \bigvee_{k=1}^K \mathbf{1}\left[\hat{y}_i^{(k)} > 0.5\right] \quad (3.33)$$

As shown in Equation 3.33, vehicle v_i is flagged by the GAT component if any single attack-type head exceeds the detection threshold; the combined binary detection is strictly at least as sensitive as any individual head.

The multi-task class-weighted binary cross-entropy training loss, summed across all K attack-type heads, is given in Equation 3.34.

Summing K independent tasks trains the shared encoder on the full diversity of attack signatures simultaneously. Per-task class weights w_k^+, w_k^- are computed independently per attack type, preventing rare attack classes from being dominated by more frequent ones.

$$\mathcal{L}_{\text{GAT}} = \sum_{k=1}^K \frac{-1}{|\mathcal{V}_{\text{train}}|} \sum_{v_i \in \mathcal{V}_{\text{train}}} \left[w_k^+ y_i^{(k)} \log \hat{y}_i^{(k)} + w_k^- (1 - y_i^{(k)}) \log(1 - \hat{y}_i^{(k)}) \right] \quad (3.34)$$

As shown in Equation 3.34, $\mathcal{L}_{\text{GAT}}$ is the sum of K independent class-weighted binary cross-entropy losses, where $w_k^+ = |\mathcal{V}_{\text{train}}| / (2|\mathcal{V}_k^+|)$ and $w_k^- = |\mathcal{V}_{\text{train}}| / (2|\mathcal{V}_k^-|)$ are the inverse-frequency class weights for attack type k , with $\mathcal{V}_k^+$ and $\mathcal{V}_k^-$ the positive and negative subsets for class k respectively.

Training proceeds over the first temporal segment of each SUMO trace, run through NS-3 with attack injection active so that ground-truth labels are available from the simulator configuration. After training converges, the classification head $(\mathbf{w}_{\text{cls}}, b_{\text{cls}})$ is discarded and only the encoder weights $\{\mathbf{W}_h, \mathbf{a}_h\}_{h=1}^H$ are retained. The same temporal segment is then re-run through NS-3 with no attack injection to produce clean-condition embeddings, from which the per-node statistics $\bar{\mathbf{x}}'_i$ and σ'_i are computed and locked before evaluation.

3.5.3.2 Temporal Autoencoder Design for Mobility Sequences

LSTM Gating Formulation. The encoder LSTM processes the beacon sequence $\mathbf{X}_i(t) \in \mathbb{R}^{L \times d}$ step by step. At each step l , the LSTM gating equations governing information flow are given in Equations 3.35–3.40.

$$\mathbf{f}_l = \sigma(\mathbf{W}_f[\mathbf{h}_{l-1}, \mathbf{b}_i(t_l)] + \mathbf{c}_f) \quad (3.35)$$

$$\mathbf{u}_l = \sigma(\mathbf{W}_u[\mathbf{h}_{l-1}, \mathbf{b}_i(t_l)] + \mathbf{c}_u) \quad (3.36)$$

$$\mathbf{g}_l = \tanh(\mathbf{W}_g[\mathbf{h}_{l-1}, \mathbf{b}_i(t_l)] + \mathbf{c}_g) \quad (3.37)$$

$$\mathbf{c}_l = \mathbf{f}_l \odot \mathbf{c}_{l-1} + \mathbf{u}_l \odot \mathbf{g}_l \quad (3.38)$$

$$\mathbf{o}_l = \sigma(\mathbf{W}_o[\mathbf{h}_{l-1}, \mathbf{b}_i(t_l)] + \mathbf{c}_o) \quad (3.39)$$

$$\mathbf{h}_l = \mathbf{o}_l \odot \tanh(\mathbf{c}_l) \quad (3.40)$$

As shown in Equation 3.35, the forget gate $\mathbf{f}_l$ determines what portion of the previous cell state is discarded based on the current beacon input. As shown in Equation 3.36, the input gate $\mathbf{u}_l$ controls how much new information is written into the cell. As shown in Equation 3.37, the candidate update $\mathbf{g}_l$ computes the new content to be potentially stored in the cell state. As shown in Equation 3.38, the cell state $\mathbf{c}_l$ is updated by combining the forget-gated previous state with the input-gated candidate, enabling the encoder to retain long-range temporal dependencies across the beacon sequence. As shown in Equation 3.39, the output gate $\mathbf{o}_l$ filters what portion of the cell state is exposed as the hidden state. As shown in Equation 3.40, the hidden state $\mathbf{h}_l$ is produced by applying the output gate to the cell state, forming the temporal representation passed to the next step. The encoder latent representation is the final hidden state: $\mathbf{h}_i(t) = \mathbf{h}_L$ where $\mathbf{f}_l$, $\mathbf{u}_l$, $\mathbf{o}_l$ are the forget, input, and output gates; $\mathbf{c}_l$ is the cell state; $\mathbf{g}_l$ is the candidate cell update; and $\odot$ is element-wise multiplication.

Mobility-Constrained Reconstruction Loss. Standard autoencoders minimize only reconstruction error. Since vehicle mobility follows physical constraints

(Equations 3.11–3.13), the mobility-constrained reconstruction loss used during training is given in Equation 3.41.

This loss function trains the autoencoder to reconstruct only physically plausible mobility sequences by penalizing reconstructions that violate kinematic constraints, ensuring that the learned normal manifold is sensitive to the specific class of mobility-consistent yet collectively falsified sequences that poisoning attacks produce.

$$\mathcal{L}_{\text{total}} = \underbrace{\frac{1}{L} \sum_{l=1}^L \|\mathbf{b}_i(t_l) - \hat{\mathbf{b}}_i(t_l)\|^2}_{\text{reconstruction}} + \beta \underbrace{\frac{1}{L} \sum_{l=2}^L \max(0, r_i(t_l) - \delta_{\text{th}})^2}_{\text{kinematic penalty}} \quad (3.41)$$

As shown in Equation 3.41, the total loss combines standard reconstruction error with a kinematic penalty term, ensuring the autoencoder learns a normal mobility manifold that is physically consistent and sensitive to kinematically implausible reconstructions produced by poisoned sequences where β controls the kinematic penalty weight and $r_i(t_l)$ is the dead-reckoning residual from Equation 3.15.

3.5.3.3 Lightweight Mode: HMAC Beacon Integrity and Rule-Based Detection

Each vehicle v_i appends an HMAC tag to its BSM beacon using session key K_i established during RSU registration. The tag includes a per-vehicle monotonically increasing nonce ν_i alongside the timestamp to prevent replay both outside and within any freshness window. The HMAC beacon integrity tag binding the beacon content, timestamp, identity, and nonce is given in Equation 3.42:

$$\text{MAC}_i(t) = \text{HMAC}_{K_i}(\mathbf{b}_i(t) \parallel t \parallel \text{ID}_i \parallel \nu_i) \quad (3.42)$$

As shown in Equation 3.42, the tag binds the beacon content, timestamp, vehicle identity, and nonce under K_i , so any in-transit modification invalidates the tag. The RSU rejects any beacon whose nonce ν_i is not strictly greater than the last accepted nonce ν_i^{last} for ID_i . At RSU handoff, the RSU retrieves ν_i^{last} from a fast shared RSU-cluster cache passed RSU-to-RSU at handoff, keeping nonce lookup within the 100 ms beacon budget while providing consistent nonce state across zone boundaries. The RSU verifies $\text{MAC}_i(t)$ upon beacon receipt, rejecting any beacon that fails verification before applying the rule-based anomaly score $\psi_i(t)$ from Equation 3.21. An alert is issued if $\psi_i(t) > \psi_{\text{th}}$, and the vehicle’s trust score is immediately decremented on-chain via a lightweight smart contract call.

Algorithm 1 presents the LW-DETECT procedure, which forms the complete lightweight mode detection pipeline executed at the RSU for every incoming beacon within a single 100 ms epoch.

Algorithm 1 Lightweight Rule-Based Beacon Detection (LW-DETECT)

```

1: procedure LW-DETECT( $\mathbf{b}_i(t), \mathbf{b}_i(t-T_b), K_i, \psi_{th}, \mathbf{w}$ )
2:   Verify  $MAC_i(t)$  via Eq. (3.42); if fails then return ALERT (authentication
   failure)
3:   if  $|t_{receive} - t| > \Delta_{HMAC}$  then return ALERT (timestamp stale)
4:   if  $\nu_i \leq \nu_i^{last}$  then return ALERT (nonce replay rejected)
5:    $\nu_i^{last} \leftarrow \nu_i$  (update local cache; push to cluster cache if handoff detected)
6:   Compute signatures TP-S1 to MP-S4 using Eqs. (3.11)–(3.20)
7:    $\psi_i(t) \leftarrow \sum_k w_k \cdot \mathbf{1}[\text{sig. } k \text{ violated}] / \sum_k w_k$  (Eq. (3.21))
8:   if  $\psi_i(t) > \psi_{th}$  then
9:     Buffer  $\mathcal{E}_j(t) \leftarrow (\text{ID}_i, \psi_i(t), t, h(\mathbf{b}_i(t)), \sigma_j^{\text{sub}})$  in local evidence queue; flush to
     SC-Trust per Tier 3 commit policy ( $T_{batch}, N_{commit}$ ); Tier 1 revocation commits
     remain synchronous (Eq. (3.67))
10:    return ALERT( $\text{ID}_i, \psi_i(t)$ )
11:  end if
12:  return PASS
13: end procedure

```

As shown in Algorithm 1, LW-DETECT first verifies the HMAC tag, then evaluates all nine rule-based signatures to produce the composite anomaly score $\psi_i(t)$. If $\psi_i(t) > \psi_{th}$, the RSU calls **SC-Trust.Update** on-chain and raises an alert. Normal beacons are passed with no blockchain interaction, keeping latency within the 100 ms beacon interval T_b . The deliberate ordering of HMAC verification before signature scoring is a fail-fast design decision: any beacon that fails integrity verification is immediately rejected without expending computational resources on the nine signature checks, which is critical for meeting the 100 ms beacon interval constraint T_b at high beacon arrival rates. The absence of blockchain interaction on the normal path is equally deliberate writing to the ledger only on anomaly detection ensures that the lightweight mode scales to dense urban deployments where hundreds of beacons per second may arrive at a single RSU without saturating the consensus layer. The composite threshold ψ_{th} therefore acts as a gating condition that separates routine beacon processing, which is entirely local to the RSU, from the exception path that triggers distributed trust management. TP-DETECT, SYB-DETECT, and MITM-DETECT return violation flag vectors that feed into the composite score $\psi_i(t)$ via Equation 3.21; the sole ALERT decision point is LW-DETECT at $\psi_i(t) > \psi_{th}$, except the HMAC failure path in MITM-DETECT which triggers immediate rejection before composite scoring, since a tag failure is a conclusive indicator of tampering rather than a probabilistic signal requiring further evidence.

Algorithm 2 presents the TP-DETECT procedure, which evaluates the five trajectory poisoning signatures TP-S1 through TP-S5 for each incoming beacon at the RSU.

Algorithm 2 Trajectory Poisoning Detection (TP-DETECT)

```

1: procedure TP-DETECT( $\mathbf{b}_i(t), \mathbf{b}_i(t-T_b), \delta_{\text{th}}, k$ )
2:   Check TP-S1:  $\text{viol}_1 \leftarrow \mathbf{1}[\|p_i(t) - p_i(t-T_b)\| > s_{\text{max}} \cdot T_b]$ 
3:   Check TP-S2:  $\text{viol}_2 \leftarrow \mathbf{1}[|\theta_i(t) - \theta_i(t-T_b)| > \omega_{\text{max}} \cdot T_b]$ 
4:   Check TP-S3:  $\text{viol}_3 \leftarrow \mathbf{1}[|a_i(t)| > a_{\text{max}} \vee s_i(t) > s_{\text{max}}]$ 
5:   Compute  $\hat{p}_i(t)$  via dead-reckoning Eq. (3.14)
6:   Compute residual  $r_i(t) \leftarrow \|p_i(t) - \hat{p}_i(t)\|$  (Eq. (3.15))
7:   Update drift window:  $D_i(k) \leftarrow \frac{1}{k} \sum_{j=1}^k r_i(t-(k-j)T_b)$  (Eq. (3.16))
8:    $\text{viol}_4 \leftarrow \mathbf{1}[D_i(k) > \delta_{\text{th}}]$ 
9:   return  $[\text{viol}_1, \text{viol}_2, \text{viol}_3, \text{viol}_4]$   $\triangleright$  Flag vector fed into  $\psi_i(t)$  in LW-DETECT
10: end procedure

```

As shown in Algorithm 2, the procedure checks kinematic feasibility (TP-S1), heading rate (TP-S2), acceleration bound (TP-S3), and dead-reckoning residual (TP-S4) sequentially for each incoming beacon. Any single violation raises an ALERT-TP with the vehicle identity and evidence flags; otherwise a PASS is returned with no blockchain interaction. The sequential rather than parallel evaluation of the four checks is intentional: TP-S1 through TP-S3 are instantaneous single-beacon tests that can be resolved in microseconds, while TP-S4 and TP-S5 require maintaining a sliding window of past residuals, so early termination on TP-S1 through TP-S3 avoids unnecessary window maintenance for obviously malicious beacons. The cumulative drift score $D_i(k)$ in TP-S5 is the primary detector for the gradual poisoning strategy described in Section 3.4.3, where an adversary stays within per-beacon kinematic bounds but accumulates displacement over the full RSU contact window $N_b \cdot \epsilon_{\text{max}}$. Together, the four violation flags provide complementary coverage: TP-S1 through TP-S3 catch aggressive single-beacon falsifications, while TP-S5 catches the slow drift attacks that individually valid beacons enable.

Algorithm 3 presents the SYB-DETECT procedure, which detects Sybil-based mobility pattern poisoning by checking identity density, beacon synchronisation, and ghost transit impossibility across the RSU coverage zone.

Algorithm 3 Sybil-Based Mobility Pattern Poisoning Detection (SYB-DETECT)

```

1: procedure SYB-DETECT( $\mathcal{B}_j(t), \mathcal{A}_j, K_{\text{sybil}}, \tau_{\text{sync}}, \rho_{\text{sync}}$ )
2:   Extract identity set  $\mathcal{ID}_j \leftarrow \{\text{ID}_i : p_i(t) \in \mathcal{A}_j\}$ 
3:   Check MP-S1:  $\text{viol}_1 \leftarrow \mathbf{1}[|\mathcal{ID}_j| > K_{\text{sybil}} + \rho_v \cdot \mathcal{A}_j]$  (Eq. (3.17))
4:   for each pair  $(\text{ID}_a, \text{ID}_b) \in \mathcal{ID}_j^2$  do
5:     Compute synchronization score via Eq. (3.18)
6:     if score  $> \rho_{\text{sync}}$  then
7:        $\text{viol}_2 \leftarrow 1$ ; flag pair  $(\text{ID}_a, \text{ID}_b)$  as co-sourced
8:     end if
9:   end for
10:  Check MP-S4 for each  $\text{ID}_i$ : ghost transit test via Eq. (3.20) across neighbouring RSUs
11:   $\text{viol}_3 \leftarrow \mathbf{1}[\exists \text{ID}_i \text{ failing ghost test}]$ 
12:  return  $[\text{viol}_1, \text{viol}_2, \text{viol}_3]$   $\triangleright$  Flag vector fed into  $\psi_i(t)$  in LW-DETECT
13: end procedure

```

As shown in Algorithm 3, the procedure first extracts the active identity set within the RSU coverage zone, then evaluates identity density anomaly (MP-S1), pairwise beacon synchronisation score (MP-S2), and ghost vehicle transit impossibility (MP-S4). A single positive flag triggers an ALERT-SYB submitted directly to the blockchain. The pairwise synchronisation loop in lines 4–9 has quadratic complexity in the number of identities, which is bounded by $K_{\text{sybil}} + \rho_v \cdot \mathcal{A}_j$ from Equation 3.8; in practice this remains tractable because the MP-S1 check in line 3 filters the identity set before the loop executes, meaning the loop only runs when the identity count is already suspicious. The ghost transit test in line 10 requires inter-RSU coordination to obtain appearance timestamps from neighbouring RSUs, which is handled through the shared blockchain ledger rather than direct RSU-to-RSU messaging, ensuring that the test remains consistent even when some neighbouring RSUs are offline or compromised. The three checks together address distinct Sybil attack strategies: MP-S1 targets volume-based Sybil injection, MP-S2 targets timing-correlated co-sourced identities, and MP-S4 targets teleporting ghost identities that no single-RSU check can detect.

Algorithm 4 presents the MITM-DETECT procedure, which verifies HMAC beacon integrity as its first gate, followed by a regional speed distribution shift check (MP-S3) to detect MitM-injected mobility data.

Algorithm 4 MitM Mobility Pattern Poisoning Detection (MITM-DETECT)

```
1: procedure MITM-DETECT( $\mathbf{b}_i(t), K_i, \kappa_{\text{th}}, \mathcal{P}_{\text{hist}}$ )
2:   Verify HMAC:  $\text{valid} \leftarrow \text{HMAC}_{K_i}(\mathbf{b}_i(t) \| t \| \text{ID}_i)$  (Eq. (3.42))
3:   if  $\neg \text{valid}$  then
4:     return ALERT_MITM( $\text{ID}_i, \text{MAC\_FAIL}$ )
5:   end if
6:   Compute current regional speed distribution  $\mathcal{P}_t$  from  $\{s_i(t)\}_{v_i \in \mathcal{V}_j}$ 
7:   Check MP-S3:  $\text{viol}_1 \leftarrow \mathbf{1}[D_{\text{KL}}(\mathcal{P}_t \| \mathcal{P}_{\text{hist}}) > \kappa_{\text{th}}]$  (Eq. (3.19))
8:   Run TP-S4 residual check on modified fields via Eq. (3.15)
9:    $\text{viol}_2 \leftarrow \mathbf{1}[r_i(t) > \delta_{\text{th}}]$ 
10:  return  $[\text{viol}_1, \text{viol}_2]$   $\triangleright$  MP-S3 and TP-S4 flags fed into  $\psi_i(t)$ ; HMAC failure
    above is the only immediate ALERT here
11: end procedure
```

As shown in Algorithm 4, the HMAC verification step acts as a hard gate — any tag failure immediately returns ALERT-MITM without further processing. Only HMAC-valid beacons proceed to the KL-divergence distribution check and dead-reckoning residual test, ensuring in-transit modifications are detected before any rule-based scoring is applied. The hard-gate design is justified by the MitM threat model: an in-transit attacker who modifies beacon content necessarily invalidates the HMAC tag computed by the originating vehicle under its session key K_i , so a tag failure is a conclusive indicator of tampering rather than a probabilistic signal requiring further evidence. The KL-divergence check MP-S3 in line 7 serves a complementary role for the case where the MitM attacker replays unmodified beacons from a different region rather than modifying individual fields, which preserves per-beacon HMAC validity but shifts the regional speed distribution away from the historical baseline $\mathcal{P}_{\text{hist}}$. The dead-reckoning residual check in line 9 then catches field-level modifications that are individually plausible but inconsistent with the vehicle’s previously reported kinematic state, providing a three-layer detection structure that covers the full space of MitM attack strategies.

3.5.3.4 Full Mode: GAT-Based Spatial Anomaly Detection

The vehicle–RSU communication structure is modelled as a dynamic graph $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t))$ using the mobility-aware edge definition given in Equation 3.27, the rolling z-score normalisation given in Equation 3.28, and the multi-head attention computation given in Equations 3.29 and 3.30, as defined in Section 3.5.3.1. The spatial anomaly score measuring node i ’s deviation from expected normal conditions is given in Equation 3.43.

$$\mathcal{S}_i(t) = \left\| \frac{\mathbf{x}'_i(t) - \bar{\mathbf{x}}'_i}{\boldsymbol{\sigma}'_i} \right\|_2 \quad (3.43)$$

As shown in Equation 3.43, $\mathcal{S}_i(t)$ measures how far the current spatial embedding deviates from the normal embedding distribution, with high values flagging

spatially anomalous vehicles, where $\bar{\mathbf{x}}'_i$ and $\boldsymbol{\sigma}'_i$ are the per-node mean and standard deviation computed from clean attack-free embeddings produced after GAT weights are locked, and are not updated during evaluation.

3.5.3.5 Full Mode: Temporal Autoencoder Sequential Anomaly Detection

The sliding window input sequence of L consecutive beacons for the temporal autoencoder is given in Equation 3.44.

This input formulation presents the beacon sequence as a time-series matrix to the LSTM encoder, enabling the temporal autoencoder to learn normal trajectory dynamics across a configurable observation window whose length L is set to satisfy the detection feasibility condition in Equation 3.4.

$$\mathbf{X}_i(t) = [\mathbf{b}_i(t - L \cdot T_b), \dots, \mathbf{b}_i(t)] \in \mathbb{R}^{L \times d} \quad (3.44)$$

As shown in Equation 3.44, the input captures the recent trajectory of vehicle v_i as a time-series matrix of L consecutive beacons. The LSTM encoder processes $\mathbf{X}_i(t)$ step by step using the gating equations defined in Equations 3.35–3.40, producing the latent representation $\mathbf{h}_i(t) = \mathbf{h}_L$.

The decoder reconstruction of the full input sequence from this latent representation is given in Equation 3.45. This reconstruction step converts the compressed latent representation back into a full beacon sequence, enabling anomaly detection by reconstruction error: sequences that deviate from learned normal mobility patterns produce higher errors than legitimate trajectories.

$$\hat{\mathbf{X}}_i(t) = \text{LSTM}_{\text{dec}}(\mathbf{h}_i(t)) \quad (3.45)$$

As shown in Equation 3.45, the decoder reconstructs the full beacon sequence from the compressed latent representation, enabling comparison between reported and expected mobility behaviour. The autoencoder is trained using the mobility-constrained reconstruction loss defined in Equation 3.41.

At inference, the per-window reconstruction error is given in Equation 3.46. This error metric quantifies how well the autoencoder can reproduce a given beacon sequence from its latent representation, serving as the primary anomaly signal for the temporal component: poisoned sequences that lie off the learned normal manifold produce systematically higher reconstruction errors than genuine trajectories.

$$\varepsilon_i(t) = \frac{1}{L} \sum_{l=1}^L \|\mathbf{b}_i(t_l) - \hat{\mathbf{b}}_i(t_l)\|^2 \quad (3.46)$$

As shown in Equation 3.46, $\varepsilon_i(t)$ measures the average per-step reconstruction error over the beacon window, with higher values indicating sequences that deviate

from the learned normal mobility manifold.

To prevent an attacker from drifting the detection baseline by slowly poisoning the live beacon stream, the detection threshold is calibrated entirely from offline reconstruction errors computed on SUMO-generated clean mobility traces, using robust statistics resistant to outlier contamination. The robust detection threshold using median and Median Absolute Deviation (MAD) of offline clean-data reconstruction errors is given in Equation 3.47:

$$\theta_{\text{ae}} = \text{median}(\mathcal{E}_{\text{clean}}) + \kappa \cdot \text{MAD}(\mathcal{E}_{\text{clean}}) \quad (3.47)$$

As shown in Equation 3.47, the threshold is fixed from clean data before deployment and is not updated from live inputs where $\mathcal{E}_{\text{clean}} = \{\varepsilon_i^{\text{clean}}\}$ is the offline reconstruction error set and $\text{MAD}(\mathcal{E}) = \text{median}(|\varepsilon - \text{median}(\mathcal{E})|)$. The autoencoder’s normal mobility manifold is similarly trained offline on SUMO traces and frozen before deployment. Online model updates are suspended whenever the fraction of flagged vehicles in any RSU zone exceeds a configurable freeze threshold α_{freeze} , preventing adversarial corruption of the learned manifold during active attacks.

3.5.3.6 Combined Detection Score and Fusion

In the full mode, the combined detection score fusing rule-based, spatial, and temporal anomaly signals is given in Equation 3.48.

This fusion equation combines the complementary strengths of the three detection components, rule-based kinematic checks, GAT spatial relational analysis, and LSTM temporal sequential analysis into a single decision variable, ensuring that attacks evading any one component are still detected by the remaining two. By normalising $\psi_i(t)$ by the rule-based alert threshold ψ_{th} , $\mathcal{S}_i(t)$ by the clean-data spatial calibration threshold $\theta_{\mathcal{S}}$, and $\varepsilon_i(t)$ by the clean-data reconstruction threshold θ_{ae} , each term reaches exactly 1.0 when the component alone attains its independent single-mechanism alerting level. Each normalised term is then clipped at 1.0 before summation. Since $\lambda_1 + \lambda_2 + \lambda_3 = 1$, this clipping makes $\Phi_i(t)$ a genuine weighted average in $[0, 1]$, giving $\Phi_{\text{th}} = 0.5$ the principled interpretation of at least half the weighted evidence signals attack. Without this clip, a single component with a large raw value dominates the sum, rendering Φ_{th} an arbitrary threshold on an unbounded quantity. Without normalising $\psi_i(t)$, the raw rule score is bounded well below Φ_{th} regardless of the learned weight λ_1 , preventing the rule term from contributing to the fusion decision even when lightweight detection succeeds.

$$\Phi_i(t) = \lambda_1^{(\hat{k}_i)} \min\left(\frac{\bar{\psi}_i(t)}{\psi_{\text{th}}}, 1\right) + \lambda_2^{(\hat{k}_i)} \min\left(\frac{\mathcal{S}_i(t)}{\theta_{\mathcal{S}}}, 1\right) + \lambda_3^{(\hat{k}_i)} \min\left(\frac{\varepsilon_i(t)}{\theta_{\text{ae}}}, 1\right) \quad (3.48)$$

As shown in Equation 3.48, $\Phi_i(t) \in [0, 1]$ is a genuine weighted average of three clipped normalised components, where θ_S is locked from a high percentile of $\mathcal{S}_i(t)$ computed over clean Phase 1b data. Here, $\bar{\psi}_i(t) = \frac{1}{L} \sum_{l=1}^L \psi_i(t_l)$ is the mean composite rule score across the L -beacon detection window, providing temporal consistency with the window-level LSTM-AE score $\varepsilon_i(t)$. When the per-window mean rule score satisfies $\bar{\psi}_i(t) \geq \psi_{\text{th}}$, the normalised rule term reaches its cap of 1.0, contributing λ_1 to $\Phi_i(t)$; if $\lambda_1 > \Phi_{\text{th}}$, the rule signal alone is sufficient to trigger a full-mode alert, ensuring the full mode subsumes the lightweight mode for sustained attacks. An additional feasibility constraint $\lambda_3^{(k)} < \lambda_1^{(k)} + \lambda_2^{(k)}$ is enforced during the Phase 2 SLSQP optimisation for every attack class k , preventing the LSTM-AE weight from dominating the fusion outright — a pathology in which a silent, untriggered autoencoder component could still cap $\Phi_i(t)$ below Φ_{th} for classes where the reconstruction error is not discriminative. This constraint was found to be necessary to resolve an impossible detection ceiling observed on attack classes a1 and a5.

The binary detection decision is given in Equation 3.49 where $\lambda_1^{(k)} + \lambda_2^{(k)} + \lambda_3^{(k)} = 1$ and each $\lambda_j^{(k)} \geq 0.05$ for all j, k ; K independent weight sets are learned in Phase 2, one per attack class, by partitioning the Phase 2 labelled validation segment by ground-truth attack type and running K constrained SLSQP optimisations in parallel. At inference, $\hat{k}_i$ from Equation 3.32 selects the weight set that best matches the predicted attack class, so each detector component is weighted according to its expected discriminative power for that specific attack type rather than an average across all attacks.

This binary decision converts the continuous fused anomaly score into an actionable flag that triggers the on-chain trust update and revocation pipeline, translating the detection output into a concrete network management action without requiring manual operator intervention.

$$\text{flag}_i(t) = \mathbf{1}[\Phi_i(t) > \Phi_{\text{th}}] \vee \text{flag}_i^{\text{GAT}} \quad (3.49)$$

As shown in Equation 3.49, a vehicle is flagged as malicious when either the fused anomaly score exceeds Φ_{th} or any individual GAT attack-type head independently fires ($\text{flag}_i^{\text{GAT}}$, Eq. 3.33), triggering the on-chain trust update and revocation pipeline. The OR condition prevents a case where a single head is confidently correct but its signal is diluted below Φ_{th} by the weighted-average fusion.

When the GAT attack-type prediction confidence is low, committing to the wrong weight set actively degrades detection. A per-head confidence gate is therefore applied: if $\hat{y}_i^{(\hat{k}_i)} < \theta_{\text{conf}}^{(\hat{k}_i)}$, the fusion falls back to the unweighted average weight set $\bar{\lambda}_j = \frac{1}{K} \sum_{k=1}^K \lambda_j^{(k)}$ rather than selecting the argmax weight set. Each $\theta_{\text{conf}}^{(k)}$ is calibrated independently per attack-type head on the Phase 2 validation set to maximise MCC, since different heads exhibit different confidence calibration.

Algorithm 5 presents the FULL-DETECT procedure, which integrates GAT spatial anomaly detection, LSTM temporal autoencoder scoring, and rule-based signatures into the complete full-mode detection pipeline executed at the SDVN controller per sliding window.

Algorithm 5 Full-Mode Spatio-Temporal Detection (FULL-DETECT)

```

1: procedure FULL-DETECT( $\mathcal{G}(t), \{\mathbf{X}_i(t)\}, \Phi_{\text{th}}, \{\boldsymbol{\lambda}^{(k)}\}_{k=1}^K$ )
2:   Retrieve beacon windows  $\{\mathbf{X}_i(t)\}$  from IPFS using on-chain hashes
3:   for each  $v_i \in \mathcal{V}(t)$  do
4:     Compute  $\psi_i(t_l)$  via LW-DETECT for each  $l \in \{1, \dots, L\}$ ;  $\bar{\psi}_i(t) \leftarrow \frac{1}{L} \sum_{l=1}^L \psi_i(t_l)$ 
       $\triangleright$  per-window mean rule score
5:     Compute GAT embedding  $\mathbf{x}'_i(t)$  via Eqs. (3.29)–(3.30)
6:     Compute  $\hat{y}_i^{(k)}$  for  $k = 1, \dots, K$  via Eq. (3.31);  $\hat{k}_i \leftarrow \arg \max_k \hat{y}_i^{(k)}$ 
      (Eq. (3.32))
7:     Compute spatial score  $\mathcal{S}_i(t)$  via Eq. (3.43)
8:     Encode  $\mathbf{h}_i(t) \leftarrow \text{LSTM}_{\text{enc}}(\mathbf{X}_i(t))$  (Eqs. (3.35)–(3.40))
9:     Reconstruct  $\hat{\mathbf{X}}_i(t) \leftarrow \text{LSTM}_{\text{dec}}(\mathbf{h}_i(t))$  (Eq. (3.45))
10:    Compute  $\varepsilon_i(t)$  via Eq. (3.46)
11:     $\Phi_i(t) \leftarrow \lambda_1^{(\hat{k}_i)} \min(\bar{\psi}_i/\psi_{\text{th}}, 1) + \lambda_2^{(\hat{k}_i)} \min(\mathcal{S}_i/\theta_{\mathcal{S}}, 1) + \lambda_3^{(\hat{k}_i)} \min(\varepsilon_i/\theta_{\text{ae}}, 1)$ 
12:     $\text{flag}_i(t) \leftarrow \mathbf{1}[\Phi_i(t) > \Phi_{\text{th}}] \vee \text{flag}_i^{\text{GAT}}$   $\triangleright$  OR-fused decision; fixes GAT-flag
      wiring gap (Eq. (3.49))  $\triangleright$  attack-conditioned weights
      selected by  $\hat{k}_i$ ; clipped weighted average in  $[0, 1]$ ;  $\psi_{\text{th}}, \theta_{\mathcal{S}}, \theta_{\text{ae}}$  fixed offline;  $\Phi_{\text{th}} = 0.5$ 
      is the evidence midpoint (Eq. (3.48))
13:    Submit  $\mathcal{E}_c(t) \leftarrow (\text{ID}_i, \Phi_i(t), t, h(\mathbf{X}_i(t)), \sigma_c^{\text{sub}})$  to blockchain as untrusted peer
      (Eq. (3.68))  $\triangleright$  SC-Trust and SC-Revoke decisions driven by RSU evidence only
14:  end for
15:  return  $\{\text{flag}_i(t)\}$ 
16: end procedure

```

As shown in Algorithm 5, for each vehicle the procedure computes the rule-based score via a LW-DETECT sub-call, the GAT spatial embedding, and the LSTM reconstruction error, then fuses them into $\Phi_i(t)$. The controller submits its result to the blockchain as one untrusted peer vote; all SC-Trust and SC-Revoke decisions remain driven by RSU evidence only. The IPFS retrieval step in line 2 is architecturally significant: by storing beacon windows off-chain with only their content hashes recorded on the ledger, the framework bridges the real-time RSU-side lightweight detection and the offline controller-side deep learning analysis without requiring the controller to maintain a direct connection to each RSU, and enables reanalysis of historical windows on demand if detection thresholds are recalibrated. The controller’s submission being treated as one untrusted peer vote rather than a privileged input reflects the threat model assumption in Section 3.2 that the controller itself may be compromised: a malicious controller that submits a falsified $\Phi_i(t)$ score is outvoted by the $2f + 1$ honest RSU submissions required by Equation 3.69. The per-vehicle loop scales linearly with the number of vehicles

in $\mathcal{V}(t)$, and in practice the GAT forward pass over the full vehicle–RSU graph is the dominant computational cost, motivating the sliding window update period $W = L \cdot T_b$ rather than per-beacon execution at the controller.

3.5.4 Full Mode Mitigation: Post-Quantum TRS and FHE Aggregate Protection

Figure 3.11 presents the corresponding UML sequence diagram for one full-mode detection epoch. .

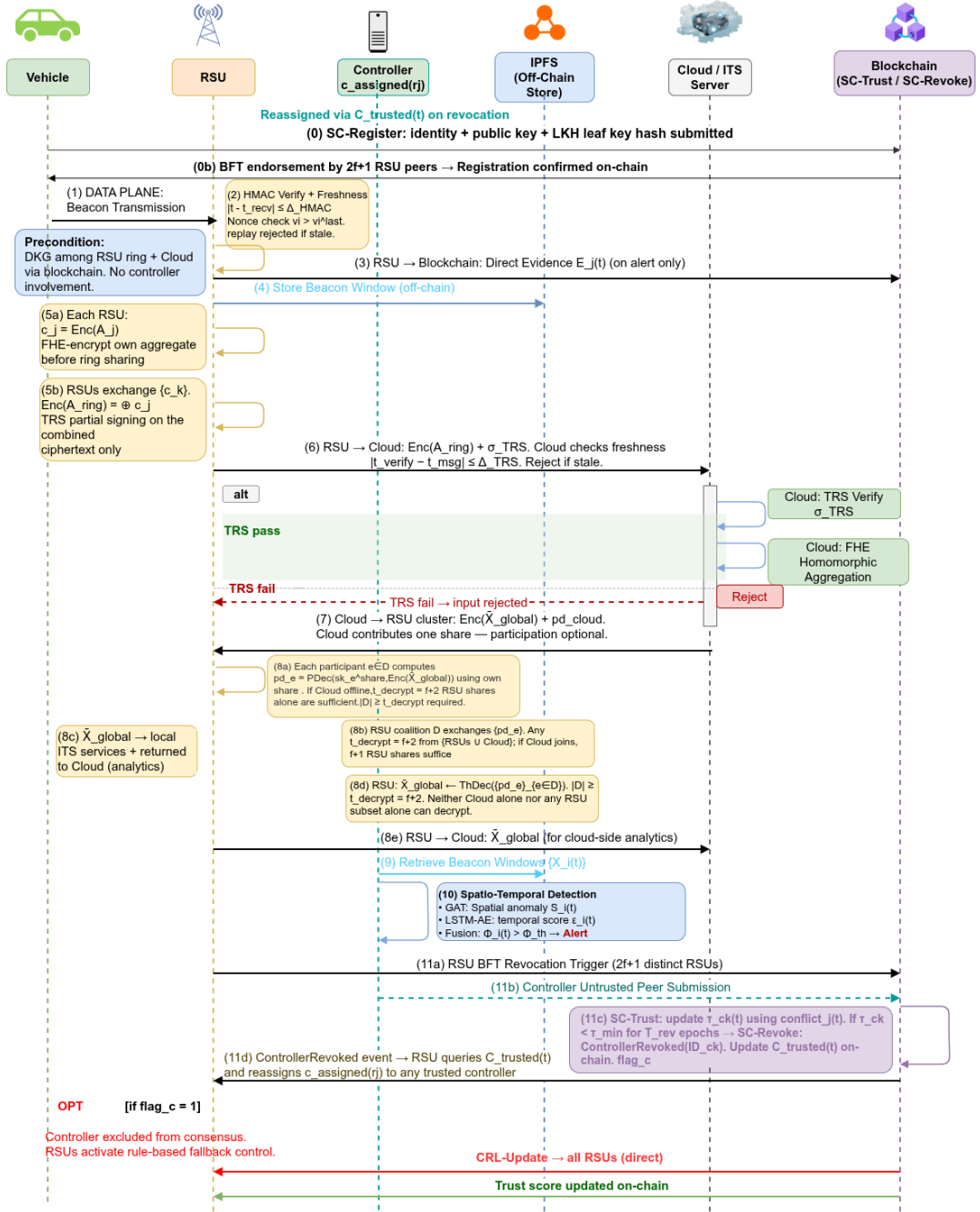


Figure 3.11: UML sequence diagram for the full operating mode across one detection epoch. Five lifelines

While Figure 3.10 captures the per-beacon timing of the lightweight path at a single RSU, the full operating mode spans five distinct entities and combines real-time RSU-side processing with offline controller-side deep learning inference. As shown in Figure 3.11, five lifelines—Vehicle, RSU, Controller $c_{assigned}(r_j)$, IPFS off-chain store, Cloud/ITS Server, and the Blockchain (SC-Trust / SC-Revoke)—

participate across a single detection epoch. As a precondition, the threshold signing keys and FHE key shares are established once by distributed key generation among the RSU ring and the Cloud (Equations 3.25 and 3.26), with no controller involvement; each entity then registers via SC-Register and is confirmed on-chain only after BFT endorsement by $2f+1$ RSU peers.

The epoch begins on the data plane with the vehicle broadcasting its beacon, which the RSU first authenticates through the HMAC tag, freshness window, and monotonic anti-replay nonce (Equation 3.42); stale or replayed beacons are dropped at this fail-fast gate. The RSU evaluates the nine rule-based signatures and the composite score $\psi_i(t)$ (LW-DETECT); a direct evidence tuple $\mathcal{E}_j(t)$ is submitted to the blockchain only on the alert path (Equation 3.67), preserving the lightweight skip-on-pass behaviour, while the beacon window itself is stored off-chain in IPFS with only its hash retained on-chain.

The privacy-preserving aggregation path then executes in the order mandated by the threat model. Each RSU first FHE-encrypts its own local aggregate, $c_j = \text{Enc}(\mathbf{A}_j)$ (Equations 3.50–3.51), before any ring sharing (step 5a). The ring members then exchange ciphertexts and homomorphically combine them into $\text{Enc}(\mathbf{A}_{\text{ring}}) = \bigoplus_j c_j$ (Equation 3.52), and the post-quantum threshold ring signature is produced over the combined ciphertext only (Equations 3.53–3.55, step 5b)—so no plaintext mobility data ever crosses the ring. The signed aggregate $(\text{Enc}(\mathbf{A}_{\text{ring}}), \sigma_{\text{TRS}})$ is sent to the Cloud, which first checks message freshness (Equation 3.57) and then verifies the threshold ring signature (Equation 3.56). This verification is a hard gate: any aggregate failing it is rejected before homomorphic computation begins, and only a passing aggregate proceeds to global homomorphic aggregation (Equation 3.58).

Recovering the plaintext result is itself a threshold operation. The Cloud contributes one partial decryption (Equation 3.59), but its participation is optional; each member of the decryption coalition $\mathcal{D}$ computes its own partial using its key share (Equation 3.61), and the global aggregate is reconstructed by threshold decryption only when $|\mathcal{D}| \geq t_{\text{decrypt}} = f+2$ (Equation 3.62). Consequently neither the Cloud alone nor any sub-threshold subset of RSUs can decrypt unilaterally; if the Cloud is offline, $f+2$ RSU shares suffice on their own. The decrypted aggregate $\bar{X}_{\text{global}}$ is delivered to local ITS services and returned to the Cloud for analytics.

The AI detection path runs at the assigned controller. It retrieves the beacon windows $\{\mathbf{X}_i(t)\}$ from IPFS, computes the GAT spatial anomaly score $\mathcal{S}_i(t)$ (Equation 3.43) and the LSTM-autoencoder temporal reconstruction error $\varepsilon_i(t)$ (Equation 3.46), and fuses them into $\Phi_i(t)$; an alert is raised when $\Phi_i(t) > \Phi_{\text{th}}$ (Equations 3.48–3.49). Crucially, the controller submits its result to the blockchain as a single untrusted peer vote (Equation 3.68): revocation is driven by RSU evidence, requiring $2f+1$ distinct RSU submissions over the temporal window (Equation 3.69), not by the controller alone.

Finally, SC-Trust updates the controller trust score $\tau_{c_k}(t)$ using the directional

conflict indicator $\text{conflict}_j(t)$ (Equations 3.64 and 3.72); if τ_{c_k} remains below $\tau_{\min}$ for T_{rev} epochs, SC-Revoke issues **ControllerRevoked** and updates the trusted-controller set $\mathcal{C}_{\text{trusted}}(t)$ on-chain (Equation 3.1), prompting affected RSUs to re-assign $c_{\text{assigned}}(r_j)$ to any trusted controller. In the optional CP-DETECT branch, when the controller-compromise flag is set (Equation 3.73) the controller is excluded from consensus and the RSUs fall back to rule-based control (LW-DETECT); the resulting CRL update is broadcast directly to all RSUs, bypassing the controller, and the on-chain trust scores are updated (Equation 3.63).

3.5.4.1 RSU Aggregate Construction and Pre-Coordination FHE Encryption

Each RSU r_j first computes a local plaintext aggregate from the beacon messages received within its own coverage zone $\mathcal{V}_j(t)$. The local plaintext aggregate computed by RSU r_j from its own coverage zone is shown in Equation (3.50):

$$\mathbf{A}_j(t) = \frac{1}{|\mathcal{V}_j(t)|} \sum_{v_i \in \mathcal{V}_j(t)} \mathbf{b}_i(t) \quad (3.50)$$

As shown in Equation (3.50), the aggregate represents the field-wise mean of beacon values across all vehicles in the RSU's own coverage zone. Since RSU r_j legitimately serves $\mathcal{V}_j(t)$, computing this aggregate in plaintext introduces no privacy concern at this stage.

The privacy threat arises during TRS ring coordination, where t -of- n neighboring RSUs must collaborate to sign a combined aggregate, exposing each RSU's aggregate to all ring participants. To prevent cross-RSU aggregate leakage, each RSU encrypts its own local aggregate using FHE before sharing it with the ring. The FHE encryption of the local aggregate by RSU r_j prior to ring sharing is shown in Equation (3.51):

$$c_j = \text{Enc}(pk_{\text{FHE}}, \mathbf{A}_j(t)) \quad (3.51)$$

As shown in Equation (3.51), each RSU independently encrypts its local aggregate before any inter-RSU sharing occurs, ensuring that no ring participant receives another participant's plaintext aggregate at any point during coordination.

Upon receiving encrypted aggregates from all t signing ring members, each RSU performs homomorphic addition across the received ciphertexts to produce a combined encrypted aggregate without decryption. The combined encrypted aggregate produced by homomorphic addition across the signing ring is shown in Equation (3.52):

$$\text{Enc}(\mathbf{A}_{\text{ring}}) = \bigoplus_{j \in \mathcal{S}} c_j \quad (3.52)$$

As shown in Equation (3.52), the combined aggregate is produced entirely on ciphertext no RSU decrypts any other RSU's contribution at any point during this computation.

3.5.4.2 Post-Quantum Threshold Ring Signature on Combined Encrypted Aggregate

The TRS is applied to the combined encrypted aggregate rather than any individual RSU's plaintext data. The message committed to the TRS signing process binds the combined encrypted aggregate to the signing ring identity, timestamp, and a per-ring monotonically increasing nonce ν_S to prevent replay of valid signatures within any freshness window. The TRS signed message incorporating timestamp and nonce for full replay protection is shown in Equation (3.53):

$$m = (\text{Enc}(\mathbf{A}_{\text{ring}}), t, \nu_S, \text{ID}_S, h(\mathcal{S})) \quad (3.53)$$

As shown in Equation (3.53), the signed message contains the combined ciphertext, timestamp, nonce, signing ring identity, and a hash of the signing member set — no plaintext aggregate content is exposed during signing. The cloud verifier checks that ν_S strictly exceeds the last accepted nonce for ring $\mathcal{S}$, in addition to the timestamp freshness check in Eq. (3.57). Each RSU r_j in the signing subset $\mathcal{S}$ generates a partial signature over m using its lattice-based Dilithium secret key. The partial signature generated by each ring member is shown in Equation (3.54):

$$\sigma_j = \text{Sign}(sk_j, m), \quad r_j \in \mathcal{S} \quad (3.54)$$

As shown in Equation (3.54), each partial signature is computed independently without requiring access to any other RSU's plaintext data. The partial signatures are then aggregated into a single compact threshold ring signature. The aggregated threshold ring signature combining all partial signatures is shown in Equation (3.55):

$$\sigma_{\text{TRS}} = \text{Aggregate}(\{\sigma_j\}_{j \in \mathcal{S}}) \quad (3.55)$$

As shown in Equation (3.55), the aggregate signature is compact and conceals which specific t members signed. Verification at the cloud or SDN controller confirms that at least t legitimate RSUs endorsed the combined encrypted aggregate. The TRS verification condition at the cloud is shown in Equation (3.56):

$$\text{Verify}(\{pk_j\}_{j=1}^n, m, \sigma_{\text{TRS}}) \in \{0, 1\} \quad (3.56)$$

As shown in Equation (3.56), a result of 0 indicates that fewer than t_{sign} honest RSUs endorsed the combined aggregate, causing the cloud to reject the input as potentially poisoned.

To prevent replay of valid but stale aggregate submissions, the verifier enforces a freshness window Δ_{TRS} on the timestamp t_m embedded in m . The TRS freshness acceptance condition rejecting stale submissions regardless of signature validity is shown in Equation (3.57):

$$|t_{\text{verify}} - t_m| \leq \Delta_{\text{TRS}} \quad (3.57)$$

As shown in Equation (3.57), any $(\text{Enc}(\mathbf{A}_{\text{ring}}), \sigma_{\text{TRS}})$ pair whose timestamp falls outside the acceptance window is rejected at the cloud, preventing an attacker from replaying a previously valid aggregate to re-inject stale poisoned data.

3.5.4.3 Cloud-Side Blind Homomorphic Computation and Result Decryption

Following TRS verification, the cloud receives combined encrypted aggregates from multiple RSU signing rings and performs further homomorphic computations across them without decryption. The global encrypted result computed homomorphically across M ring aggregates at the cloud is shown in Equation (3.58):

$$\text{Enc}(\bar{X}_{\text{global}}) = \frac{1}{M} \bigoplus_{k=1}^M \text{Enc}(\mathbf{A}_{\text{ring},k}) \quad (3.58)$$

As shown in Equation (3.58), the cloud computes the global aggregate entirely on ciphertext – no individual RSU aggregate is ever decrypted at the cloud, preserving each RSU’s aggregate privacy throughout cloud-side computation. Since the Cloud holds only one threshold share and cannot decrypt independently, the Cloud returns $\text{Enc}(\bar{X}_{\text{global}})$ to the originating RSU cluster for cooperative threshold decryption. The Cloud computes its own partial decryption using its key share as one of the required t cooperating participants. The partial decryption computed by the Cloud using its share is shown in Equation (3.59):

$$pd_{\text{cloud}} = \text{PDec}(sk_{\text{cloud}}^{\text{share}}, \text{Enc}(\bar{X}_{\text{global}})) \quad (3.59)$$

As shown in Equation (3.59), the Cloud’s partial decryption contribution is computed using only its own share and transmitted to the RSU cluster along with $\text{Enc}(\bar{X}_{\text{global}})$. No entity decrypts the result independently at any stage.

3.5.4.4 Threshold FHE Decryption at RSU Cluster

To enable RSUs to locally recover the global result without any single RSU holding the complete FHE secret key which would allow that RSU to unilaterally decrypt other RSUs’ encrypted aggregates received during ring coordination the FHE secret key is split into n shares across all RSU ring members under a (t, n) threshold secret sharing scheme. Since the Cloud belongs to the control plane threat boundary (Section 3.2), it must not hold unilateral decryption capability. The FHE secret key is therefore split into $n + 1$ shares across all n RSU ring members and the Cloud, under a $(t, n + 1)$ threshold secret sharing scheme. The threshold FHE key generation producing a shared public key, n RSU key shares, one Cloud key share, and an evaluation key is shown in Equation (3.60):

$$(pk_{\text{FHE}}, \{sk_j^{\text{share}}\}_{j=1}^n, sk_{\text{cloud}}^{\text{share}}, evk) \leftarrow \text{ThGen}(1^\lambda, n + 1, t_{\text{decrypt}}) \quad (3.60)$$

As shown in Equation (3.60), neither any individual RSU nor the Cloud holds the full secret key sk_{FHE} . Each entity holds exactly one share, which is computationally insufficient for decryption alone. Homomorphic aggregation operations at the Cloud use only the evaluation key evk and require no secret key, so removing the

Cloud's full key does not affect its ability to perform homomorphic computations across ring aggregates.

Upon receiving $\text{Enc}(\bar{X}_{\text{global}})$ and pd_{cloud} from the Cloud (when the Cloud participates), each RSU r_j in the decryption coalition $\mathcal{D}$ computes a partial decryption using its own key share. When the Cloud participates, it counts as one of the required t_{decrypt} shares, so only $t_{\text{decrypt}} - 1 = f + 1$ additional RSU shares are needed. If the Cloud is unavailable, $t_{\text{decrypt}} = f + 2$ RSU shares alone suffice. The partial decryption computed independently by each cooperating RSU is shown in Equation (3.61):

$$pd_j = \text{PDec}(sk_j^{\text{share}}, \text{Enc}(\bar{X}_{\text{global}})), \quad r_j \in \mathcal{D}, \quad |\mathcal{D}| \geq t - 1 \quad (3.61)$$

As shown in Equation (3.61), each partial decryption uses only the RSU's own share without any RSU learning another's share or being able to proceed to full decryption independently. The threshold combination of any t_{decrypt} partial decryptions from the combined pool $\mathcal{R} \cup \{\text{Cloud}\}$ to recover the global aggregate is shown in Equation (3.62):

$$\bar{X}_{\text{global}} = \text{ThDec}(\{pd_e\}_{e \in \mathcal{D}}), \quad \mathcal{D} \subseteq \mathcal{R} \cup \{\text{Cloud}\}, \quad |\mathcal{D}| \geq t_{\text{decrypt}} \quad (3.62)$$

As shown in Equation (3.62), any $t_{\text{decrypt}} = f + 2$ participants from the combined pool may cooperate for decryption. Cloud participation is sufficient but not mandatory if the Cloud is unavailable, t_{decrypt} RSU shares alone suffice, eliminating the availability single point of failure that mandatory Cloud participation would create. When the Cloud participates, its one share counts toward the required t_{decrypt} , reducing the RSU share requirement to $t_{\text{decrypt}} - 1 = f + 1$. Since f RSUs plus the Cloud total only $f + 1$ shares, which is below $t_{\text{decrypt}} = f + 2$, no attacker controlling f RSUs and the Cloud can decrypt unilaterally. TRS signing uses $t_{\text{sign}} = f + 1$ (Eq. (3.54)) and BFT revocation uses $2f + 1$ (Eq. (3.69)), providing consistent fault tolerance across all three mechanisms.

Following threshold decryption, $\bar{X}_{\text{global}}$ is available locally at the RSU cluster for ITS services such as congestion monitoring, eco-driving guidance, and dynamic routing, and is also returned to the Cloud for cloud-side analytics use. Since $\bar{X}_{\text{global}}$ is a mean of RSU-level means, no individual RSU's zone-level aggregate is recoverable from it, preserving zone-level privacy even after plaintext delivery.

Algorithm 6 presents the PQ-FHE-TRS procedure, which formalises the pre-coordination FHE encryption, homomorphic aggregation, and post-quantum threshold ring signing steps executed by the RSU cluster to securely process mobility data without exposing local plaintexts.

As shown in Algorithm 6, the procedure first computes the local plaintext aggregate and immediately encrypts it under the FHE public key (line 3) before any inter-RSU sharing occurs. This structural reordering ensures that when RSUs exchange data to coordinate the threshold ring signature (lines 4–5), they operate entirely on ciphertexts. The ring members then homomorphically combine these ciphertexts into a single encrypted aggregate without decryption (line 6). Finally,

Algorithm 6 Pre-Coordination FHE Encryption and PQ-TRS Signing (PQ-FHE-TRS)

```

1: procedure PQ-FHE-TRS( $\mathcal{V}_j(t)$ ,  $\mathcal{R}$ ,  $\{sk_j\}$ ,  $sk_j^{\text{share}}$ ,  $pk_{\text{FHE}}$ ,  $evk$ ,  $t$ )  $\triangleright sk_j^{\text{share}}$ 
   distributed via LKH at setup; no RSU holds full  $sk_{\text{FHE}}$ 
2:   Compute local plaintext aggregate  $\mathbf{A}_j(t)$  via Eq. (3.50)
3:   if  $\exists f \in \mathcal{F} : A_j^{(f)}(t) \notin [L_f^{\text{SUMO}}, U_f^{\text{SUMO}}]$  then log field anomaly to SC-Trust and
   abort  $\triangleright$  Pre-enc. range check against SUMO-derived bounds
4:   Encrypt local aggregate before ring sharing:  $c_j \leftarrow \text{Enc}(pk_{\text{FHE}}, \mathbf{A}_j(t))$ 
   (Eq. (3.51))
5:   Broadcast  $c_j$  to all  $r_k \in \mathcal{S}$ ,  $r_k \neq r_j$ 
6:   Receive  $\{c_k\}_{k \in \mathcal{S}, k \neq j}$  from ring peers
7:   Compute combined encrypted aggregate:  $\text{Enc}(\mathbf{A}_{\text{ring}}) \leftarrow \bigoplus_{j \in \mathcal{S}} c_j$  (Eq. (3.52))
8:   Form TRS message:  $m \leftarrow (\text{Enc}(\mathbf{A}_{\text{ring}}), t, \nu_{\mathcal{S}}, \text{ID}_{\mathcal{S}}, h(\mathcal{S}))$  (Eq. (3.53))
9:   for each  $r_j \in \mathcal{S}$  do
10:     $\sigma_j \leftarrow \text{Sign}(sk_j, m)$  (Eq. (3.54))
11:   end for
12:    $\sigma_{\text{TRS}} \leftarrow \text{Aggregate}(\{\sigma_j\}_{j \in \mathcal{S}})$  (Eq. (3.55))
13:   Transmit  $(\text{Enc}(\mathbf{A}_{\text{ring}}), \sigma_{\text{TRS}})$  to cloud
14:   return  $\sigma_{\text{TRS}}, \text{Enc}(\mathbf{A}_{\text{ring}})$ 
15: end procedure

```

the procedure binds this combined ciphertext to the signing ring identity, coordinates the partial signing across the t -of- n subset (lines 8–11), and aggregates them into the final compact threshold signature σ_{TRS} (line 12) for transmission to the cloud. This ensures that the cloud receives only privacy-protected, threshold-verified data.

3.5.5 Blockchain Trust Layer with Smart Contract Design

The blockchain layer is implemented on a permissioned consortium chain (e.g., Hyperledger Fabric) shared among RSUs and the SDN controller, with RSUs acting as endorsing peers and the controller as the ordering service. Although every registered RSU holds a peer certificate and is therefore eligible to endorse, the endorsing committee is not the full RSU set. At runtime the system selects the $2f + 1$ endorsers as the highest-trust members of the trusted-RSU set $R_{\text{trusted}}(t)$, re-evaluating this ranking periodically (~ 1 Hz) from the live on-chain trust scores maintained by SC-Trust. This keeps endorsement on the most reliable peers and avoids the cost of treating all RSUs as active endorsers. When trust scores are uninitialised or tied (e.g. at bootstrap), the selection falls back to a uniform random draw over $R_{\text{trusted}}(t)$. IPFS is integrated for off-chain storage of raw beacon sequences: each RSU stores beacon windows on IPFS and records only the content hash on-chain, enabling real-time retrieval by the GAT and temporal autoencoder components without incurring on-chain storage costs.

Algorithm 7 Threshold FHE Decryption at RSU Cluster (THRESH-DEC)

```

1: procedure THRESH-DEC( $\text{Enc}(\bar{X}_{\text{global}}), \{sk_e^{\text{share}}\}_{e \in \mathcal{D}}, t_{\text{decrypt}}$ )  $\triangleright \mathcal{D} \subseteq \mathcal{R} \cup \{\text{Cloud}\};$ 
   Cloud not mandatory
2:   if  $|\mathcal{D}| < t_{\text{decrypt}}$  then
3:     abort — insufficient shares for decryption
4:   end if
5:   for each  $e \in \mathcal{D}$  do
6:      $pd_e \leftarrow \text{PDec}(sk_e^{\text{share}}, \text{Enc}(\bar{X}_{\text{global}}))$  (Eq. (3.61))
7:     Broadcast  $pd_e$  to all  $e' \in \mathcal{D}, e' \neq e$ 
8:   end for
9:    $\bar{X}_{\text{global}} \leftarrow \text{ThDec}(\{pd_e\}_{e \in \mathcal{D}})$  (Eq. (3.62))
10:  if  $\bar{X}_{\text{global}} \in [\mathbf{L}^{\text{SUMO}}, \mathbf{U}^{\text{SUMO}}]$  then
11:     $\bar{X}_{\text{global}}^{\text{valid}} \leftarrow \bar{X}_{\text{global}}$   $\triangleright$  accept; update last-valid aggregate
12:  else
13:    log aggregate anomaly to SC-Trust (best-effort)
14:     $\bar{X}_{\text{global}} \leftarrow \bar{X}_{\text{global}}^{\text{valid}}$   $\triangleright$  reject; reuse last valid window. Static envelope, no live
    ground truth, no per-RSU attribution
15:  end if
16:  Use  $\bar{X}_{\text{global}}$  for local ITS services and return to Cloud for analytics
17:  return  $\bar{X}_{\text{global}}$ 
18: end procedure

```

3.5.5.1 Registration Smart Contract (SC-Register)

Before participating in the framework, each vehicle v_i or RSU r_j submits its identity, public key, and LKH leaf key hash directly to the blockchain, as shown in Algorithm 8. SC-Register commits the registration only if at least $2f+1$ endorsements are collected from the top-trust $2f+1$ members of $R_{\text{trusted}}(t)$, using the same BFT threshold defined in Eq. (3.69). This eliminates any reliance on a centralized registration authority – a single compromised RSU or controller cannot unilaterally onboard a fabricated identity. SC-Trust processes anomaly evidence exclusively for identities with a valid on-chain registration record, and SC-Revoke can only revoke registered identities, closing the identity lifecycle on-chain from registration through revocation.

3.5.5.2 Trust Score Smart Contract (SC-Trust)

The on-chain trust score $\tau_i(t)$ update for vehicle v_i after each detection epoch is given in Equation 3.63. This exponential moving average update accumulates anomaly evidence from multiple RSUs across time, preventing a single false positive from triggering revocation while ensuring that sustained malicious behaviour gradually reduces the trust score below the revocation threshold regardless of how

Algorithm 8 Registration Smart Contract (**SC-Register**)

```

1: procedure SC-REGISTER( $ID_i, pk_i, h(\mathcal{K}_{u_i}), \{\sigma_j^{\text{endorse}}\}_{j \in \mathcal{S}}$ )
2:   if  $ID_i$  already exists on-chain then
3:     reject – duplicate identity
4:   end if
5:    $n_{\text{endorse}} \leftarrow \sum_{r_j \in \mathcal{R}} \mathbf{1}[\text{Verify}(pk_j, ID_i \| pk_i \| h(\mathcal{K}_{u_i}), \sigma_j^{\text{endorse}})]$ 
6:   if  $n_{\text{endorse}} < 2f + 1$  then
7:     reject – insufficient RSU endorsements
8:   end if
9:   Commit  $(ID_i, pk_i, h(\mathcal{K}_{u_i}), t_{\text{reg}})$  to ledger
10:  Set  $\tau_i \leftarrow \tau_{\text{init}}, \text{status}_i \leftarrow \text{ACTIVE}$ 
11:  emit Registered( $ID_i, t_{\text{reg}}$ )
12: end procedure

```

slowly the attack progresses.

$$\tau_i(t) = \begin{cases} \alpha \tau_i(t-1) + (1-\alpha) \left(1 - \frac{1}{|\mathcal{R}_i^{\text{obs}}(t)|} \sum_{r_j \in \mathcal{R}_i^{\text{obs}}(t)} \psi_j^{(i)}(t) \right) & \mathcal{R}_i^{\text{obs}}(t) \neq \emptyset \\ \tau_i(t-1) & \mathcal{R}_i^{\text{obs}}(t) = \emptyset \end{cases} \quad (3.63)$$

where $\mathcal{R}_i^{\text{obs}}(t) \triangleq \{r_j \in \mathcal{R}_{\text{trusted}}(t) : v_i \in \mathcal{V}_j(t)\}$ is the set of currently trusted RSUs that observed vehicle v_i in epoch t , $\alpha \in (0, 1)$ is the EMA decay factor, and trust is held constant when no trusted RSU observed the vehicle. In the lightweight mode, $\psi_j^{(i)}(t)$ from Eq. (3.21) is used directly; in the full mode, $\Phi_i(t)$ from Eq. (3.48) substitutes for $\psi_j^{(i)}(t)$.

SC-Trust maintains identical trust score records for all registered controllers in $\mathcal{C}$ alongside vehicles. RSU evidence about controller behaviour is the binary conflict indicator $\text{conflict}_j(t)$ defined in Eq. (3.72). The trust score update for controller c_k driven by RSU conflict evidence submitted directly to the blockchain is shown in Equation (3.64):

$$\tau_{c_k}(t) = \alpha \tau_{c_k}(t-1) + (1-\alpha) \left(1 - \frac{1}{|\mathcal{R}_{c_k}^{\text{obs}}(t)|} \sum_{r_j \in \mathcal{R}_{c_k}^{\text{obs}}(t)} \text{conflict}_j(t) \right) \quad (3.64)$$

where $\mathcal{R}_{c_k}^{\text{obs}}(t) \triangleq \{r_j \in \mathcal{R}_{\text{trusted}}(t) : c_{\text{assigned}}(r_j) = c_k\}$ is the set of currently trusted RSUs assigned to controller c_k in epoch t . As shown in Equation (3.64), a controller whose decisions systematically conflict with the RSUs assigned to it accumulates decreasing trust. When $\tau_{c_k}(t)$ remains below τ_{min} for T_{rev} consecutive epochs, SC-Trust emits a revocation request to SC-Revoke, triggering RSU reassignment to $\mathcal{C}_{\text{trusted}}(t)$ as defined in Eq. (3.1). As shown in Equation 3.63, the trust score is updated as an exponential moving average, where higher anomaly

scores from RSUs reduce $\tau_i(t)$, and sustained low scores eventually trigger revocation. The SC-Trust contract enforces the following state machine: $\tau_i \in [\tau_{\min}, 1]$; if $\tau_i(t) < \tau_{\text{warn}}$ the vehicle is placed in a probationary state receiving reduced routing priority; if $\tau_i(t)$ remains below $\tau_{\min}$ for T_{rev} consecutive epochs, SC-Trust emits a **RevocationRequest** event to SC-Revoke.

Tiered Commit Strategy. Per-beacon synchronous blockchain commits are impractical at the 100ms beacon rate: each Hyperledger Fabric consensus round-trip would block the RSU processing pipeline and inflate end-to-end latency. A four-tier commit strategy is therefore adopted to balance security guarantees against operational feasibility.

Tier 1 — Immediate synchronous commit. Operations whose outcome must be reflected in $\mathcal{R}_{\text{trusted}}(t)$ or $\mathcal{C}_{\text{trusted}}(t)$ before the next epoch is processed are committed synchronously: **Revoke**(ID_{*i*}) and **ControllerRevoked** events from SC-Revoke, and RSU controller-reassignment updates to $c_{\text{assigned}}(r_j)$. These events change which RSU evidence counts toward the BFT threshold and must therefore achieve ledger finality immediately.

Tier 2 — Per-epoch commit (every $L \cdot T_b$ seconds). Trust score updates that cross a threshold boundary — the trusted-to-probationary transition ($\tau < \tau_{\text{warn}}$) and the probationary-to-client transition ($\tau < \tau_{\min}$) — are committed once per detection epoch, as are TRS aggregate signatures σ_{TRS} and RSU lifecycle state changes. One Fabric transaction per epoch per event type replaces per-beacon writes for these events.

Tier 3 — Batched periodic commit (every T_{batch} seconds or N_{commit} items). Anomaly evidence submissions $\mathcal{E}_j(t)$ (Eq. 3.67) that have not triggered a threshold crossing are accumulated in a local per-RSU evidence buffer and flushed to the ledger as a single batched Fabric transaction every T_{batch} seconds or when the buffer reaches N_{commit} entries, whichever occurs first. The flush executes asynchronously in a background thread without blocking the simulation or detection pipeline. The BFT security guarantee of Equation 3.69 is preserved: the revocation condition counts distinct RSU submissions within temporal window T_w , and since $T_{\text{batch}} \ll T_w$, all batched submissions arrive within the evidence accumulation window regardless of batching.

Tier 4 — Off-chain only (IPFS, no chain commit). Raw beacon windows are stored on IPFS with only their content hash committed on-chain as already specified; individual per-beacon $\psi_i(t)$ scores, GAT and LSTM-AE detection scores $\Phi_i(t)$, and controller evidence submissions $\mathcal{E}_c(t)$ (Eq. 3.68) are stored off-chain and their hashes included in the next Tier 3 batch, since these are untrusted peer contributions that do not require immediate ledger finality.

The design trade-off introduced by Tiers 2 and 3 is that trust score transitions are reflected on-chain at epoch or batch granularity rather than per-beacon. The revocation decision itself (Tier 1) remains immediate, so the consequential network action — exclusion from $\mathcal{R}_{\text{trusted}}(t)$ — is unaffected by the batching delay.

SC-Trust additionally maintains a trust score $\tau_{r_j}(t)$ for every registered RSU

$r_j \in \mathcal{R}$, instantiating the RSU decay-and-revocation mechanism anticipated in the cryptographic mitigation scope analysis. Because a compromised RSU is an endorsing peer rather than a monitored vehicle, its evidence cannot be scored against an external anomaly detector; instead it is scored against the collective decision of its honest peers. For each vehicle v_i that r_j reports on within window T_w , let $\text{flag}_i^{\text{rsu},j}$ be r_j 's own binary verdict (Eq. (3.71)), and let $q_i(t) \in \{0, 1\}$ be the $2f + 1$ quorum outcome of Eq. (3.69) computed only over the current trusted endorser set $\mathcal{R}_{\text{trusted}}(t)$. The per-window RSU misbehaviour signal is the mean disagreement between the two, shown in Equation (3.65):

$$m_j(t) = \frac{1}{|\mathcal{V}_j(t)|} \sum_{v_i \in \mathcal{V}_j(t)} |\text{flag}_i^{\text{rsu},j}(t) - q_i(t)| \quad (3.65)$$

where $\mathcal{V}_j(t)$ is the set of vehicles r_j reported on in the window. Equation (3.65) penalises both under-reporting (a malicious vehicle the trusted quorum caught but r_j cleared, $\text{flag} = 0$, $q_i = 1$) and over-reporting (a benign vehicle the trusted quorum cleared but r_j flagged, $\text{flag} = 1$, $q_i = 0$). Evaluating $q_i(t)$ against $\mathcal{R}_{\text{trusted}}(t)$ alone prevents a coalition of demoted RSUs from mutually validating one another's fabricated verdicts. The RSU trust score then decays by the same exponential moving average used for vehicles (Eq. (3.63)), driven by $m_j(t)$, as shown in Equation (3.66):

$$\tau_{r_j}(t) = \alpha \tau_{r_j}(t-1) + (1 - \alpha) (1 - m_j(t)) \quad (3.66)$$

SC-Trust enforces a three-state lifecycle on each RSU, reusing the vehicle thresholds. While $\tau_{r_j}(t) \in [\tau_{\min}, 1]$ the RSU is trusted: its evidence counts toward the $2f + 1$ quorum of Eq. (3.69) and it is eligible to endorse. If $\tau_{r_j}(t) < \tau_{\text{warn}}$ it enters a probationary trusted state. If $\tau_{r_j}(t) < \tau_{\min}$ it is demoted to the client state: it may still submit evidence—so its score can recover—but those submissions carry zero quorum weight and it may not endorse. A client-state RSU whose subsequent $m_j(t)$ realigns with $\mathcal{R}_{\text{trusted}}(t)$ and whose score climbs back to $\tau_{\min}$ is promoted to the trusted state. If $\tau_{r_j}(t)$ remains below $\tau_{\min}$ for T_{rev} consecutive windows, SC-Trust emits a **RevocationRequest** for r_j to SC-Revoke, permanently removing it from $\mathcal{R}_{\text{trusted}}$. The BFT safety guarantee requires $|\mathcal{R}_{\text{trusted}}(t)| \geq 3f + 1$ at all times; if demotion events would violate this floor, the lowest-trust probationary RSU is retained as trusted until a replacement is registered and endorsed.

3.5.5.3 Revocation Smart Contract (SC-Revoke)

Since the threat model includes a compromised controller, SC-Revoke is triggered exclusively by RSU peer evidence submitted directly to the blockchain, bypassing the controller entirely. The signed evidence tuple submitted by each RSU r_j directly to the blockchain is given in Equation 3.67.

This signed evidence tuple enables each RSU to submit tamper-evident anomaly reports directly to the blockchain without routing through the controller, ensuring

that a compromised controller cannot suppress or modify RSU observations before they reach the revocation smart contract.

$$\mathcal{E}_j(t) = (\text{ID}_i, \psi_j^{(i)}(t), t, h(\mathbf{b}_i(t)), \sigma_j^{\text{sub}}) \quad (3.67)$$

As shown in Equation 3.67, each RSU independently submits a signed evidence tuple containing the vehicle identity, anomaly score, timestamp, beacon hash, and its own verifiable signature, ensuring tamper-evident RSU-side evidence on-chain where σ_j^{sub} is RSU r_j 's signature over the submission, verifiable on-chain using pk_j . The supplementary evidence tuple that the controller may submit is given in Equation 3.68.

This submission structure treats the controller's anomaly score as one additional peer vote subject to the same BFT verification as RSU submissions, preventing a compromised controller from gaining disproportionate influence over revocation decisions while still incorporating full-mode AI detection results when the controller is honest.

$$\mathcal{E}_c(t) = (\text{ID}_i, \Phi_i(t), t, h(\mathbf{X}_i(t)), \sigma_c^{\text{sub}}) \quad (3.68)$$

As shown in Equation 3.68, the controller's submission is treated as one additional peer vote and never a prerequisite for revocation.

Since a vehicle typically interacts with only one RSU at a time, requiring $2f + 1$ simultaneous RSU observations of the same vehicle is geometrically infeasible under real mobility. Revocation is therefore defined over the number of distinct RSUs that have independently flagged vehicle v_i within a temporal window T_w , accumulating evidence across all RSUs a vehicle visits over time. Counting each RSU at most once removes the risk of a single malicious RSU inflating the evidence count through repeated submissions. The revocation condition requiring at least $2f + 1$ distinct RSUs to have independently flagged vehicle v_i within window T_w is given in Equation 3.69:

$$\left| \left\{ r_j \in \mathcal{R}_{\text{trusted}}(t) : \exists t' \in [t - T_w, t], \psi_j^{(i)}(t') > \psi_{\text{th}} \right\} \right| \geq 2f + 1 \implies \text{emit } \text{Revoke}(\text{ID}_i) \quad (3.69)$$

As shown in Equation 3.69, each RSU is counted at most once regardless of how many beacon intervals it reports anomalies for vehicle v_i , so no single RSU can inflate the evidence count. Revocation requires $2f + 1$ independent RSU witnesses accumulated over time rather than simultaneously, making it reachable under normal single-RSU-at-a-time mobility while remaining robust against single-source false positives. The controller compromise detection condition is defined at the binary decision level to avoid undefined comparisons between heterogeneous continuous scores. The binary detection flag produced by the controller for vehicle v_i is shown in Equation (3.70):

$$\text{flag}_i^{\text{ctrl}} = \mathbf{1}[\Phi_i(t) > \Phi_{\text{th}}] \quad (3.70)$$

As shown in Equation (3.70), the controller produces a binary alert or no-alert decision from its fusion score. The binary detection flag produced independently by RSU r_j from its local rule-based score is shown in Equation (3.71):

$$\text{flag}_i^{\text{rsu},j} = \mathbf{1} \left[\psi_j^{(i)}(t) > \psi_{\text{th}} \right] \quad (3.71)$$

As shown in Equation (3.71), each RSU independently produces its own binary decision without consulting the controller. A conflict is defined directionally: only the case where the controller classifies a vehicle as benign while the RSU flags it as anomalous constitutes a conflict. The reverse case the controller flagging a vehicle that the lightweight RSU rules miss is the controller performing superior detection and must not be penalised. The directional conflict indicator that fires only when the controller fails to flag what RSUs flag is shown in Equation (3.72):

$$\text{conflict}_j(t) = (1 - \text{flag}_i^{\text{ctrl}}) \cdot \text{flag}_i^{\text{rsu},j} \quad (3.72)$$

As shown in Equation (3.72), a conflict fires only when $\text{flag}_i^{\text{ctrl}} = 0$ and $\text{flag}_i^{\text{rsu},j} = 1$, identifying controller suppression without penalising superior full-mode detection. The controller compromise flag aggregating binary conflicts across RSU peers is shown in Equation (3.73):

$$\text{flag}_c = \mathbf{1} \left[\sum_{r_j \in \mathcal{R}} \text{conflict}_j(t) \geq f + 1 \right] \quad (3.73)$$

As shown in Equation (3.73), when at least $f + 1$ independent RSU–controller disagreements are detected, SC-Revoke sets $\text{flag}_c = 1$, excludes $\mathcal{E}_c(t)$ from all subsequent voting rounds, and delegates routing decisions to RSU-level rule-based control. RSUs then automatically reassign to any available trusted controller from $\mathcal{C}_{\text{trusted}}(t)$ per Eq. (3.1), with no manual failover required. Upon revocation, SC-Revoke broadcasts a signed **CRL-Update** directly to all RSUs, without controller relay.

SC-Revoke applies identically to controllers in $\mathcal{C}$. When SC-Trust triggers revocation of controller c_k , SC-Revoke marks c_k as untrusted on-chain, updates $\mathcal{C}_{\text{trusted}}(t)$ per Eq. (3.1), and broadcasts a **ControllerRevoked**(ID $_{c_k}$) event to all RSUs. RSUs assigned to c_k query the blockchain for $\mathcal{C}_{\text{trusted}}(t)$ and independently reassign to any available trusted controller, maintaining uninterrupted network operation without a designated replacement or manual failover.

Algorithm 9 presents the CP-DETECT procedure, which verifies the post-quantum TRS aggregate signature at the RSU level and cross-checks the controller’s blockchain submission against RSU peer evidence to detect control-plane compromise.

Algorithm 9 Control-Plane Poisoning Detection (CP-DETECT)

```

1: procedure CP-DETECT( $\mathbf{A}_j(t), \sigma_{\text{TRS}}, \{pk_j\}_{j=1}^n, \{\mathcal{E}_j(t)\}_{r_j \in \mathcal{R}}$ )
2:   Verify TRS at RSU level:  $\text{valid} \leftarrow \text{Verify}(\{pk_j\}, m_j, \sigma_{\text{TRS}})$  (Eq. (3.56))
3:   if  $\neg \text{valid}$  then
4:     RSUs emit ALERT_CP( $\text{ID}_{r_j}, \text{TRS\_FAIL}$ ) directly to blockchain
5:     return
6:   end if
7:   Retrieve controller submission  $\mathcal{E}_c(t)$  from blockchain
8:   For each  $r_j \in \mathcal{R}$ : compute  $\text{conflict}_j(t) = (1 - \text{flag}_i^{\text{ctrl}}) \cdot \text{flag}_i^{\text{rsu},j}$  (Eq. (3.72))
9:   Compute  $\text{flag}_c$  via Eq. (3.73)
10:  if  $\text{flag}_c = 1$  then
11:    SC-Revoke excludes  $\mathcal{E}_c(t)$  from consensus
12:    RSUs reassign to  $\mathcal{C}_{\text{trusted}}(t)$  (Eq. (3.1)); no manual failover
13:    return ALERT_CP( $\text{ID}_c, \text{CTRL\_COMPROMISED}$ )
14:  end if
15:  return PASS
16: end procedure

```

As shown in Algorithm 9, a TRS verification failure immediately causes RSUs to emit an alert directly to the blockchain, bypassing the controller entirely. If verification passes but more than $f + 1$ RSUs report a conflict with the controller submission, SC-Revoke excludes the controller from consensus and activates RSU-level rule-based fallback control. The fallback control mechanism delegates routing and safety decisions to the distributed rule-based signature detection running at each RSU independently, using the lightweight mode pipeline from Algorithm 1, which ensures that the network continues to operate at a reduced but functional security level even when the controller is excluded from consensus. The conflict detection in lines 8–10 distinguishes between a malicious controller and a merely slow or partitioned one by requiring $f + 1$ conflicting RSU submissions rather than a single discrepancy: a controller that is temporarily delayed will produce consistent evidence once its submission arrives, whereas a malicious controller will consistently submit scores that contradict the RSU-observed anomaly levels across multiple epochs. The signed evidence structure in Equation 3.67 is what makes this cross-checking possible on-chain without requiring RSUs to communicate directly with each other, as each RSU’s submission is independently verifiable using its public key pk_j recorded on the permissioned ledger.

CP-DETECT’s conflict computation combines two distinct vantage points on the same epoch, and the framework’s logging deliberately keeps them separate rather than collapsing them into a single verdict. The RSU-view verdict $\text{flag}_i^{\text{rsu},j}$ is produced locally at each RSU r_j from its own lightweight or full-mode detection pipeline, before anything is forwarded to the controller; the controller-view verdict $\text{flag}_i^{\text{ctrl}}$ is the decision the controller reports back on-chain after receiving the RSU’s submission. This two-vantage-point separation reflects the threat model directly: a compromised controller can misreport its own verdict without being able to alter

what an honest RSU already observed and signed. Every logged beacon-evaluation record is tagged with which vantage point produced it, so that the conflict indicator $\text{conflict}_j(t)$ (Eq. 3.72) can be recomputed and audited independently of the live CP-DETECT run.

3.5.5.4 Real-Time vs. Offline Detection Mode

The proposed framework explicitly supports two temporal modes. Real-time detection executes the rule-based signature check (Algorithm 1) at the RSU per beacon arrival (latency $\leq T_b = 100\text{ms}$), the TRS signing per aggregate epoch, and the SC-Trust update after each epoch, all without waiting for controller-side AI inference. Offline (semi-real-time) detection executes the GAT (Algorithm 5) and temporal autoencoder at the controller over a sliding window updated every $W = L \cdot T_b$ seconds, with results fed back to SC-Trust as a correction signal. IPFS-stored beacon sequences enable the controller to retrieve historical windows on demand for reanalysis, bridging the real-time and offline modes without data loss.

Figure 3.12 presents the detection logic flow diagrams for trajectory poisoning detection (TP-DETECT) and Sybil-based mobility pattern poisoning detection (SYB-DETECT), showing the sequential signature checks and decision branching for each attack class.

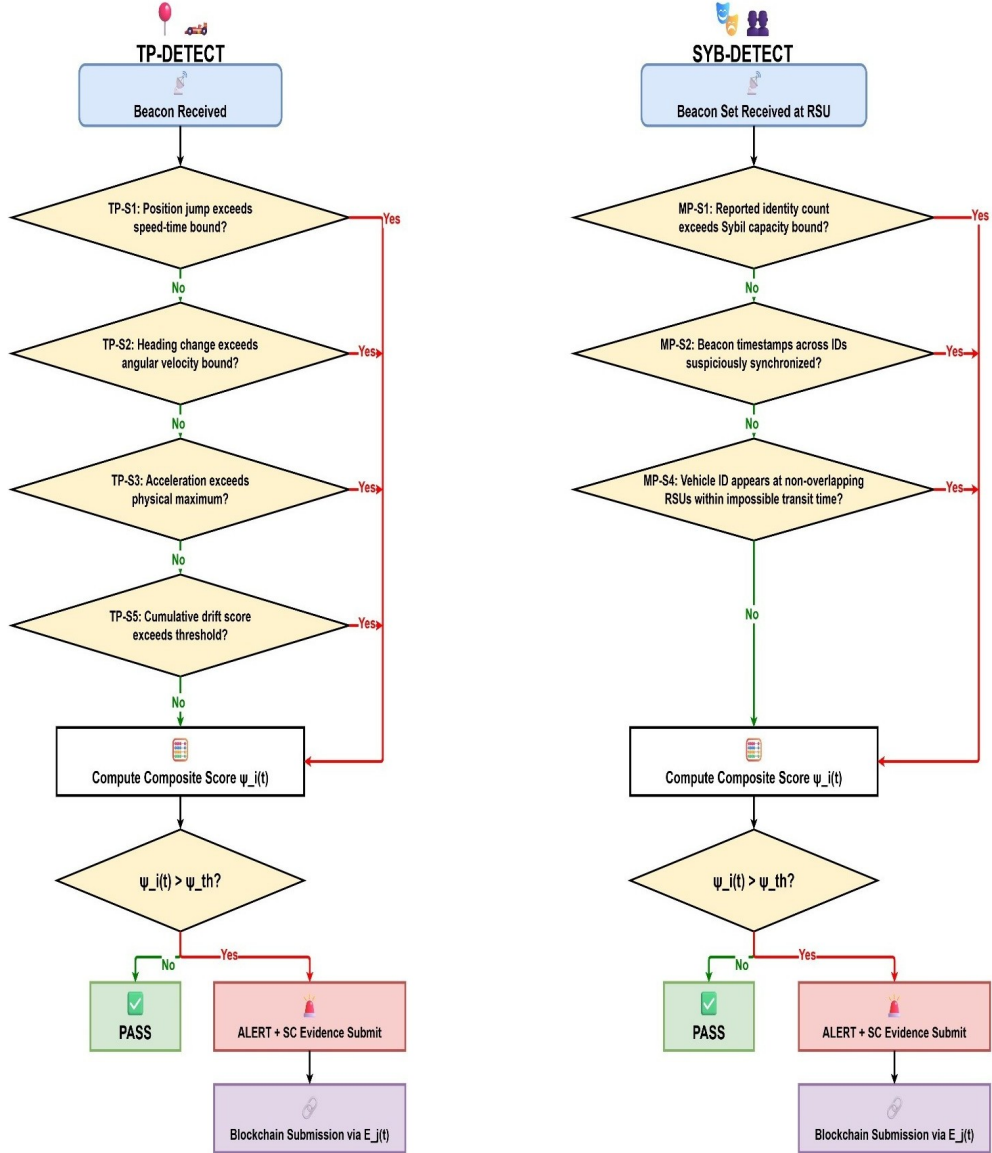


Figure 3.12: Detection flow diagrams for TP-DETECT and SYB-DETECT.

As shown in Figure 3.12, TP-DETECT (left) evaluates four signature checks sequentially TP-S1 (position jump), TP-S2 (heading rate), TP-S3 (acceleration bound), and TP-S5 (cumulative drift score) where any violation feeds into the composite score $\psi_i(t)$; if $\psi_i(t) > \psi_{th}$ an ALERT is raised and signed evidence is submitted directly to the blockchain via $E_j(t)$, otherwise a PASS is returned with no blockchain interaction. SYB-DETECT (right) checks MP-S1 (identity density anomaly), MP-S2 (synchronized beacon timing), and MP-S4 (ghost vehicle transit impossibility) before applying the same composite score threshold decision.

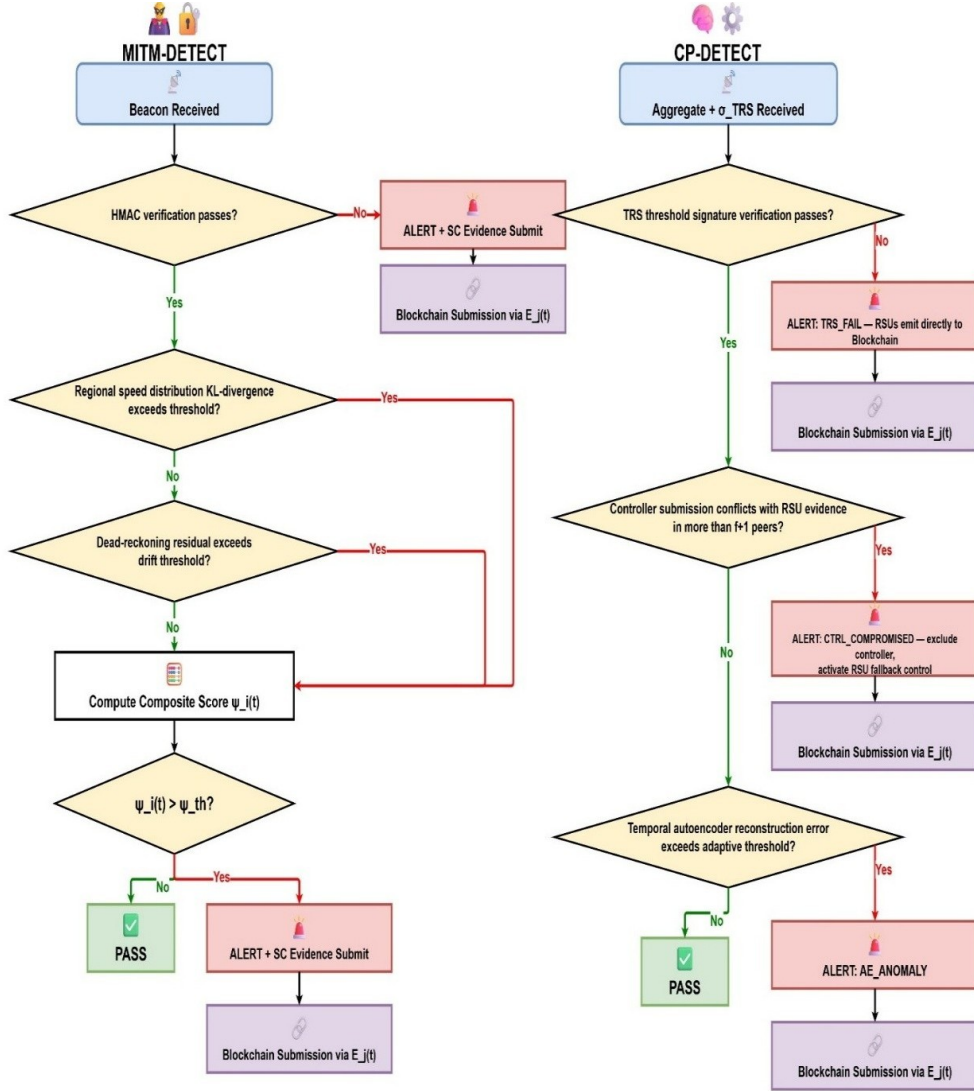


Figure 3.13: Detection flow diagrams for MITM-DETECT and CP-DETECT.

As shown in Figure 3.13, MITM-DETECT (left) first verifies the HMAC tag a failure immediately triggers an ALERT without further processing then checks MP-S3 (KL-divergence speed distribution shift) and the dead-reckoning residual before computing the composite score. CP-DETECT (right) operates at the aggregate level: TRS threshold verification failure triggers a TRS_FAIL alert emitted directly by RSUs to the blockchain; if verification passes, controller submission conflicts with more than $f + 1$ RSU peers trigger CTRL_COMPROMISED and activate RSU fallback control; finally, the temporal autoencoder reconstruction error is checked against the adaptive threshold, raising AE_ANOMALY if exceeded.

Chapter 4

Experimental Setup

4.1 Baseline Selection and Benchmarking Rationale

4.1.1 External Baselines

Three external baselines are selected from post 2020 literature that address mobility data anomaly detection or vehicular network poisoning in related settings.

B1 - Ghaleb et al. (2014) [8]: Selected as the most directly comparable work addressing mobility pattern misbehavior detection in vehicular networks using rule based threshold checks. Despite its age, no post 2020 SDVN specific mobility poisoning detector exists in the literature, making this the canonical baseline for the detection component. It represents the class of threshold based approaches against which our signature characterized detection must demonstrate superiority.

B2 - Ercan et al. (2022) [19]: Selected as a post 2020 machine learning baseline that specifically targets position falsification misbehavior detection in VANETs, representing feature engineered supervised classification over beacon data. Comparison against B2 evaluates whether our mobility specific spatial (GAT) and temporal (LSTM-AE) formulation provides detection gains over engineered per beacon features classified in isolation.

B3 - Sharma and Liu (2021) [22]: Selected as a post 2020 data centric machine learning misbehavior detection baseline for the Internet of Vehicles using plausibility based feature engineering on basic safety messages. Comparison against B3 isolates the contribution of our graph relational spatial fusion and sequential temporal modeling over a strong but message level data centric classifier.

4.1.2 Ablation Study Design

Eleven ablation variants are defined to isolate every independently ablatable design component of the proposed framework. Each variant removes or replaces exactly one component while holding all others fixed, enabling clean attribution of ob-

served performance changes to the removed component. The five cryptographic and blockchain variants (AB6–AB11) use ablation-only metrics (PARR, COO, BWO, TCL, FRR) that cannot be evaluated against external baselines B1–B3 because those baselines share no equivalent mechanism. The six detection variants (AB1–AB5, AB8–AB9) additionally report metrics that are directly comparable across baselines. Table 4.1 summarises the full design.

Table 4.1: Ablation study design: eleven independent component isolation experiments. Metric abbreviations follow Section 4.2.

ID	Subsystem	Component removed / modified	X variable	Y metrics
AB1	Detection	Rule-based signatures (TP-S1–MP-S4)	Attack penetration rate ρ_a	MCC, TTD, PBPO
AB2	Detection	HMAC beacon integrity and nonce anti-replay	MitM penetration rate ρ_{MitM}	MCC, TTD, CDER
AB3	Detection	GAT spatial detector	Coordinated attacker count n_{coord}	MCC, TTD, CDER
AB4	Detection	LSTM-AE temporal detector	Stealth drift cap ϵ_{max}	MCC, TTD, CDER, TPE
AB5	Detection	Full AI layer (GAT + LSTM-AE)	Vehicle speed regime	MCC, TTD, CDER, TDEE, TPE, PBPO
AB6	Cryptography	TRS threshold signing gate	Compromised RSU fraction f/n	PARR, CDER, COO
AB7	Cryptography	FHE pre-coordination encryption	RSU ring size n	COO, TRS signing latency (PQ vs. ECDSA), $\text{BWO}_{\text{ratio}}$, DKG setup latency
AB8	Trust	RSU three-state trust lifecycle	Malicious RSU fraction f/n	MCC, CDER, $\text{FRR}_{\text{revoke}}$, $\text{FRR}_{\text{demote}}$
AB9	Trust	Multi-controller architecture and controller revocation	Controller compromise onset $t_{\text{comp}}/T_{\text{sim}}$	MCC, CDER
AB10	Trust	Blockchain trust layer (SC-Trust / SC-Revoke)	Revocation-triggering vehicle count	CDER, $\text{FRR}_{\text{revoke}}$, $\text{FRR}_{\text{demote}}$, $\text{TCL}_{\text{confirm}}$, $\text{TCL}_{\text{reassign}}$
AB11	Key Mgmt.	LKH replaced with unicast key distribution	Vehicle zone density $ \mathcal{V}_j $	$\text{BWO}_{\text{scale}}$, PBPO

AB1 — Rule-Based Signatures Removed. The nine rule-based attack signatures (TP-S1 through MP-S4) and the composite anomaly score $\psi_i(t)$ (Eq. 3.21) are disabled. HMAC beacon integrity verification, GAT, LSTM-AE, and the blockchain trust layer remain fully active. The system relies entirely on the HMAC integrity gate and AI detection with no rule-based pre-

filter. Attack penetration rate ρ_a is varied across $\{0.05, 0.10, 0.20, 0.30, 0.40\}$. MCC quantifies detection quality loss without the kinematic pre-filter; TTD quantifies the detection delay increase when early kinematic violations can no longer trigger immediate lightweight alerts; PBPO captures the per-beacon overhead change when the signature scoring stage is bypassed. Isolates the rule-based signatures' contribution to detection speed and lightweight beacon pre-filtering before AI inference.

AB2 — HMAC Removed. HMAC tag generation, freshness window verification, and nonce anti-replay enforcement (Eq. 3.42) are disabled. Rule-based signatures, GAT, LSTM-AE, and the blockchain trust layer remain fully active. MitM adversaries can modify beacon content in transit without triggering the integrity gate before rule-based scoring. MitM penetration rate ρ_{MitM} is varied across $\{0.05, 0.10, 0.20, 0.30, 0.40\}$. MCC captures the detection quality loss under unrestricted in-transit modification; TTD captures how much slower detection becomes when modified beacons bypass the fail-fast HMAC gate and must be caught by downstream components; CDER captures the end-to-end control decision degradation from undetected in-transit poisoning. Isolates HMAC's specific contribution against data-plane interception, which is the only mechanism that catches modification before any rule-based or AI scoring is applied.

AB3 — GAT Disabled. The GAT spatial detector is removed; $\lambda_2^{(k)} = 0$ for all k , and the remaining fusion weights $\lambda_1^{(k)}, \lambda_3^{(k)}$ are renormalized to sum to one within each attack-class set. HMAC, rule-based signatures, LSTM-AE, and blockchain remain fully active. The number of simultaneously coordinating attack vehicles n_{coord} is varied across $\{1, 2, 5, 10, 20\}$ while total penetration rate is held fixed at $\rho_a = 0.20$. Higher n_{coord} increases the spatial collective inconsistency that the GAT is specifically designed to detect. MCC captures detection quality loss on coordinated attacks; TTD captures the delay increase when neighborhood-level spatial inconsistency can no longer be detected at the embedding level; CDER captures the end-to-end control decision impact of undetected coordinated poisoning. Isolates the GAT's unique contribution to detecting multi-vehicle spatial configurations that individually pass all per-beacon and per-vehicle checks.

AB4 — LSTM-AE Disabled. The LSTM temporal autoencoder is removed; $\lambda_3^{(k)} = 0$ for all k , and the remaining fusion weights $\lambda_1^{(k)}, \lambda_2^{(k)}$ are renormalized within each attack-class set. HMAC, rule-based signatures, GAT, and blockchain remain fully active. The stealth perturbation magnitude ϵ_{max} is varied across $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ m per beacon while penetration rate is fixed at $\rho_a = 0.20$. Lower ϵ_{max} produces trajectories that individually pass all kinematic checks and produce spatially plausible embeddings, isolating the regime where only cumulative temporal drift is detectable. MCC captures detection quality loss on near-invisible stealth attacks; TTD captures

the detection delay increase when sequential temporal drift can no longer be accumulated into a reconstruction error signal; CDER captures the control decision degradation; TPE captures the trajectory prediction accuracy lost when gradual drift goes undetected. Isolates the LSTM-AE’s unique contribution to catching gradual temporal trajectory poisoning that stays within per-beacon kinematic bounds throughout the attack.

AB5 — Full AI Layer Removed (Lightweight Mode Only). Both GAT and LSTM-AE are removed; $\lambda_2^{(k)} = \lambda_3^{(k)} = 0$ for all k . Detection relies solely on HMAC and rule-based signatures. The vehicle speed regime is the independent variable, evaluated across the three scenarios: urban (≤ 30 km/h), rural (30–80 km/h), and highway (> 80 km/h). This directly exposes the Detection Infeasible Zone (Eq. 3.4): at high speed, the beacon sampling window N_b falls below $N_{\min}$ and rule-based cumulative drift detection degrades while AI-based sequential anomaly scoring would recover detection power through multi-RSU offline analysis. MCC across speed regimes shows the detection quality degradation pattern; TTD shows how detection delay increases in high-speed regimes; CDER and both attack-impact metrics (TDEE, TPE) quantify the control-plane consequences of this degradation; PBPO confirms that the lightweight mode satisfies $\leq T_b = 100$ ms throughout. Isolates the entire AI layer’s added value over rule-based detection and validates the mobility amplification formulation.

AB6 — TRS Threshold Signing Gate Removed. Post-quantum threshold ring signature generation and verification (Eqs. 3.54–3.56) are disabled. Aggregates are forwarded to the cloud without threshold endorsement. FHE, detection pipeline, and blockchain remain active. The compromised RSU fraction f/n is varied across $\{0/4, 1/4, 2/4, 3/4\}$ (where $n = 4$ per the simulation configuration). PARR measures the fraction of poisoned aggregates that would have been structurally rejected at the TRS gate, showing it drops to zero by definition without TRS; the comparison between AB6 and full-mode PARR isolates TRS’s structural blocking contribution independent of AI detection accuracy. CDER captures the downstream control decision degradation from unblocked poisoned aggregates. COO quantifies the cryptographic computation saved by removing TRS, establishing its per-epoch cost. Ablation only.

AB7 — FHE Pre-Coordination Encryption Removed. FHE encryption of local RSU aggregates before ring sharing (Eq. 3.51) and homomorphic ring combination (Eq. 3.52) are disabled. RSUs exchange plaintext aggregates during TRS ring coordination. TRS threshold signing, detection pipeline, and blockchain remain active. RSU ring size n is varied across $\{4, 6, 8, 10, 12\}$ to expose how the post-quantum cryptographic pipeline scales with ring membership, benchmarked against a classical ECDSA-class (Shamir-Schnorr-P256) baseline at each ring size. COO quantifies the per-epoch computation

cost of the full pipeline directly; the per-RSU TRS signing latency is additionally compared against the classical baseline to isolate the fixed post-quantum signing premium; BWO_{ratio} quantifies the ciphertext expansion factor of the ring coordination message traffic; and the one-time DKG plus key-generation setup latency (Eqs. 3.25–3.26) is reported as an amortised deployment cost. Together, these metrics justify FHE and post-quantum TRS deployment by making their cost explicit against the privacy and structural-integrity guarantees they provide. Ablation only.

AB8 — RSU Three-State Trust Lifecycle Removed. The RSU misbehavior signal (Eq. 3.65), RSU trust EMA update (Eq. 3.66), and three-state demotion/revocation mechanism for RSUs are disabled. RSUs remain permanently trusted once registered, regardless of observed behavior. All vehicle and controller trust mechanisms, detection pipeline, and cryptographic components remain active. The malicious RSU fraction f/n is varied across $\{0/4, 1/4, 2/4, 3/4\}$. MCC captures detection quality loss when compromised RSUs are never demoted despite systematic misbehavior, since their evidence continues to count toward the BFT quorum with equal weight. CDER captures the end-to-end control decision degradation from sustained insider RSU poisoning that the TRS alone cannot prevent (compromised ring members hold valid keys and produce valid signatures). FRR_{revoke} and FRR_{demote} validate that the lifecycle is safe: honest RSUs must not be incorrectly demoted or revoked by the misbehavior signal under normal operating conditions. Isolates the RSU three-state trust lifecycle’s contribution to containing insider RSU threats that exist within an otherwise valid key ring.

AB9 — Multi-Controller Architecture Removed. The trusted controller set $\mathcal{C}_{\text{trusted}}(t)$ (Eq. 3.1), controller trust score updates (Eq. 3.64), and controller revocation pathway are disabled. A single fixed controller replaces the distributed set; controller compromise has no recovery path. Detection pipeline, cryptographic components, and RSU/vehicle trust mechanisms remain active. Controller compromise onset time $t_{\text{comp}}/T_{\text{sim}}$ is varied across $\{0.1, 0.25, 0.5, 0.75\}$ (fraction of total simulation time elapsed before compromise). Earlier compromise onset gives the malicious controller more time to corrupt control decisions before the simulation ends. MCC captures detection quality degradation when the SDVN controller is a permanent unrecoverable attack source that cannot be excluded from evidence handling. CDER captures the cumulative control decision corruption that accumulates without a reassignment mechanism. Isolates the multi-controller architecture and controller revocation pathway’s contribution to resilience under sustained control-plane attacks.

AB10 — Blockchain Trust Layer Removed. SC-Trust and SC-Revoke are disabled. Trust decisions revert to centralized controller-side revocation with

no BFT guarantee and no controller-bypass path. Detection pipeline, cryptographic components, and entity trust score computations remain active but cannot trigger on-chain revocation events. The number of distinct honest vehicles, RSUs, and controllers that accumulate enough evidence to trigger revocation is varied from $\{5, 10, 20, 50, 100\}$ over the simulation duration by adjusting the attack penetration rate. CDER quantifies the end-to-end control decision degradation from non-BFT, controller-dependent revocation. FRR_{revoke} and FRR_{demote} confirm that safety is also preserved: without the on-chain BFT threshold, a single malicious controller can suppress legitimate revocations or fabricate false ones, which should surface as elevated CDER and abnormal FRR patterns. TCL_{confirm} and TCL_{reassign} drop to zero by definition (no blockchain transactions), establishing the latency baseline for the full-mode TCL comparison. Isolates the blockchain trust layer’s BFT-conditioned revocation contribution independent of detection quality. Ablation only.

AB11 — LKH Replaced with Unicast Key Distribution. The Logical Key Hierarchy (Eq. 3.22) is replaced with direct per-vehicle unicast rekeying, incurring $O(|\mathcal{V}_j|)$ rekey messages per vehicle leave or revocation event instead of the $O(\log_2 |\mathcal{V}_j|)$ LKH cost (Eq. 3.24). All other components remain unchanged. Vehicle zone density $|\mathcal{V}_j|$ is varied across $\{10, 25, 50, 100, 200\}$ to produce the empirical growth curve for both LKH and unicast. BWO_{scale} (Eq. 4.12) records the measured rekey message count for each zone size under both approaches and compares the empirical growth curves against the theoretical $\log_2 |\mathcal{V}_j|$ and $|\mathcal{V}_j|$ predictions. PBPO captures whether the additional unicast rekey traffic affects the per-beacon detection latency budget at high zone densities. Isolates LKH’s logarithmic scalability advantage and empirically validates the closed-form complexity claim in Equation 3.24. Ablation only.

4.2 Primary Evaluation Metrics

Eleven metrics are defined across four evaluation dimensions. Six metrics are evaluated against external baselines B1–B3 and internal ablation variants; five metrics are ablation-study only because the external baselines share no equivalent mechanism (e.g. no TRS gate, no blockchain, no on-chain revocation), making a fair head-to-head comparison undefined. Table 4.2 maps each metric to the research question it primarily answers.

Table 4.2: Metric-to-RQ mapping and evaluation scope.

Metric	Eq.	Primary RQ	Scope
MCC	4.1	RQ2, RQ3, RQ4	All baselines
TTD	4.2	RQ1, RQ3, RQ4	All baselines
CDER	4.4	RQ2, RQ6	All baselines
TDEE	4.7	RQ2, RQ4, RQ5	All baselines
TPE	4.8	RQ1, RQ4	All baselines
PBPO	4.9	RQ3, RQ5	All baselines
PARR	4.3	RQ5	Ablation only
FRR	4.5, 4.6	RQ6	Ablation only
COO	4.10	RQ5	Ablation only
BWO	4.11, 4.12	RQ5, RQ6	Ablation only
TCL	4.13, 4.14	RQ6	Ablation only

4.2.1 Detection Performance Metrics

Matthews Correlation Coefficient (MCC) is used as the primary detection quality metric, robust to class imbalance between attack and normal beacon samples. The MCC is given in Equation 4.1.

MCC is selected as the sole detection quality metric because it summarises all four cells of the confusion matrix in a single balanced scalar, remaining informative under the class imbalance inherent when attack nodes are a small minority of the graph. Separate detection-rate, F1, or FPR metrics are redundant given MCC: a detector that improves MCC necessarily reduces false negatives and false alarms jointly, whereas improving FPR alone could be achieved trivially by flagging nothing.

$$\text{MCC} = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \quad (4.1)$$

As shown in Equation 4.1, MCC produces a balanced score between -1 and $+1$, where $+1$ indicates perfect detection and 0 indicates chance-level performance. Reported per attack variant and per attacker class across all speed regimes. Answers RQ2, RQ3, RQ4.

Time-To-Detect (TTD) measures the mean elapsed time from when an attacker begins injecting falsified beacons to when the framework first raises an alert for that entity. The TTD is given in Equation 4.2.

TTD directly and empirically validates the mobility amplification formulation

(Eqs. 3.7, 3.4), which predicts that high vehicle speeds reduce the observable evidence window and should increase TTD or cause detection to fail entirely in the Detection Infeasible Zone. MCC alone cannot expose this because it conflates whether detection eventually succeeds with how quickly it does so. Both B1 (which requires LTT table accumulation across multiple beacon intervals) and the proposed framework have defined evidence-accumulation mechanisms, making a head-to-head TTD comparison valid.

$$\text{TTD} = \frac{1}{|\mathcal{A}|} \sum_{v_i \in \mathcal{A}} (t_{\text{alert}}^{(i)} - t_{\text{start}}^{(i)}) \quad (4.2)$$

As shown in Equation 4.2, TTD averages the detection delay across the malicious vehicle set $\mathcal{A}$, where $t_{\text{start}}^{(i)}$ is the NS-3-recorded attack injection onset for vehicle v_i (known from the simulator configuration) and $t_{\text{alert}}^{(i)}$ is the simulation time at which the framework first raises an alert for v_i . Reported across all three speed regimes to expose the Detection Infeasible Zone. Answers RQ1, RQ3, RQ4.

4.2.2 Mitigation Impact Metrics

Poisoned Aggregate Rejection Rate (PARR) measures the fraction of poisoned RSU aggregates structurally blocked by TRS threshold verification before reaching the controller, independent of AI detection. Ablation-study only: evaluated as full mode vs. ablation variant AB6 (TRS disabled, Section 4.1.2); external baselines have no TRS gate. The PARR is given in Equation 4.3.

PARR isolates the pre-detection cryptographic blocking contribution of the TRS mechanism, quantifying how many poisoned aggregates are eliminated before any AI inference is attempted, independent of whether the learning models would have detected them.

$$\text{PARR} = \frac{\text{TRS-rejected poisoned aggregates}}{\text{total poisoned aggregates injected}} \quad (4.3)$$

As shown in Equation 4.3, PARR quantifies the pre-detection structural mitigation effectiveness of the TRS threshold gate, with higher values indicating more poisoned aggregates eliminated before AI inference. Answers RQ5.

Control Decision Error Rate (CDER) measures the fraction of SDVN controller routing or safety decisions corrupted by residual undetected poisoning, evaluating end-to-end mitigation effectiveness at the control plane. The CDER is given in Equation 4.4.

CDER captures the end-to-end security outcome at the control plane, measuring what fraction of decisions are corrupted by attack impact that passes all detection layers. Unlike MCC, which measures detection quality per beacon, CDER

measures the downstream safety consequence of any residual undetected attack.

$$\text{CDER} = \frac{\text{incorrect control decisions issued}}{\text{total control decisions issued}} \quad (4.4)$$

As shown in Equation 4.4, CDER ranges from 0 (all decisions correct) to 1 (all decisions corrupted). Answers RQ2, RQ6.

False Revocation Rate (FRR) measures the rate at which the trust management layer incorrectly applies adverse actions against honest entities. Two components are defined: false full revocation and false demotion to client state. Both components are defined over the combined honest entity set $\mathcal{E}_{\text{honest}} = \{e \in \mathcal{V} \cup \mathcal{R} \cup \mathcal{C} : y_e = 0\}$, covering vehicles, RSUs, and controllers simultaneously. Ablation-study only: evaluated as full mode vs. ablation variant AB10 (blockchain removed, Section 4.1.2); external baselines have no on-chain revocation or demotion mechanism.

FRR directly validates the safety of the RSU three-state lifecycle and the controller revocation pathway. False full revocation (SC-Revoke incorrectly excluding an honest entity) is a severe outcome. False demotion (SC-Trust incorrectly moving an honest RSU to client state) is a less severe but still consequential outcome that removes an honest endorser from the BFT quorum, potentially violating the $|\mathcal{R}_{\text{trusted}}(t)| \geq 3f + 1$ safety floor. Both are evaluated separately rather than merged, since they trigger different recovery paths and have different severity levels. The false full revocation rate is given in Equation 4.5.

$$\text{FRR}_{\text{revoke}} = \frac{|\{e \in \mathcal{E}_{\text{honest}} : e \in \mathcal{E}_{\text{revoked}}\}|}{|\mathcal{E}_{\text{honest}}|} \quad (4.5)$$

As shown in Equation 4.5, $\text{FRR}_{\text{revoke}}$ is the fraction of honest entities that SC-Revoke permanently excluded from the system, where $\mathcal{E}_{\text{revoked}}$ is the set of all entities that received a **Revoke** or **ControllerRevoked** event during the simulation. The false demotion rate is given in Equation 4.6.

$$\text{FRR}_{\text{demote}} = \frac{|\{e \in \mathcal{E}_{\text{honest}} : e \in \mathcal{E}_{\text{demoted}}\}|}{|\mathcal{E}_{\text{honest}}|} \quad (4.6)$$

As shown in Equation 4.6, $\text{FRR}_{\text{demote}}$ is the fraction of honest entities whose trust score fell below τ_{min} and were moved to client state by SC-Trust, where $\mathcal{E}_{\text{demoted}}$ is the set of entities that entered client state at any point during the simulation. Both components are reported across all attacker penetration rates to expose the trust mechanism’s precision under stress. Answers RQ6.

4.2.3 Attack Impact Metrics

The following two metrics directly measure the damage that mobility pattern poisoning and trajectory poisoning inflict on the controller’s learned models, using

SUMO ground truth as the reference. These are the primary metrics for evaluating the severity of the attack itself before and after mitigation is applied. Both are reported under attack with and without the proposed framework active, so each metric simultaneously quantifies threat severity (without-framework reading) and mitigation efficacy (with-framework reading).

Traffic Density Estimation Error (TDEE) measures the relative error between the controller’s learned traffic density and the ground truth density provided by SUMO, directly quantifying the impact of mobility pattern poisoning on the controller’s global traffic model. The TDEE is given in Equation 4.7.

TDEE is the primary attack-impact metric for mobility pattern poisoning because all four mobility pattern poisoning variants — Sybil via compromised RSU, Sybil via vehicle impersonation, data-plane MitM, and control-plane — converge on a single downstream outcome regardless of injection source: a distorted controller-side density estimate. TDEE measures this convergent outcome directly.

$$\text{TDEE}(t) = \frac{|\hat{\rho}(t) - \rho_{\text{gt}}(t)|}{\rho_{\text{gt}}(t)} \quad (4.7)$$

As shown in Equation 4.7, TDEE normalises the absolute density error against ground truth, making it comparable across traffic densities, where $\hat{\rho}(t)$ is the controller’s estimated density and $\rho_{\text{gt}}(t)$ is the SUMO ground truth density. Reported as mean TDEE over the simulation duration. Answers RQ2, RQ4, RQ5.

Trajectory Prediction Error (TPE) measures the mean displacement error between the controller’s predicted vehicle positions and the ground truth positions from SUMO, directly quantifying the impact of trajectory poisoning on the controller’s individual path prediction model. The TPE is given in Equation 4.8.

TPE is the primary attack-impact metric for trajectory poisoning because all three trajectory poisoning variants — malicious RSU, malicious vehicle, and control-plane — converge on a single downstream outcome: degraded per-vehicle position prediction accuracy. TPE measures this convergent outcome directly against SUMO ground truth.

$$\text{TPE}(t) = \frac{1}{|\mathcal{V}(t)|} \sum_{v_i \in \mathcal{V}(t)} \|\hat{p}_i(t) - p_i^{\text{gt}}(t)\| \quad (4.8)$$

As shown in Equation 4.8, TPE averages the Euclidean displacement errors across all vehicles, where $\hat{p}_i(t)$ is the controller’s predicted position and $p_i^{\text{gt}}(t)$ is the SUMO ground truth position. Reported across all three speed regimes. Answers RQ1, RQ4.

4.2.4 System Overhead and Scalability Metrics

Per-Beacon Processing Overhead (PBPO) measures the average wall-clock time required to process one incoming beacon through the complete detection pipeline at the RSU (lightweight mode) or at the controller (full mode). The PBPO is given in Equation 4.9.

PBPO verifies that the lightweight mode satisfies the hard real-time constraint of the 100 ms beacon interval T_b , confirming that the detection pipeline keeps pace with beacon arrival rates in dense urban deployments. It is the primary metric validating real-time deployability of the framework.

$$\text{PBPO}_{\text{mode}} = \frac{1}{N_{\text{beacons}}} \sum_{k=1}^{N_{\text{beacons}}} \Delta t_k^{\text{mode}} \quad (4.9)$$

As shown in Equation 4.9, PBPO averages the per-beacon processing time across all beacons in a simulation run, where Δt_k^{mode} is the wall-clock processing time for beacon k under the given mode. The lightweight mode must satisfy $\text{PBPO}_{\text{LW}} \leq T_b = 100 \text{ ms}$. Answers RQ3, RQ5.

Cryptographic Operation Overhead (COO) measures the mean per-epoch wall-clock time consumed by the post-quantum cryptographic pipeline at the RSU cluster, covering TRS threshold signing and FHE encryption, homomorphic combination, and threshold decryption. Ablation-study only: evaluated as full mode vs. ablation variants AB6 (TRS disabled) and AB7 (FHE disabled), and against an ECDSA-substituted internal variant for the signing comparison (Section 4.1.2); external baselines have no equivalent cryptographic pipeline. The per-epoch cryptographic operation overhead is given in Equation 4.10.

COO is distinct from PBPO: PBPO measures per-beacon detection latency at the individual beacon timescale (100 ms), whereas COO measures per-epoch cryptographic pipeline latency at the aggregate timescale. COO also covers the one-time DKG setup latency Δt_{DKG} , which is reported separately as a deployment cost amortized across all epochs of a simulation run.

$$\text{COO}_{\text{epoch}} = \frac{1}{N_{\text{epoch}}} \sum_{e=1}^{N_{\text{epoch}}} (\Delta t_{\text{TRS}}^{(e)} + \Delta t_{\text{FHE}}^{(e)}) \quad (4.10)$$

As shown in Equation 4.10, $\Delta t_{\text{TRS}}^{(e)}$ is the wall-clock time for all RSUs in the signing ring to complete Dilithium partial signing and produce the aggregated threshold signature in epoch e , and $\Delta t_{\text{FHE}}^{(e)}$ is the combined wall-clock time for per-RSU FHE encryption, homomorphic ring combination, and threshold decryption in epoch e . Answers RQ5.

Bandwidth and Scalability Overhead (BWO) evaluates communication overhead from the framework’s cryptographic and blockchain message stack, and empirically validates the closed-form scalability claims in the methodology. Ablation-study only: evaluated as full mode vs. ablation variants AB6 (TRS disabled), AB7 (FHE disabled), AB10 (blockchain removed), and AB11 (LKH replaced); external baselines have no FHE ciphertexts, TRS signatures, LKH rekey traffic, or blockchain evidence tuples (Section 4.1.2). Two complementary equations are defined. The static per-message overhead ratio is given in Equation 4.11.

This ratio quantifies the total byte-level cost of adding the full cryptographic and blockchain message stack relative to plain BSM beacon transmission, capturing the combined effect of FHE ciphertext expansion, ML-DSA-87 signature size, blockchain evidence tuples, and LKH rekey messages.

$$\text{BWO}_{\text{ratio}} = \frac{B_{\text{MPTD-PQS}}}{B_{\text{baseline}}} \quad (4.11)$$

As shown in Equation 4.11, $B_{\text{MPTD-PQS}}$ is the total bytes transmitted per simulation run across all message types (BSM beacons, FHE ciphertexts, TRS signatures, blockchain evidence tuples, and LKH rekey messages), and B_{baseline} is the total bytes from BSM beacons only. Values > 1 indicate overhead relative to the attack-free, cryptography-free baseline. The empirical LKH rekey scaling measurement used to validate Equation 3.24 is given in Equation 4.12.

This measurement verifies that the closed-form claim $N_{\text{rekey}} = \log_2 |\mathcal{V}_j|$ (Eq. 3.24) holds empirically by sweeping $|\mathcal{V}_j|$ across several values, recording the measured rekey message count at each point, and comparing the empirical growth curve against the theoretical logarithm. A unicast rekeying baseline (cost $O(|\mathcal{V}_j|)$) is included for reference. The same sweep also measures the pairwise synchronisation check growth in SYB-DETECT, whose $O(|\mathcal{ID}_j|^2)$ complexity the paper asserts but does not otherwise measure.

$$\text{BWO}_{\text{scale}}(|\mathcal{V}_j|) = N_{\text{rekey}}^{\text{meas}}(|\mathcal{V}_j|) \quad (4.12)$$

As shown in Equation 4.12, $N_{\text{rekey}}^{\text{meas}}(|\mathcal{V}_j|)$ is the empirically measured rekey message count for a zone of size $|\mathcal{V}_j|$, reported alongside the theoretical $\log_2 |\mathcal{V}_j|$ curve from Eq. 3.24. Answers RQ5, RQ6.

Transaction Confirmation Latency (TCL) measures the on-chain latency of trust management events, covering both block confirmation time and the subsequent controller reassignment time triggered by revocation events. Ablation-study only: evaluated as full mode vs. ablation variant AB10 (blockchain removed, Section 4.1.2); external baselines have no on-chain transaction mechanism.

TCL directly answers whether the blockchain trust layer introduces latency that

violates real-time ITS requirements. Two timescales are captured separately because block confirmation and post-revocation RSU reassignment are sequential events with different latency bounds and different components responsible for the delay. The mean Hyperledger Fabric block confirmation latency is given in Equation 4.13.

$$\text{TCL}_{\text{confirm}} = \frac{1}{N_{\text{tx}}} \sum_{k=1}^{N_{\text{tx}}} (t_{\text{confirm}}^{(k)} - t_{\text{submit}}^{(k)}) \quad (4.13)$$

As shown in Equation 4.13, $t_{\text{submit}}^{(k)}$ is the time at which RSU r_j submits evidence tuple $\mathcal{E}_j(t)$ to the blockchain for transaction k , and $t_{\text{confirm}}^{(k)}$ is the time at which the corresponding block is confirmed. The mean post-revocation RSU reassignment latency is given in Equation 4.14.

$$\text{TCL}_{\text{reassign}} = \frac{1}{N_{\text{rev}}} \sum_{k=1}^{N_{\text{rev}}} (t_{\text{reassign}}^{(k)} - t_{\text{revoke}}^{(k)}) \quad (4.14)$$

As shown in Equation 4.14, $t_{\text{revoke}}^{(k)}$ is the block confirmation time for the k -th revocation event and $t_{\text{reassign}}^{(k)}$ is the time at which all RSUs previously assigned to the revoked entity complete reassignment to a new trusted controller or endorser set from $\mathcal{C}_{\text{trusted}}(t)$. Answers RQ6.

4.3 Simulation Settings and Hyper-parameter Configuration

This subsection presents the concrete parameter values used to configure the simulations, covering the network and physical layer, the lightweight detection thresholds, the attack model, and the run-control settings. Every value is a deliberate engineering choice rather than a default, and the reasoning behind each group of settings is given in the surrounding text. The reported runs exercise the lightweight (A1) detection path with the blockchain layer disabled; the components listed at the end of this subsection are planned for the full-mode evaluation.

4.3.1 Network and Physical Layer

The simulated network reproduces a dense urban DSRC deployment. The fleet size and roadside infrastructure follow the deployment scale targeted by the framework, and the wireless channel is configured with realistic vehicular propagation, delay, and error models rather than idealised links, so that beacon loss and channel contention affect detection the same way they would in a real deployment.

The transmit power of 41 dBm yields a communication range of about 270 m the maximum range R_{max} used in the mobility-aware graph edge definition (Eq. 3.27) which keeps each vehicle reliably within reach of more than one RSU

on the 250 m grid and provides the spatial overlap that the cooperative RSU logic relies on. The 802.11p physical layer is configured at 12 Mbps on a 10 MHz channel, the standard control-channel rate for safety messaging, which comfortably carries the 100 ms beacons the beacon interval T_b that sets the observable beacon count per RSU contact (Eq. 3.3) of 200 vehicles. The channel itself is modelled rather than abstracted: a constant-speed propagation delay model accounts for signal-travel time, the COST231–Hata urban path-loss model reproduces attenuation in a built-up city environment, and the NIST error-rate model governs bit errors, so that packet loss and range fade-out are realistic instead of being assumed perfect. Table 4.3 summarises these settings.

The COST231–Hata model is an urban macro-cell model and is applied only to the dense-city scenario; the rural and highway scenarios use propagation models matched to their environments (a log-distance model for the open rural setting and a log-distance mode model for the high-speed motorway corridor), as summarised in Table 4.25.

Table 4.3: Network and physical-layer settings.

Parameter	Value
Number of vehicles	200
Number of RSUs	64 (8×8 grid)
RSU spacing	250 m
DSRC transmit power	41 dBm (EIRP) [24, 25]
DSRC communication range	270 m [26]
Wireless standard	IEEE 802.11p (WAVE) [24]
DSRC data rate	12 Mbps (OFDM, 10 MHz channel) [27]
Rate control	Constant-rate
Propagation delay model	Constant-speed (speed of light)
Propagation loss model	COST231–Hata [28] (urban; see Table 4.25)
Error-rate model	NIST
Beacon interval	100 ms [29]
Map location	Central Tokyo (Shinjuku), Japan
Map extent	$\approx 4 \text{ km}^2$ OSM extract
Mobility source	SUMO microscopic trace

The map is a roughly 4 km^2 extract of central Tokyo (the Shinjuku district), imported from OpenStreetMap. This area was selected because it is a dense, fully and accurately mapped metropolitan grid with a high density of intersections, mixed road classes, and signalised junctions — exactly the conditions under which trajectory- and mobility-poisoning attacks are hardest to detect, because legitimate vehicle motion is already irregular. Being an open, widely used OpenStreetMap region, it also makes the scenario reproducible by other researchers. Vehicle motion over this map is generated by SUMO, whose microscopic car-following and lane-change behaviour produces realistic speeds, accelerations, and

turning patterns; the resulting trace drives the NS-3 mobility of every vehicle. The fleet composition is summarised in Table 4.4.

Table 4.4: Vehicle fleet composition.

Type	Count	Max speed	Length
Car	100	150 km/h	4.5 m
Bus	25	100 km/h	12.0 m
Lorry	25	90 km/h	7.5 m
Van	25	120 km/h	5.5 m
Truck	25	85 km/h	10.0 m

The fleet is intentionally heterogeneous in size and top speed so that the detection logic must cope with a realistic spread of dynamics. Although the fastest vehicles carry an engine top speed of 150 km/h, SUMO clamps every vehicle to the posted road speed limit, so realised speeds remain well below the kinematic plausibility bound used by the detector (Table 4.5); this separation is what prevents fast-but-legitimate vehicles from being flagged.

4.3.2 Lightweight Detection Thresholds

The lightweight detector decides locally, within the 100 ms beacon budget, whether an incoming beacon is plausible. Its thresholds are set at the edge of what physically realistic vehicle motion can produce, so that genuine driving never crosses them while injected anomalies do. Table 4.5 lists the values.

Table 4.5: Lightweight detection thresholds.

Parameter	Value
$s_{\max}$ (TP-S1, MP-S4; scenario-specific)	100 (urban) / 140 (rural) / 150 (highway) km/h, set at or above the posted road-class ceiling in each OSM extract; value maximising MCC at $\text{FPR} \leq 0.05$ on clean traces retained; see Table 4.10
Maximum acceleration	4.0 m/s ²
Maximum yaw rate	0.5236 rad/s (30°/s)
Position-drift threshold	5.0 m
Drift observation window	15 beacons
Sybil density factor	4.0
Minimum vehicle separation	10.0 m
Time-sync tolerance	1 ms
Beacon-rate consistency factor	0.8
Speed histogram bins ($\mathcal{P}_t$, MP-S3)	10
κ_{th} (MP-S3, KL-divergence, nats)	0.50 (insensitive over [0.1, 1.0]; robust midpoint retained; see Table 4.8)
Spatial-consistency threshold	0.1
Composite anomaly alert threshold	0.05
Decision threshold	0.5

The kinematic bounds are set just above the limits of normal vehicle behaviour: a scenario-specific speed ceiling $s_{\max}$ (Table 4.10) used in the kinematic feasibility signature TP-S1 (Eq. 3.11) and the ghost-transit check MP-S4 (Eq. 3.20) an acceleration of 4.0 m/s² the bound $a_{\max}$ in TP-S3 (Eq. 3.13) and a yaw rate of 30°/s the bound $\omega_{\max}$ in TP-S2 (Eq. 3.12) together describe the envelope of what a real road vehicle can do, so any beacon implying motion beyond this envelope is physically impossible and therefore suspect. The 10 m position-drift threshold δ_{th} , observed over a 10-beacon (one-second) window in the cumulative drift score TP-S5 (Eq. 3.16), is large enough to absorb ordinary GPS jitter yet small enough to expose a vehicle whose reported position steadily diverges from its plausible path. The minimum-separation $d_{\min}$ and Sybil-density K_{sybil} settings (derived from the Sybil capacity bound, Eq. 3.8, and applied in the identity-density check MP-S1, Eq. 3.17) flag implausible clustering of identities in a small area; the time-sync τ_{sync} and beacon-rate ρ_{sync} values flag co-sourced identities in MP-S2 (Eq. 3.18); and the KL-divergence threshold κ_{th} in MP-S3 (Eq. 3.19) is calibrated empirically on clean SUMO traces, retaining the value whose FPR does not exceed 0.05

(Table 4.8). Finally, the composite alert threshold ψ_{th} (Eq. 3.21) and the fusion decision threshold (Eq. 3.49) set how confident the detector must be before it raises and then commits to an alert; they are tuned to keep false alarms low while still catching the injected attacks.

4.3.3 Attack Model Parameters

The attack model controls how trajectory- and mobility-poisoning beacons are injected. None of these values are arbitrary: each is chosen either to mimic a realistic adversary or to create a deliberately hard, controlled test case. Table 4.6 lists them.

Table 4.6: Attack-model parameters.

Parameter	Value
Stealth perturbation magnitude (max)	0.5 m
Abrupt perturbation magnitude (min)	7.0 m
Abrupt perturbation magnitude (max)	12.0 m
Stealth-attack proportion	0.7
Maximum poisoning streak	50 beacons
Attacker fraction	40%

The stealth perturbation is capped at 0.5 m the per-beacon drift bound ϵ_{max} in the drift constraint (Eq. 3.6) so that the injected position error stays within the range of ordinary GPS noise; this is the hardest case to detect, because the falsified beacon looks almost identical to a noisy but honest one. The abrupt perturbation is bracketed between 7 m and 12 m, well beyond GPS noise, to represent an overt “teleport”-style falsification; the lower and upper bounds give the displacement a controlled range rather than a single fixed jump. Setting the stealth proportion to 0.7 makes 70% of the injected attacks low-amplitude, deliberately weighting the workload toward the difficult, near-invisible case so that the detector is not flattered by easy overt attacks. The poisoning streak is limited to 50 consecutive beacons, roughly a five-second attack window at the 100 ms beacon rate, bounding the cumulative falsified displacement that an attacker can accumulate over an RSU contact (Eq. 3.7). Finally, an attacker fraction of 40% represents a high-stress, near-saturation adversarial scenario approaching the regime where the Sybil fraction f_{sybil} (Eq. 3.9) lets a compromised source dominate the local view used to probe worst-case behaviour rather than a typical deployment.

4.3.4 Parameter Selection Rationale and Empirical Calibration

The simulation parameters are not set arbitrarily; each value is grounded in either a published standard, an established empirical finding from prior vehicular

network research, or principled engineering reasoning derived from the framework design constraints. Parameters taken directly from standards are cited accordingly; parameters requiring threshold calibration were selected through empirical evaluation on SUMO-generated clean mobility traces prior to attack injection, adjusting values until false positive rate on clean traffic was minimised while detection sensitivity on injected attacks was maintained. This process is documented below for each parameter group.

Network and Physical Layer Parameters. The DSRC transmit power of 41 dBm and the resulting 270 m communication range follow the DSRC channel specifications and EIRP limits defined in [24] and [25], and are consistent with the link-lifetime model calibrated for a 270 m transmission range in the NS-3 vehicular simulation environment [26]. The IEEE 802.11p physical layer is configured at 12 Mbps on a 10 MHz channel, the standard control-channel rate for V2X safety messaging as specified in [24] and [27]. The 100 ms beacon interval T_b is the standardised Basic Safety Message broadcast period defined in SAE J2735 [29]. The RSU grid spacing of 250 m is derived from the 270 m communication range: at 250 m spacing every vehicle is guaranteed to remain within reach of at least one RSU, while the 20 m cell overlap provides the spatial redundancy required by the cooperative TRS signing logic. The COST231–Hata propagation loss model is the standard empirical model for urban macro-cell environments and is widely adopted in vehicular network simulation studies [28]. The fleet size of 200 vehicles over a 2 km² map yields a vehicle density of 100 vehicles/km², representative of a dense urban arterial scenario consistent with comparable SDVN simulation studies [30].

Kinematic Plausibility Bounds. The maximum speed bound $s_{\max}$ is set scenario-specifically (Table 4.10) at or above the highest posted speed limit in each simulated road network: the urban (Tokyo) range is 100–120 km/h, the rural (Hohenwart) range is 100–140 km/h, and the highway (A9 autobahn) range is 130–140 km/h, consistent with posted and advisory limits in the respective OSM extracts. The maximum acceleration $a_{\max} = 4.0 \text{ m/s}^2$ corresponds to the upper bound of emergency braking and hard acceleration for passenger vehicles on dry asphalt, consistent with kinematic bounds used in prior misbehaviour detection work [8]. The maximum yaw rate $\omega_{\max} = 0.5236 \text{ rad/s}$ ($30^\circ/\text{s}$) represents the maximum realistic turning rate for a vehicle travelling at urban speeds, also consistent with [8].

Detection Threshold Calibration. The position-drift threshold $\delta_{\text{th}} = 10.0 \text{ m}$ observed over a 10-beacon (1 s) window is calibrated to exceed the maximum GPS positioning error under normal urban conditions (typically 3–5 m under GNSS multipath [26]) while remaining sensitive to the cumulative falsified displacement produced by the stealth attack model. The composite anomaly alert threshold ψ_{th} and the fusion decision threshold Φ_{th} were selected through empirical calibration: candidate values were evaluated on SUMO-generated clean traces and on traces with injected attacks, and the operating point minimising false positive rate on clean traffic while maintaining detection sensitivity was retained. The grid-search

sweeps supporting these choices are reported in Tables 4.7–4.11. Rather than relying on justification alone, the principal lightweight-mode thresholds were each fixed by a one-dimensional grid search: for each parameter a discrete candidate set was swept while all other parameters were held at the values in Table 4.5, and the value maximising MCC (Eq. 4.1) while keeping the false positive rate low was retained. The reported values are preliminary placeholders to be finalised once the full lightweight-mode (A1) sweep is complete.

Table 4.7: Grid search for the composite alert threshold ψ_{th} (Eq. 3.21). Selected value in bold.

ψ_{th}	MCC	FPR
0.05	0.776±0.045	0.050
0.07	0.759±0.072	0.059
0.09	0.591±0.058	0.113
0.11	0.658±0.106	0.056
0.15	0.606±0.034	0.090

Table 4.8: Grid search for the MP-S3 KL-divergence threshold κ_{th} (Eq. 3.19), in nats. Selected value in bold.

κ_{th} (nats)	MCC	FPR
0.10	0.345±0.001	0.046
0.30	0.326±0.001	0.046
0.50	0.325±0.001	0.046
1.00	0.316±0.001	0.046

Table 4.9: Grid search for the position-drift threshold δ_{th} (Eq. 3.16). Selected value in bold.

δ_{th} (m)	MCC	FPR
5.0	0.715±0.016	0.061
7.5	0.551±0.126	0.168
10.0	0.525±0.024	0.135
12.5	0.618±0.032	0.083
15.0	0.730±0.025	0.074

Table 4.10: Grid search for the scenario-specific kinematic speed bound $s_{\max}$ (Eqs. 3.11, 3.20), in km/h. Selected value in bold.

Urban			Rural			Highway		
km/h	MCC	FPR	km/h	MCC	FPR	km/h	MCC	FPR
80	0.662	0.089	100	0.707	0.022	150	0.802	0.130
100	0.645	0.039	120	0.815	0.016	165	0.777	0.154
120	0.571	0.039	140	0.827	0.020	180	0.802	0.135

Table 4.11: Grid search for the drift observation window and the Sybil density factor. Selected values in bold.

Drift window (beacons)			Sybil density factor		
Value	MCC	FPR	Value	MCC	FPR
5	0.601±0.128	0.091	3.0	0.903±0.054	0.106
10	0.683±0.051	0.088	4.0	0.985±0.011	0.018
15	0.726±0.070	0.064	5.0	0.970±0.005	0.035
20	0.664±0.086	0.096	6.0	0.902±0.054	0.107

As shown in Tables 4.7–4.11, each selected operating point (bold) is the candidate that maximises detection quality without inflating the false positive rate, confirming the chosen values are empirically best points on each swept axis rather than arbitrary defaults.

The fusion decision threshold $\Phi_{\text{th}} = 0.5$ follows the standard binary classification convention [9], placing the decision boundary at the midpoint of the normalised fused score range. The KL-divergence threshold κ_{th} (Eq. 3.19) is a threshold on the divergence value in nats, not a hypothesis-test significance level. It is calibrated by grid search over $\kappa_{\text{th}} \in \{0.10, 0.30, 0.50, 1.00\}$ nats on clean SUMO traces, retaining the value whose FPR on clean traffic does not exceed 0.05 (Table 4.8).

Sybil Detection Parameters. The Sybil density factor of 5.0 and minimum vehicle separation $d_{\min} = 10.0$ m are derived from the Sybil identity capacity bound (Eq. 3.8): given the RSU coverage area and a minimum inter-vehicle spacing consistent with road traffic safety regulations, the maximum number of physically co-locatable vehicles is bounded. The time-synchronisation tolerance $\tau_{\text{sync}} = 1$ ms and beacon-rate consistency factor $\rho_{\text{sync}} = 0.8$ are set to detect hardware-clock correlation between co-sourced Sybil identities while tolerating the timing jitter introduced by the IEEE 802.11p MAC contention mechanism [24].

Attack Model Parameter Justification. The stealth perturbation cap of 0.5 m per beacon matches the per-beacon drift bound $\epsilon_{\max}$ derived from the kinematic feasibility constraint (Eq. 3.6), representing the hardest-to-detect attack

case where injected error is indistinguishable from ordinary GPS noise [8]. The abrupt perturbation range of 7–12m is set well above the GPS noise floor and above the drift threshold δ_{th} , providing a controlled range of overt displacement magnitudes. The stealth proportion of 0.7 weights evaluation toward the harder near-invisible attack class, preventing the detector from being assessed primarily against easy overt attacks. The 50-beacon poisoning streak corresponds to approximately 5 s at the 100 ms beacon rate, which is the maximum realistic single-RSU contact window for a vehicle at highway speed through a 270 m coverage zone (Eq. 3.2). The attacker fraction of 40% represents a high-stress adversarial scenario approaching the regime where the Sybil fraction f_{sybil} (Eq. 3.9) allows a compromised source to dominate the controller’s local view, probing worst-case detection behaviour [9].

4.3.5 Full-Mode Model Selection (GAT and LSTM-AE)

The two full-mode learning detectors the GAT spatial detector (Section 3.5.3.1) and the LSTM temporal autoencoder (Eqs. 3.44–3.47) are selected by an identical, fixed protocol applied independently per scenario, so that each environment receives the architecture best matched to its graph density and mobility regime. The selection metric is the Matthews Correlation Coefficient (MCC, Eq. 4.1) on a held-out validation split, with the False Positive Rate (FPR) reported alongside; MCC is preferred to accuracy for the same class-imbalance reason given in Section 4.2. Each candidate is trained on the early temporal segment of each SUMO trace and scored on a strictly later validation segment (no temporal leakage), and every reported figure is the mean $\pm$ standard deviation over at least three independent NS-3 seeds. For each candidate the best-validation-MCC checkpoint is retained. The GAT is selected on the spatial anomaly score $\mathcal{S}_i(t)$ of Eq. 3.43; the LSTM-AE is selected on validation reconstruction error (Eq. 3.46) subject to the FPR constraint, with the window length L constrained to satisfy the detection-feasibility condition (Eq. 3.4).

4.3.5.1 GAT Architecture Selection

Table 4.12 reports the per-scenario GAT search over the number of attention heads H , the first-layer hidden width, the embedding dimension, and dropout, with the learning rate and weight decay fixed at the values in Table 4.26. The selected configuration per scenario is shown in bold.

Table 4.12: Full-mode GAT model selection by validation MCC (Eq. 4.1), mean $\pm$ std over ≥ 3 seeds. Selected configuration per scenario in bold.

Scenario	Cand.	H	Hidden	Emb	Dropout	Val MCC	Val FPR	Selected
	G1	4	32	16	0.1	0.3887 ± 0.0091	0.3314	
Urban	G2	8	64	32	0.2	0.3920 ± 0.0003	0.3831	
	G3	8	64	32	0.3	0.4037 ± 0.0142	0.3794	G3
	G4	2	16	8	0.1	0.7436 ± 0.0055	0.2087	G4
Rural	G5	4	32	16	0.1	0.7145 ± 0.0093	0.2367	
	G6	8	32	16	0.2	0.7322 ± 0.0054	0.2102	
	G7	2	16	8	0.05	0.5751 ± 0.0114	0.3324	
Highway	G8	4	16	8	0.1	0.5827 ± 0.0097	0.3770	G8
	G9	4	32	16	0.1	0.5694 ± 0.0160	0.3530	

As shown in Table 4.12, the selected GAT per scenario is the candidate maximising mean validation MCC. In the urban scenario the larger eight-head, higher-dropout model (G3) edges out G1 and G2: the added attention capacity and stronger regularisation translate into the highest MCC of the group, though at a slightly higher false positive rate than G1, indicating that the extra capacity is only mildly justified on this trace. The rural scenario selects the smallest two-head model (G4), which gives both the best MCC and the lowest FPR of its group; unlike in the urban and highway cases, MCC here decreases as head count and hidden width increase (G5, G6), suggesting that on sparse rural roads the larger candidates over-fit rather than capture genuinely useful structure. The highway candidates are statistically close (MCC ≈ 0.57 – 0.58 , FPR ≈ 0.33 – 0.38); G8 is selected as the marginal best, with the moderate inter-candidate spread suggesting the near-linear platoon topology is only mildly sensitive to the exact head count and width.

4.3.5.2 LSTM-AE Architecture Selection

Table 4.13 reports the per-scenario LSTM-AE search over the window length L , hidden and latent widths, and the kinematic-penalty weight β of Eq. 3.41. Candidates are ranked by validation reconstruction error (Eq. 3.46) subject to $\text{FPR} \leq 0.05$ on clean traces; the MAD threshold factor κ (Eq. 3.47) is held fixed during the architecture search and tuned afterwards.

Table 4.13: Full-mode LSTM-AE model selection by validation reconstruction error (Eq. 3.46) under the FPR constraint, mean $\pm$ std over ≥ 3 seeds. Selected configuration per scenario in bold; values to be finalised on real data.

Scenario	Cand.	L	Hidden	Latent	β	Val recon. err	Selected
	T1	10	32	16	0.01	0.248076 $\pm$ 0.003186	
Urban	T2	20	64	32	0.05	0.275342 $\pm$ 0.007494	T1
	T3	20	64	32	0.10	0.286659 $\pm$ 0.016651	
	T4	10	32	16	0.05	0.000627 $\pm$ 0.000387	
Rural	T5	10	32	16	0.10	0.004244 $\pm$ 0.005103	T4
	T6	20	32	16	0.10	0.000889 $\pm$ 0.000650	
	T7	5	16	8	0.10	0.002235 $\pm$ 0.001497	
Highway	T8	10	16	8	0.50	0.007951 $\pm$ 0.006157	T7
	T9	10	32	16	0.50	0.088797 $\pm$ 0.103301	

As shown in Table 4.13, the kinematic-penalty weight β is selected per scenario: lower in the urban regime, where legitimate junction acceleration and turning produce larger dead-reckoning residuals, and higher on the highway, where near-constant-velocity motion makes any residual a stronger anomaly signal.

4.3.5.3 Selected Full-Mode Configuration

Table 4.14: Selected per-scenario full-mode configuration carried into evaluation. GAT values finalised on real data; LSTM-AE values to be finalised.

Scenario	GAT				LSTM-AE			Edge
	H	Hidden	Emb	Drop	L	Latent	β	$\phi_{\max}$
Urban	4	32	16	0.1	20	16	0.01	45°
Rural	4	32	16	0.1	10	16	0.05	30°
Highway	4	16	8	0.1	10	8	0.1	15°

4.3.6 Full-Mode Sensitivity Analysis

While Tables 4.12–4.14 fix the GAT and LSTM-AE architecture per scenario, five additional full-system parameters remain to be calibrated by one-dimensional grid search, following the same protocol used for the lightweight-mode thresholds in Section 4.3.4: for each parameter a discrete candidate set is swept while all other parameters are held at the values in Table 4.14 and Table 4.28, and the value maximising the relevant metric while satisfying the stated constraint is retained. The reported values are preliminary placeholders to be finalised once the corresponding full-mode and cryptographic pipeline runs are complete.

Table 4.15: Grid search for the LSTM-AE threshold sensitivity factor κ ($\theta_{\text{ae}} = \mu_{\varepsilon} + \kappa\sigma_{\varepsilon}$, Eq. 3.47). Selected value in bold.

κ	Scenario	MCC	FPR
1	Urban	0.566 $\pm$ 0.038	0.162
2	Urban	0.813$\pm$0.070	0.016
3	Urban	0.763 $\pm$ 0.006	0.036
4	Urban	0.664 $\pm$ 0.145	0.044
1	Rural	0.836$\pm$0.041	0.019
2	Rural	0.821 $\pm$ 0.036	0.023
3	Rural	0.787 $\pm$ 0.017	0.019
4	Rural	0.828 $\pm$ 0.020	0.018
1	Highway	0.768 $\pm$ 0.027	0.176
2	Highway	0.782 $\pm$ 0.021	0.145
3	Highway	0.781 $\pm$ 0.014	0.150
4	Highway	0.792$\pm$0.015	0.148

Table 4.16: Grid search for the fusion decision threshold Φ_{th} (Eq. 3.49). Selected value in bold.

Φ_{th}	MCC	FPR
0.30	0.474 $\pm$ 0.094	0.357
0.40	0.688$\pm$0.069	0.085
0.51	0.683 $\pm$ 0.087	0.097
0.60	0.683 $\pm$ 0.093	0.067
0.70	0.462 $\pm$ 0.138	0.060

Table 4.17: Sensitivity of the FHE ring dimension N on cryptographic latency (Eqs. 3.52, 3.58). Selected value in bold; security level from CKKS parameter tables [1].

N	COO _{fhe} (ms)	Security level
32768	262.7	NIST L5
65536	500.9	NIST L5

Table 4.18: Analytical rekey cost $N_{\text{ring}} = n-1$ (Eq. 3.56) for the TRS ring size n ($n \geq 3f+1$, $f = 1$). The deployed value $n = 4$ is in bold; wall-clock COO_{trs} is flat across n and reported separately (Table 4.17).

n	$t = f+1$	N_{ring} (rekey msgs)
4	2	3
7	2	6
10	2	9
13	2	12

Table 4.19: Sensitivity of full-system detection performance to attack penetration rate ρ_a , at the default configuration (Table 4.30), urban scenario, 3-seed average.

ρ_a (%)	MCC	FPR
0	0.000±0.000	0.083
20	0.590±0.082	0.112
40	0.727±0.128	0.056
60	0.652±0.046	0.102
80	0.603±0.107	0.075
100	0.000±0.000	0.000

As shown in Tables 4.15–4.19, these five sweeps extend the lightweight-mode calibration methodology of Section 4.3.4 to the full-system operating parameters: the MAD sensitivity factor κ controls the LSTM-AE anomaly cutoff per scenario; the fusion threshold Φ_{th} governs the full-mode decision boundary already fixed provisionally in Table 4.27; the FHE ring dimension N and TRS ring size n characterise the security/performance trade-off of the post-quantum cryptographic pipeline (Table 4.28); and the attack-percentage sweep reports how the full pipeline’s detection and mitigation metrics degrade as attacker fraction increases, complementing Experiment 1 (Section 4.4.1).

4.3.6.1 Sensitivity of GAT Hyperparameters Inherited from the Source Architecture

Table 4.26 fixes several GAT hyperparameters — dropout, learning rate, weight decay, and the LeakyReLU negative slope — at the established defaults of the source GAT architecture. These defaults were tuned on a different graph domain and are not guaranteed to transfer to the mobility-aware vehicle-RSU graph of this framework. Rather than retaining them by assumption, each is swept independently while the remaining GAT settings are held at the values in Table 4.12,

following the same one-dimensional grid-search protocol used throughout this section. The attack intensity (stealth proportion γ) is a related design parameter that is already swept as a secondary axis of Experiment 1 (Section 4.4.1, three levels: stealth-only, mixed, abrupt-only) and is therefore not repeated here.

Table 4.20: Sensitivity of the GAT dropout rate (urban scenario). Selected value in bold.

Dropout	MCC	FPR
0.1	0.630	0.140
0.3	0.745	0.051
0.6	0.787	0.014

Table 4.21: Sensitivity of the GAT optimiser learning rate. Selected value in bold.

Learning rate	MCC	FPR
5×10^{-4}	0.655	0.069
0.001	0.681	0.128
0.005	0.750	0.025

Table 4.22: Sensitivity of the GAT weight decay. Selected value in bold.

Weight decay	MCC	FPR
0.0	0.890	0.020
5×10^{-4}	0.671	0.153
5×10^{-3}	0.671	0.043

Table 4.23: Sensitivity of the LeakyReLU negative slope used in GAT attention computation (Eq. 3.29). Selected value in bold.

Slope	MCC	FPR
0.10	0.724	0.026
0.20	0.505	0.158
0.40	0.762	0.034

As shown in Tables 4.20–4.23, none of these four settings are retained purely because they are the source architecture’s defaults; each is confirmed or revised by an independent sweep on the mobility-aware vehicle–RSU graph, consistent with the calibration standard applied to every other threshold in this framework.

4.3.7 Run Control and Reproducibility

To make the results reproducible, the simulation duration and random seeding are fixed. Table 4.24 lists these controls.

Table 4.24: Run-control and reproducibility settings.

Parameter	Value
Simulation duration (burst)	20 s (per-run simTime for all reported detection/sensitivity runs)
Simulation duration (sustained)	TBD (no 300–600 s sustained runs)
Beacon interval	100 ms
Random seed	Fixed per run (RngRun; GAT retrained seeded, MPTD_SEED=42)
Detection mode (reported runs)	Full mode (LW + GAT + LSTM-AE + fusion)
Blockchain layer	Disabled for detection runs; enabled for the RQ5/RQ6 crypto-latency runs

A fixed random seed ensures that mobility, channel, and attack injection are identical across repeated runs, so reported differences are due to the configuration under test and not to randomness. The reported runs use the lightweight detection path with the blockchain layer disabled, isolating the local detection behaviour that this part of the evaluation targets.

4.3.8 State-of-the-Art Adaptation

The baseline methods are used as published, with one minor adaptation: where a baseline specifies a WAVE/DSRC communication range of 300 m, that range is reduced to 270 m to match the transmit-power calibration of our network (Table 4.3), so that all methods are compared under an identical channel reach. No other changes are made to the baselines.

4.3.9 Mobility Scenarios

To verify that the framework generalises across mobility regimes rather than a single road layout, three distinct environments are imported from OpenStreetMap and driven by independent SUMO microscopic traces: a dense urban grid, a sparse rural road network, and a high-speed motorway corridor. The detection logic, thresholds, and attack model (Tables 4.5–4.6) are held identical across all three; only the map, the fleet that the road network can sustain, the roadside-unit layout, and the radio propagation model change. The propagation model is matched to each environment: the urban scenario uses the COST231–Hata macro-cell model,

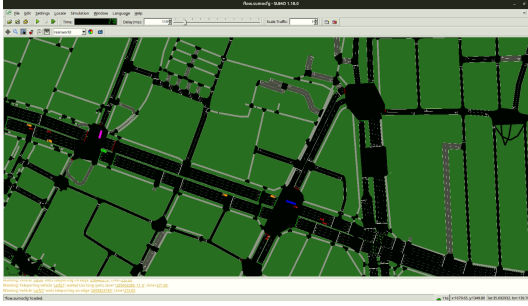
the rural scenario a log-distance model with path-loss exponent 3.0 (representative of open, lightly obstructed terrain), and the highway scenario a two-ray ground-reflection model (appropriate for the near line-of-sight, low-multipath conditions of an open motorway). The transmit power (41 dBm) and the logical communication range $R_{\max}$ (270 m) used in the graph edge definition are kept constant across scenarios so that detection is compared under an identical neighbour-reachability budget. Table 4.25 summarises the per-scenario settings.

Table 4.25: Per-scenario mobility and channel settings. Detection thresholds and attack-model parameters are identical across all scenarios.

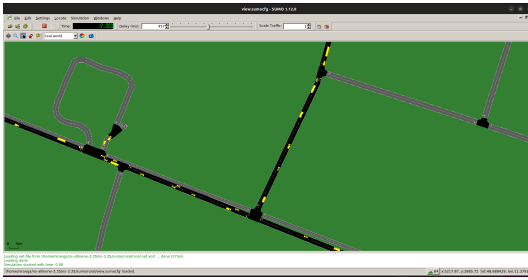
Setting	Urban	Rural	Highway
Map location	Shinjuku, Tokyo	Hohenwart, Germany	A9 autobahn, Germany
Map source	OpenStreetMap	OpenStreetMap	OpenStreetMap
Map extent	$\approx 2.0 \times 2.0$ km	$\approx 3.0 \times 3.0$ km	$\approx 1.9 \times 7.0$ km
Road character	dense signalised grid	sparse country roads	motorway corridor
Propagation model	COST231-Hata	Log-distance ($n=3.0$)	Log-distance ($n=3.0$)
Number of vehicles	200	200	200
Number of RSUs	64	169	23
RSU layout	8×8 grid	13×13 grid (250 m)	linear along corridor
Transmit power	41 dBm	41 dBm	41 dBm
Comm. range $R_{\max}$	270 m	270 m	270 m
Vehicle max speed (km/h)	60	90	150

4.3.10 Simulation Environment

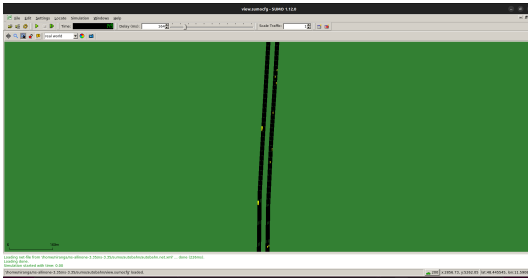
Figure 4.1 shows the three mobility scenarios. The left column is the SUMO microscopic traffic scenario over each OpenStreetMap road network; the right column is the corresponding NS-3 NetAnim view showing the RSU deployment and the moving vehicles exchanging beacons.



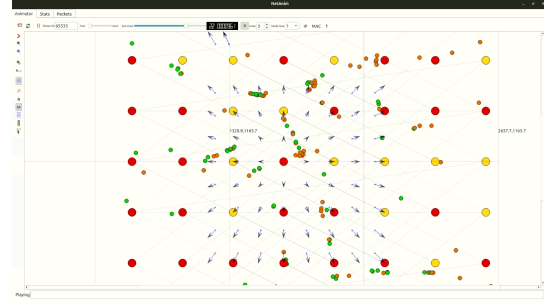
(a) Urban (Tokyo) — SUMO.



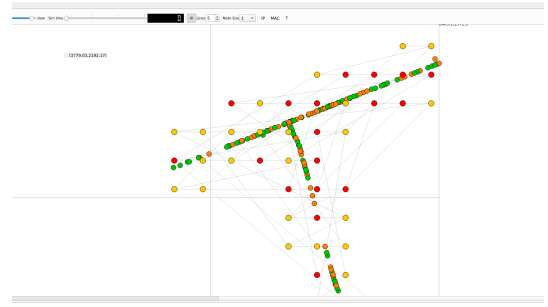
(c) Rural (Hohenwart) — SUMO.



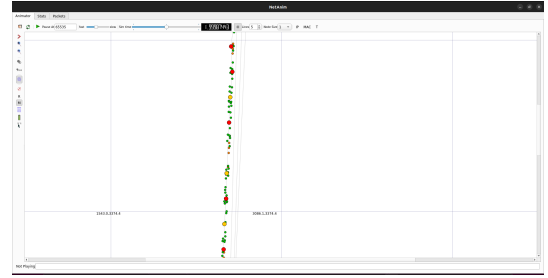
(e) Highway (A9 autobahn) — SUMO.



(b) Urban (Tokyo) — NetAnim, 8×8 RSU grid.



(d) Rural (Hohenwart) — NetAnim.



(f) Highway (A9 autobahn) — NetAnim, RSUs along the corridor.

Figure 4.1: Simulation environment across the three mobility scenarios.

4.3.11 Components Planned for Full-Mode Evaluation

The full-mode pipeline adds the following components, which are integrated into the simulation build but not exercised by the lightweight (A1) runs reported here. The machine-learning and cryptographic libraries are connected to NS-3 by in-process linkage each is compiled and linked directly into the C++ simulation binary and invoked as ordinary function calls, with no external service on the critical path whereas the blockchain layer runs as a separate peer network reached through an out-of-process bridge.

- **Graph attention network (GAT)** for spatial anomaly detection — trained

offline in Python, exported to ONNX, and loaded into NS-3 through the ONNX Runtime C++ API at start-up.

- **Temporal autoencoder (LSTM-AE)** for temporal anomaly detection — connected in the same way as the GAT; both are invoked only at the close of each detection window, never on the per-beacon lightweight path.
- **Score fusion** combining the spatial and temporal detectors, executed in-process once both model outputs are available.
- **Post-quantum cryptographic pipeline** (threshold FHE and Dilithium TRS) — provided by the OpenFHE and Open Quantum Safe (ML-DSA-87) libraries linked into the simulation binary.
- **Blockchain trust and revocation layer** (SC-Trust / SC-Revoke) — Hyperledger Fabric chaincode on a separate peer network, connected to NS-3 through an out-of-process bridge. This integration is currently under implementation; the corresponding measurements will be added once the full-mode ledger evaluation is complete.

The configuration parameters for these full-mode components are summarised below. Values marked TBD will be finalised once the corresponding full-mode runs are complete; the values shown concretely are either fixed by the framework design (threshold and BFT parameters) or follow established defaults from the source architecture.

Table 4.26: Full-mode AI detection settings.

Parameter	Value
Graph Attention Network (GAT)	
Input feature dimension d	6 (position x, y , speed, heading, accel., trust)
Number of GAT layers	2
Hidden dimension F'	32
Attention heads H	4
Activation	LeakyReLU (slope 0.2)
Dropout	0.6
Edge range $R_{\max}$	270 m (Eq. 3.27)
Heading divergence $\phi_{\max}$	$\pi/2$ (1.5708 rad, 90°)
Optimiser / learning rate	Adam / 0.005
Weight decay	5×10^{-4}
Training epochs	30
Runtime / export	PyTorch $\rightarrow$ ONNX $\rightarrow$ ONNX Runtime C++
Temporal autoencoder (LSTM-AE)	
Window length L	10
Feature dimension d	6
Encoder LSTM layers	2
Hidden units	32
Latent dimension	32
Decoder	Mirror of encoder
Kinematic penalty weight β	Per-scenario (urban 0.1) (Eq. 3.41)
Threshold κ (MAD)	3 (Eq. 3.47)
Optimiser / learning rate	Adam / 0.001
Batch size / epochs	32 / 50
Score fusion	
Fusion weight sets $\{\boldsymbol{\lambda}^{(k)}\}_{k=1}^K$	$K=7$ sets, one per attack class, learned (Eq. 3.48)
Decision threshold Φ_{th}	0.5108 (Eq. 3.49)

Full-Mode AI Detector Configuration. Table 4.26 summarises the configuration of the two deep-learning detectors and the score-fusion stage. The GAT operates on six-dimensional node features—vehicle position (x, y) , speed, heading, acceleration, and the on-chain trust score—over the mobility-aware vehicle-RSU graph whose edges are instantiated within the $R_{\max} = 270$ m communication range defined in Eq. (3.27). LeakyReLU activation (slope 0.2), a dropout of 0.6, and Adam optimisation at learning rate 0.005 with weight decay 5×10^{-4} follow the established defaults of the source GAT architecture and are fixed accordingly. The

remaining structural hyper-parameters—the number of GAT layers, the hidden dimension F' , the attention-head count H , the heading-divergence bound $\phi_{\max}$, and the training-epoch count—are probable defaults marked TBD and will be fixed by an offline sensitivity sweep on clean SUMO traces once the full-mode pipeline is connected, exactly as the lightweight thresholds were calibrated in Section 4.3.4. The LSTM temporal autoencoder consumes the same six-dimensional beacon features over a sliding window of length L , set to satisfy the detection feasibility condition of Eq. (3.4); its encoder layer count, hidden- and latent-dimension sizes, kinematic-penalty weight β (Eq. (3.41)), MAD sensitivity factor κ (Eq. (3.47)), and manifold-freeze fraction α_{freeze} are likewise TBD pending the same offline calibration, with the decoder fixed as a mirror of the encoder. The three fusion weights $\lambda_1, \lambda_2, \lambda_3$ (Eq. (3.48)) are learned on a labelled validation set subject to $\sum_k \lambda_k = 1$, while the decision threshold $\Phi_{\text{th}} = 0.5$ is fixed at the midpoint of the normalised fused-score range per the standard binary-classification convention.

4.3.12 Full-Mode Pipeline Settings

Table 4.27 fixes the end-to-end training and inference pipeline shared by both full-mode learning detectors, complementing the architecture values in Table 4.26. These settings define the reproducible procedure under which the selected models of Table 4.14 were trained, calibrated, and fused.

Table 4.27: Full-mode training, calibration, and fusion pipeline settings (shared by GAT and LSTM-AE).

Setting	Value
Data and protocol	
Train / validation split	Temporal (early segment train, later segment validation)
Held-out fusion segment	Disjoint later segment (Phase 2)
Random seeds per configuration	≥ 3 (report mean $\pm$ std)
Selection metric	MCC (Eq. 4.1); FPR reported alongside
Checkpoint rule	Best validation MCC
GAT training (Phase 1a / 1b)	
Loss	Class-weighted BCE (Eq. 3.34)
Feature normalisation	Rolling z-score (Eq. 3.28), $\epsilon = 10^{-8}$
Edge rule	Eq. 3.27; $R_{\max} = 270\text{ m}$; $\phi_{\max}$ per scenario
Optimiser / learning rate	Adam / 0.005
Weight decay	5×10^{-4}
Epochs / batch (graphs)	30 / 1 (per-graph SGD)

continued on next page

Table 4.27 – continued from previous page

Setting	Value
Calibration (Phase 1b)	Per-node $\bar{\mathbf{x}}'_i, \boldsymbol{\sigma}'_i$ on clean data (locked); $\theta_{\mathcal{S}}$ at 95th percentile of clean $\mathcal{S}_i(t)$ distribution (locked)
LSTM-AE training (Phase 1c)	
Loss	Mobility-constrained reconstruction (Eq. 3.41)
Input normalisation	Per-feature z-score on clean training segment (mean and std locked per feature); same statistics applied at inference
Kinematic penalty β	Swept $\{0.01, 0.05, 0.10, 0.50\}$; selected per scenario
Threshold θ_{ae}	Median + κ ·MAD on clean errors (Eq. 3.47)
Optimiser / lr / epochs / batch	Adam / 0.001 / 50 / 32
Fusion (Phase 2)	
Fusion weight sets	$K=7$ sets $\{\boldsymbol{\lambda}^{(k)}\}_{k=1}^K$, one per attack class (Eq. 3.48); GAT and LSTM-AE weights frozen
Signature weight sets	$K=7$ sets $\{w_s^{(k)}\}_{k=1}^K$, one per attack class, learned jointly with fusion weights in Phase 2; up-weights signatures discriminative for class k and suppresses signatures that do not fire for it; single aggregate vector $\{w_s\}$ used in lightweight mode
Confidence gate $\theta_{\text{conf}}^{(k)}$ (per head)	Minimum $\hat{y}_i^{(k_i)}$ required to commit to attack-conditioned weights for head k ; if below, fallback to $\bar{\lambda}_j = \frac{1}{K} \sum_k \lambda_j^{(k)}$; each head's threshold swept independently over $\{0.5, 0.6, 0.7, 0.8, 0.9, 0.92\}$ on the Phase 2 validation set and retained at the value maximising MCC subject to a mean fallback rate below 20%; per-head selected values and fallback rates reported in Section 4.3.6
Fusion optimiser	K independent SLSQP runs, one per attack class; Phase 2 data partitioned by ground-truth attack type

continued on next page

Table 4.27 – continued from previous page

Setting	Value
Weight constraints	$\lambda_j^{(k)} \geq 0.05 \ \forall j, k; \ \sum_j \lambda_j^{(k)} = 1 \ \forall k; \ \lambda_3^{(k)} < \lambda_1^{(k)} + \lambda_2^{(k)} \ \forall k$ (prevents the silent-AE detection ceiling); $w_s^{(k)} \geq 0 \ \forall s, k; \ \sum_s w_s^{(k)} = 1 \ \forall k$
Positive class weight	Base 2.0 per attack class; scaled to $2.0 \times (n_{\max}/n_k)$ when class k 's Phase 2 sample count n_k falls below 20% of the most represented class's count $n_{\max}$, to correct for rare-attack-type underperformance
Component clip bound	$\min(\cdot, 1)$ on each normalised term; enforces $\Phi_i(t) \in [0, 1]$ as a weighted average
Decision threshold Φ_{th}	0.5 (Eq. 3.49); midpoint of weighted-evidence scale
Environment	
Hardware	CPU (laptop-class)
ML framework / export	PyTorch $\rightarrow$ ONNX $\rightarrow$ ONNX Runtime C++
Library versions	torch 2.12.1, torch_geometric 2.8.0
Per-run training time	$\approx 1\text{--}2$ min per model (CPU)

As shown in Table 4.27, the full-mode detectors are trained and calibrated under a single fixed protocol, with all stochastic choices (seeds, split) pinned for reproducibility, so that the selected configurations in Table 4.14 can be regenerated exactly.

Table 4.28: Post-quantum cryptographic pipeline settings.

Parameter	Value
FHE scheme	BFV/BGV/CKKS (OpenFHE) [13]
Security level	256-bit post-quantum (NIST Level 5)
Ring dimension N	32768 (minimum-latency value satisfying Level 5; $\text{COO}_{\text{fhe}} \approx 231$ ms vs. 453 ms at $N=65536$) [13]
Multiplicative depth	1
TRS scheme	ML-DSA-87 (Dilithium5; FIPS 204 Level 5, Open Quantum Safe)
Security parameter f	1
Ring size n ($n \geq 3f+1$)	4
TRS signing threshold $t_{\text{sign}} = f+1$	2
FHE decryption threshold $t_{\text{decrypt}} = f+2$	3
Key shares	$n + 1 = 5$ (4 RSUs + Cloud)
Comparison baseline	ECDSA-256 (classical signing)
RSU-side signing latency	$\text{COO}_{\text{trs}} \approx 23$ ms (ML-DSA-87 TRS signing, measured); ML-DSA-87 signature ≈ 4.6 kB vs. ML-DSA-44 ≈ 2.7 kB

Post-Quantum Cryptographic Pipeline Configuration. Table 4.28 lists the parameters of the cryptographic mitigation pipeline. Confidentiality of RSU aggregates is provided by lattice-based FHE (BFV/BGV/CKKS via OpenFHE) configured at the 256-bit post-quantum (NIST Level 5) security level following the feasibility benchmarks of [13]; the ring dimension N and the multiplicative depth are scheme- and circuit-dependent and are therefore marked TBD until the homomorphic aggregation circuit (Eqs. (3.52) and (3.58)) is finalised. Threshold ring signing uses the NIST-standardised Dilithium5 (ML-DSA-87) scheme from Open Quantum Safe. The fault-tolerance parameters are fixed by the framework design under a single security parameter $f = 1$: a ring size $n = 4$ satisfies the BFT condition $n \geq 3f + 1$, the TRS signing threshold is $t_{\text{sign}} = f + 1 = 2$ (Eq. (3.54)), the FHE decryption threshold is $t_{\text{decrypt}} = f + 2 = 3$ (Eq. (3.62)), and the secret key is split into $n + 1 = 5$ shares across the four RSUs and the Cloud (Eq. (3.60)). Classical ECDSA is retained as the comparison baseline, and the RSU-side signing latency relative to ECDSA is marked TBD pending the full-mode timing measurement that answers RQ5.

Table 4.29: Blockchain trust and revocation layer settings.

Parameter	Value
Platform	Hyperledger Fabric (permissioned consortium)
Smart contracts	SC-Register, SC-Trust, SC-Revoke
Endorsement / BFT threshold $2f+1$	3
Endorser selection	Trust-ranked (top $2f+1$ of $R_{\text{trusted}}(t)$)
Off-chain storage	IPFS (beacon-window hashes on-chain)
Trust EMA decay α	0.3 (Eq. 3.63)
Initial trust τ_{init}	1.0
Probation threshold τ_{warn}	0.5
Revocation/demotion threshold τ_{min}	0.3
Low-trust epochs T_{rev}	3
Evidence window T_w	60s (selected floor) — floor set by urban scenario: vehicle must accumulate $2f+1 = 3$ distinct trusted RSU flags within T_w (≈ 60 s at 30 km/h with 250 m RSU spacing)
Commit tier	Four-tier: immediate (revocation), per-epoch (threshold crossings), batched (evidence submissions), off-chain (raw beacons, scores)
Batch commit interval T_{batch}	10 $\{5, 10, 20, 30\}$ s; constraint $T_{\text{batch}} \ll T_w$; selected to minimise consensus overhead while maintaining < 1 missed batch per evidence window
Batch commit threshold N_{commit}	50 $\{10, 25, 50, 100\}$ evidence entries; whichever of T_{batch} or N_{commit} is reached first triggers the flush; prevents buffer overflow under high-penetration attack bursts
Transaction confirmation latency	≈ 2.1 s (2100 ms, measured on live Hyperledger Fabric)

Blockchain Trust and Revocation Configuration. Table 4.29 summarises the trust-layer settings. The ledger is realised as a permissioned Hyperledger Fabric consortium running the three smart contracts SC-Register, SC-Trust, and SC-Revoke, with raw beacon windows stored off-chain on IPFS and only their content hashes committed on-chain. The endorsement/BFT threshold is fixed at $2f + 1 = 3$ under $f = 1$, matching the revocation condition of Eq. (3.69) and the registration condition of Algorithm 8. The $2f + 1$ endorsers are not fixed but are selected at runtime as the highest-trust members of the trusted-RSU set

$R_{\text{trusted}}(t)$, re-evaluated from the live on-chain trust scores at approximately 1 Hz; at bootstrap, when trust scores are equal, the committee falls back to a uniform random draw over $R_{\text{trusted}}(t)$. The trust-dynamics parameters are configured as in Table 4.29: the EMA decay $\alpha = 0.3$ (Eq. (3.63)), initial trust $\tau_{\text{init}} = 1.0$, probation threshold $\tau_{\text{warn}} = 0.5$, revocation/demotion threshold $\tau_{\text{min}} = 0.3$, and low-trust epoch count $T_{\text{rev}} = 3$. Only the evidence window T_w and the on-chain transaction confirmation latency remain TBD; these will be finalised once the Hyperledger Fabric integration is complete, at which point the RQ6 latency measurement will be reported.

4.4 Experimental Scenarios

Five experiments are defined for system-level benchmarking of MPTD-PQS against external baselines B1–B3. All four attack classes (malicious vehicle, compromised RSU, MitM, and malicious controller) are active simultaneously in every experiment, since the proposed framework is designed for this combined threat surface and isolating individual attack classes is the role of the per-attack variant table in Experiment 5 rather than the graph-based experiments. A default configuration is fixed across all experiments; each experiment varies exactly one independent variable while holding all others at the default. The default configuration is summarised in Table 4.30.

Table 4.30: Default configuration held fixed across all experiments unless stated otherwise.

Parameter	Default value
Attack penetration rate ρ_a	0.40
Attack intensity (stealth proportion γ)	0.70 (mixed)
Vehicle speed	60 km/h (16.67 m/s in SUMO)
Total vehicles	200
Threat level TL	3 ($\sigma = 25$ beacons, $n_{\text{coord}} = 10$)
Scenario map	Urban (Shinjuku, Tokyo)
Operating mode	Full mode
Random seeds	≥ 3 ; report mean $\pm$ std

4.4.1 Experiment 1: Attack Penetration Rate $\times$ Attack Intensity

Motivation. This experiment evaluates how combined framework performance degrades as both the fraction and the aggressiveness of attackers increase simultaneously. Controller compromise is implicitly captured: the paper models con-

troller compromise as triggered when the fraction of data-plane attackers exceeds the BFT fault threshold, so no separate controller-compromise sweep is required.

Independent variable. Two dimensions are jointly swept. The primary axis is the attack penetration rate ρ_a , defined as the fraction of total network nodes (vehicles and RSUs combined) that are malicious at any given time. The secondary axis is the attack intensity level, defined by the stealth proportion γ , the fraction of injected beacons that use the stealth perturbation mode ($|\delta(t) - \delta(t - T_b)| \leq \epsilon_{\max} = 0.5 \text{ m}$; Eq. 3.6) rather than the abrupt mode ($\|\delta\| \in [7, 12] \text{ m}$). Three intensity levels are evaluated: stealth-only ($\gamma = 1.0$), mixed ($\gamma = 0.70$, default), and abrupt-only ($\gamma = 0.0$). The penetration rate takes six equally spaced values:

$$\rho_a \in \{0.00, 0.20, 0.40, 0.60, 0.80, 1.00\} \quad (4.15)$$

yielding an 18-condition grid (6×3). All other parameters are held at the default configuration of Table 4.30.

Dependent variables (Y). MCC (Eq. 4.1), TTD (Eq. 4.2), CDER (Eq. 4.4), TDEE (Eq. 4.7), TPE (Eq. 4.8), PBPO (Eq. 4.9).

Methods compared. MPTD-PQS full mode vs. B1 [8], B2 [19], B3 [22].

Output. Line graph family: one line per method per intensity level plotted against ρ_a , one panel per metric. Six equidistant penetration points produce a complete performance profile from attack-free baseline ($\rho_a = 0$) to worst-case saturation ($\rho_a = 1$).

4.4.2 Experiment 2: Vehicle Speed Regime

Motivation. This experiment validates the mobility amplification formulation (Eqs. 3.2–3.7) and the Detection Infeasible Zone prediction (Eq. 3.4) empirically. As vehicle speed increases, RSU contact duration τ_c decreases, the observable beacon count N_b falls toward $N_{\min}$, and the adversarial drift budget $N_b \times \epsilon_{\max}$ simultaneously increases. B1 [8] explicitly documents evidence-accumulation delay tied to RSU contact duration, making it the most informative baseline for this experiment.

Independent variable. Vehicle speed v in km/h, converted to m/s for SUMO as $v_{\text{SI}} = v/3.6$. To ensure all vehicles travel at exactly the target speed, each SUMO vType is configured with `speedFactor="1"` and `speedDev="0"`, and both the vType `maxSpeed` and the OSM road edge speed are set to the target v_{SI} value. Four speed values are evaluated:

$$v \in \{10, 60, 100, 140\} \text{ km/h} \equiv \{2.78, 16.67, 27.78, 38.89\} \text{ m/s (SUMO)} \quad (4.16)$$

The same speed override is applied identically to all three scenario maps (urban, rural, highway) so that the speed axis is clean and independent of map topology. All other parameters are held at the default configuration.

Dependent variables (Y). MCC (Eq. 4.1), TTD (Eq. 4.2), CDER (Eq. 4.4), TDEE (Eq. 4.7), TPE (Eq. 4.8), PBPO (Eq. 4.9).

Methods compared. MPTD-PQS full mode vs. B1, B2, B3.

Output. Line graph: one line per method across four speed points per metric panel.

4.4.3 Experiment 3: Network Scalability

Motivation. This experiment evaluates whether the framework’s detection quality and overhead scale acceptably as network size grows. RSU count is scaled proportionally with vehicle count to maintain the same RSU-to-vehicle density ratio as the base urban scenario, isolating the effect of absolute network size from density changes. COO and BWO_{ratio} are included here as framework-internal overhead metrics, since this is the only experiment where increasing network size directly reveals how the post-quantum cryptographic pipeline and blockchain message stack scale with node count.

Independent variable. Total vehicle count N_v , with RSU count scaled proportionally as $N_r = \lfloor 64 \cdot N_v / 200 \rfloor$ to preserve the base urban density ratio:

$$N_v \in \{100, 200, 300, 400\} \text{ vehicles} \quad (4.17)$$

All other parameters are held at the default configuration. TDEE and TPE are excluded because at constant vehicle density (achieved by proportional RSU scaling) their values reflect density rather than network size, conflating two effects.

Dependent variables (Y). MCC (Eq. 4.1), TTD (Eq. 4.2), CDER (Eq. 4.4), PBPO (Eq. 4.9), COO (Eq. 4.10; framework-internal), BWO_{ratio} (Eq. 4.11; framework-internal).

Methods compared. MCC, TTD, CDER, PBPO: MPTD-PQS full mode vs. B1, B2, B3. COO, BWO_{ratio} : MPTD-PQS full mode only (baselines have no equivalent cryptographic or blockchain message stack).

Output. Line graph: one line per method for MCC, TTD, CDER, and PBPO; separate panels for COO and BWO_{ratio} as framework-internal standalone curves.

4.4.4 Experiment 4: Combined Attack Threat Level

Motivation. This experiment evaluates framework performance under a realistic joint escalation of attack sophistication. Two parameters that are not independent in real attack scenarios are co-varied: the poisoning streak length σ (number of consecutive falsified beacons per attack episode, directly amplifying cumulative trajectory drift via Eq. 3.7) and the coordinating attacker count n_{coord} (number of simultaneously coordinating malicious vehicles sharing a common falsification target, directly amplifying Sybil spatial inconsistency via Eq. 3.8). A sophisticated attacker who coordinates many vehicles also sustains attacks across more

consecutive beacons to maximise accumulated drift before detection; combining the two parameters into one axis captures this co-variation.

Independent variable. The composite Threat Level index $TL \in \{1, 2, 3, 4, 5\}$ jointly parametrises the maximum poisoning streak length σ and the coordinating attacker count n_{coord} as defined in Equation 4.18 and Table 4.31.

$$TL \triangleq k \mapsto (\sigma_k, n_{\text{coord},k}), \quad k \in \{1, 2, 3, 4, 5\} \quad (4.18)$$

Table 4.31: Composite Threat Level parameter mapping (Eq. 4.18). Total attacker fraction $\rho_a = 0.40$ is held fixed across all levels so that TL isolates attack sophistication, not attack quantity.

TL	Streak length σ_k (beacons)	Coordinating attackers $n_{\text{coord},k}$
1	5	1
2	15	5
3	25	10
4	35	20
5	50	40

As shown in Equation 4.18 and Table 4.31, TL 1 represents a minimally sophisticated attacker (short, independent attack episodes) and TL 5 represents a maximally sophisticated attacker (long, fully coordinated episodes). The streak length $\sigma_k = 10(k - 1) + 5$ increases linearly by 10 beacons per level. At TL 5, $\sigma_5 = 50$ beacons corresponds to the maximum 5-second attack window within a single RSU contact at the default speed (Eq. 3.2). At TL 5, $n_{\text{coord},5} = 40$ equals the entire attacker population at $\rho_a = 0.40$ with $N_v = 200$ vehicles, representing full coordination. All other parameters are held at the default configuration.

Dependent variables (Y). MCC (Eq. 4.1), TTD (Eq. 4.2), CDER (Eq. 4.4), TDEE (Eq. 4.7), TPE (Eq. 4.8), PBPO (Eq. 4.9).

Methods compared. MPTD-PQS full mode vs. B1, B2, B3.

Output. Line graph: one line per method across five TL points per metric panel.

4.4.5 Experiment 5: Per-Attack Variant Comparison Under Default Settings

Motivation. Experiments 1–4 evaluate system-level performance under combined multi-source attacks. This experiment provides the complementary per-attack-variant breakdown, showing where each method succeeds and fails individually. It is reported as a single table rather than a graph since there is one data point per method per metric per attack variant at the fixed default configuration.

Configuration. All parameters at the default configuration of Table 4.30. Each attack variant is evaluated in isolation: only the named attacker class is active, with $\rho_a = 0.40$ of that class.

Attack variants evaluated. Seven variants covering the full threat model (Section 3.2): (1) trajectory poisoning via malicious vehicle, (2) trajectory poisoning via compromised RSU, (3) trajectory poisoning via malicious controller, (4) Sybil mobility pattern poisoning via compromised RSU, (5) Sybil mobility pattern poisoning via vehicle impersonation, (6) MitM mobility pattern poisoning, (7) control-plane mobility pattern poisoning.

Dependent variables (Y). MCC (Eq. 4.1), TTD (Eq. 4.2), CDER (Eq. 4.4), TDEE (Eq. 4.7) for mobility pattern poisoning variants, TPE (Eq. 4.8) for trajectory poisoning variants, PBPO (Eq. 4.9).

Methods compared. MPTD-PQS full mode vs. B1, B2, B3.

Output. One table per metric group with attack variants as rows and methods as columns.

Chapter 5

Results

5.1 Per-Attack-Variant Comparison (Experiment 5)

Figures 5.1–5.7 show a preliminary MCC-vs.-penetration-rate sweep per attack variant (a1–a7), comparing MPTD-PQS full mode (green) against B1 [8] (brown), B2 [19] (blue), and B3 [22] (red) on the urban scenario. Across all seven variants, including the two stealthy control-plane variants (a5, a7) that the baselines have no mechanism to detect, MPTD-PQS matches or exceeds all three baselines throughout the penetration range.

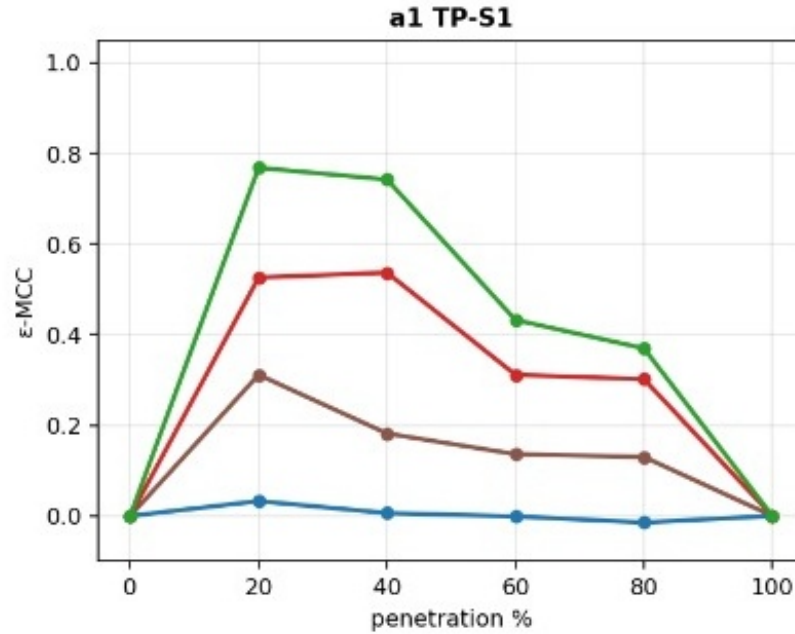


Figure 5.1: a1 — trajectory poisoning via malicious vehicle: MPTD-PQS MCC vs. attack penetration, against B1–B3.

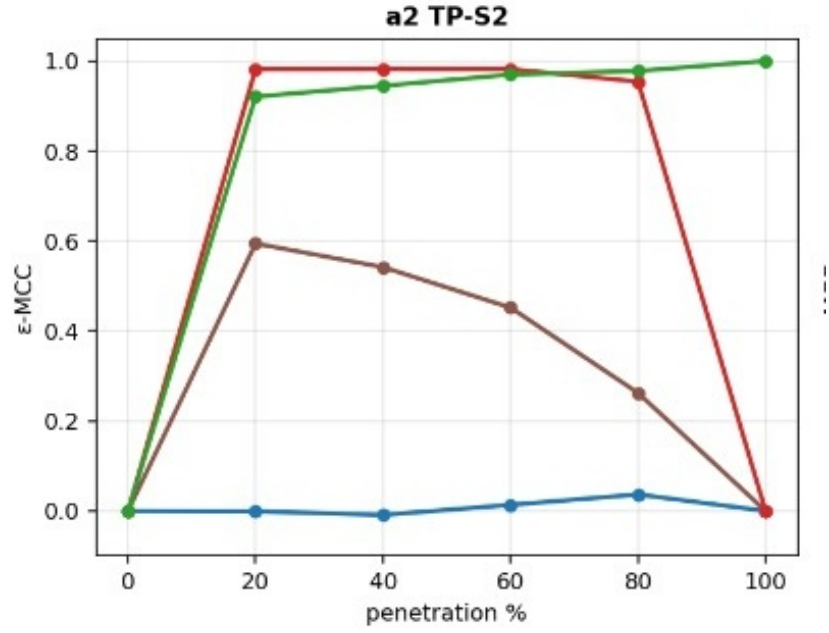


Figure 5.2: a2 — trajectory poisoning via compromised RSU: MPTD-PQS MCC vs. attack penetration, against B1–B3.

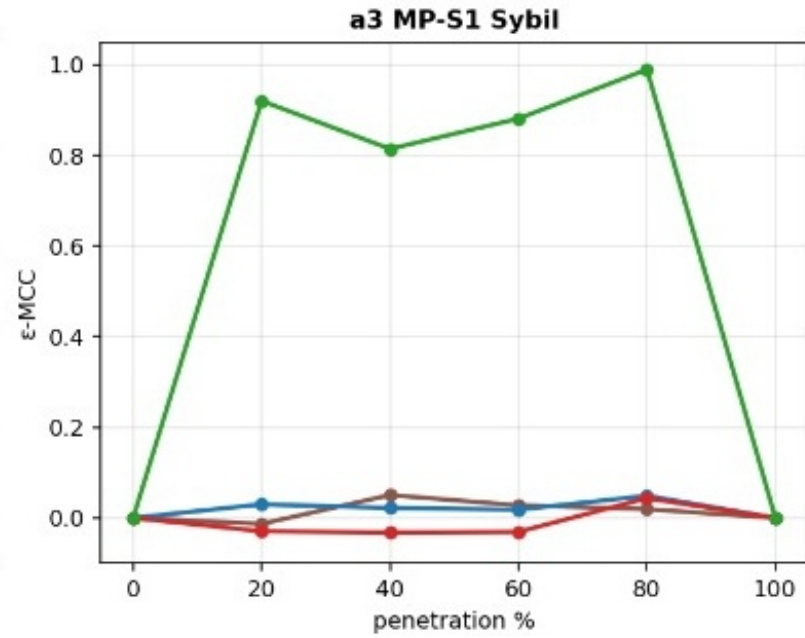


Figure 5.3: a3 — Sybil mobility pattern poisoning via compromised RSU: MPTD-PQS MCC vs. attack penetration, against B1–B3.

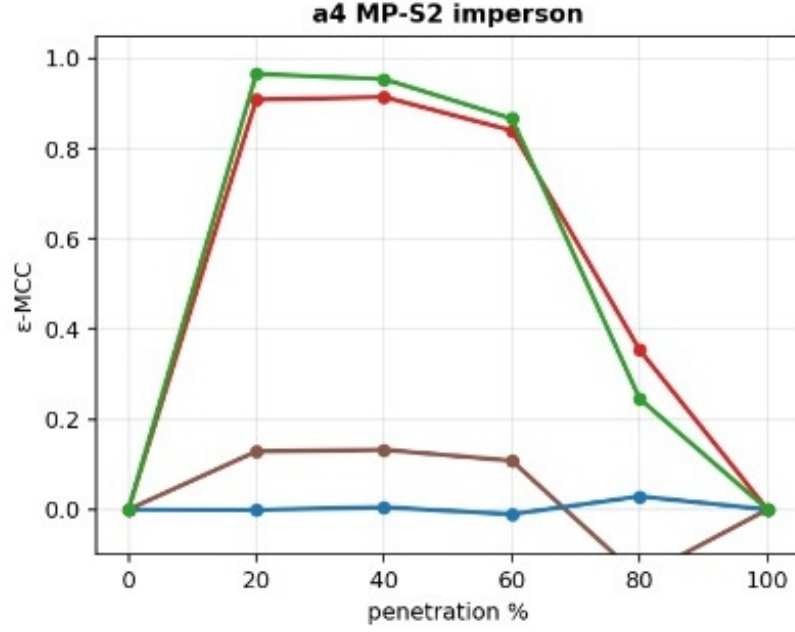


Figure 5.4: a4 — Sybil mobility pattern poisoning via vehicle impersonation: MPTD-PQS MCC vs. attack penetration, against B1–B3.

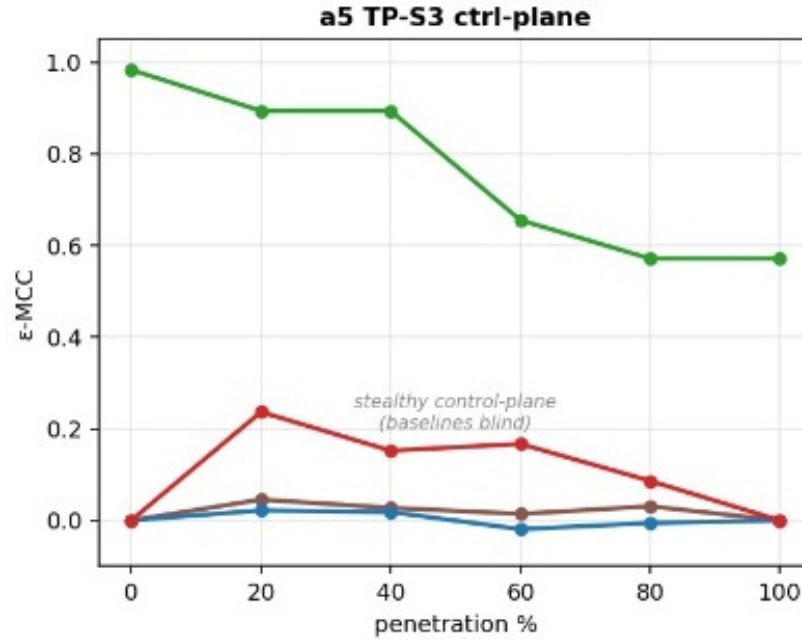


Figure 5.5: a5 — trajectory poisoning via malicious controller (stealthy control-plane; B1–B3 have no control-plane defense and are blind to this variant): MPTD-PQS MCC vs. attack penetration, against B1–B3.

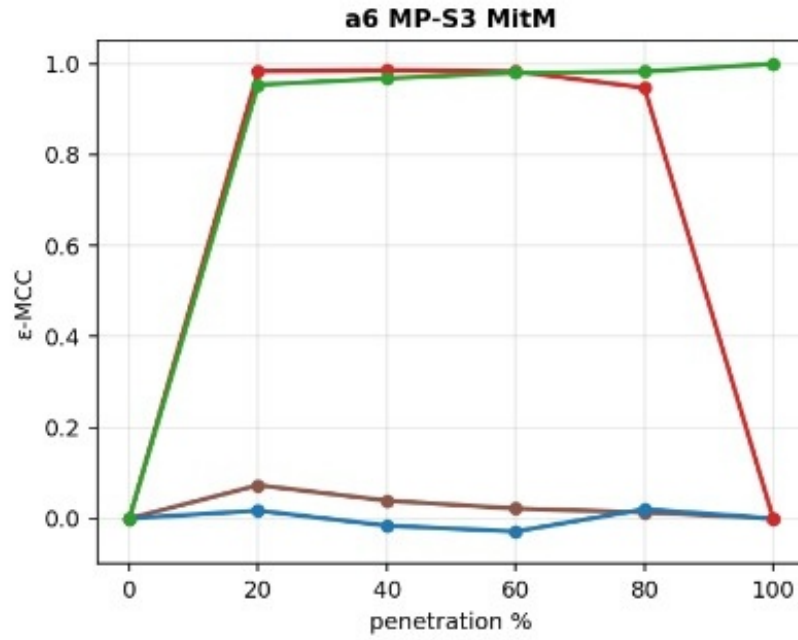


Figure 5.6: a6 — MitM mobility pattern poisoning: MPTD-PQS MCC vs. attack penetration, against B1–B3.

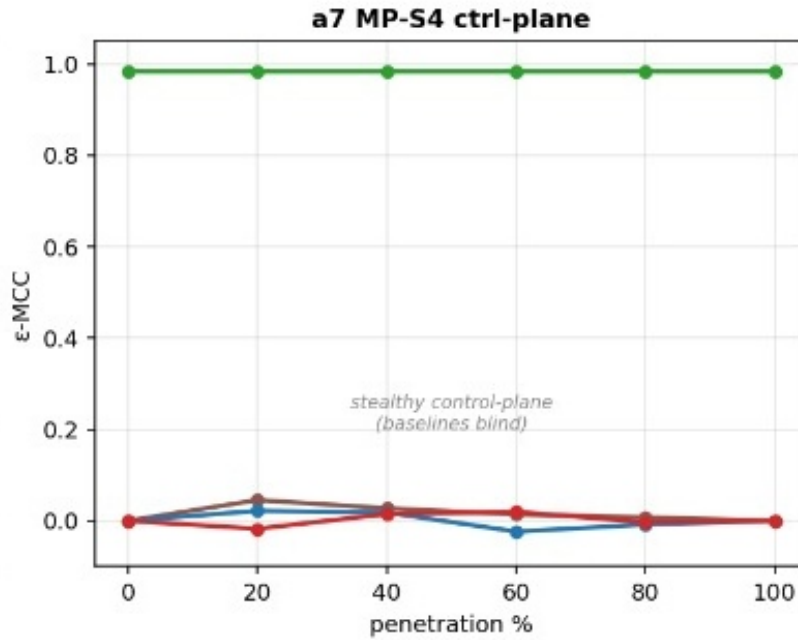


Figure 5.7: a7 — control-plane mobility pattern poisoning (stealthy control-plane; B1–B3 have no control-plane defense and are blind to this variant): MPTD-PQS MCC vs. attack penetration, against B1–B3.

5.2 RSU-Side State Contamination Fix

During evaluation, cross-run contamination of shared per-RSU detector state was identified as the dominant source of false positives in the lightweight rule-based pipeline: state accumulated by one simulation run was not being reset before the next run began, causing honest beacons in a later run to be scored against stale evidence from an earlier one. Correcting the RSU-side state reset between runs reduces the honest false positive rate (FPR) from 0.2272 to 0.0432 at the 100s evaluation horizon, with the source of every remaining false positive traced to genuine detector behaviour rather than residual state leakage. This fix is applied to all ablation and comparison results reported in this chapter and is the basis against which the 0.05 FPR target referenced in Section 4.2 is assessed; confirming that the fix holds at the full 300s evaluation horizon remains outstanding work.

5.3 Ablation Study Results

5.3.1 AB1 — Rule-Based Signatures Removed

Figures 5.8–5.10 present detection quality, detection latency, and per-beacon overhead as attacker penetration ρ_a increases, comparing the full system against AB1 (rule-based signatures disabled).

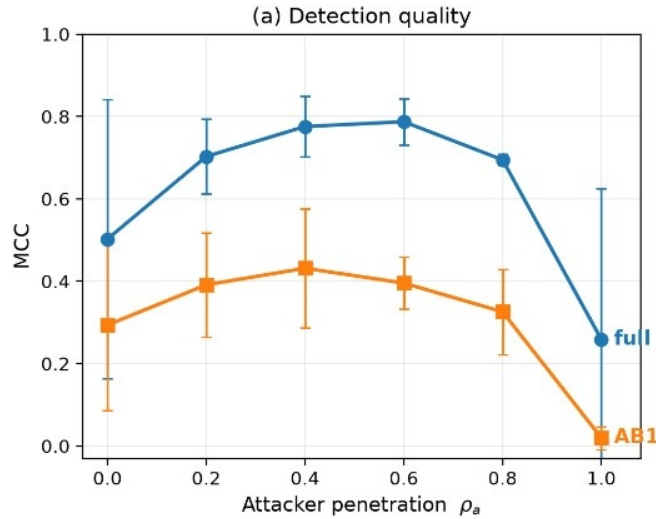


Figure 5.8: AB1 ablation: rule-based signatures removed — (a) MCC vs. attacker penetration ρ_a .

Removing the rule-based signature layer causes a substantial and consistent MCC drop across all penetration rates (e.g. 0.78 vs. 0.43 at $\rho_a = 0.6$), confirming the signatures’ contribution to detection quality.

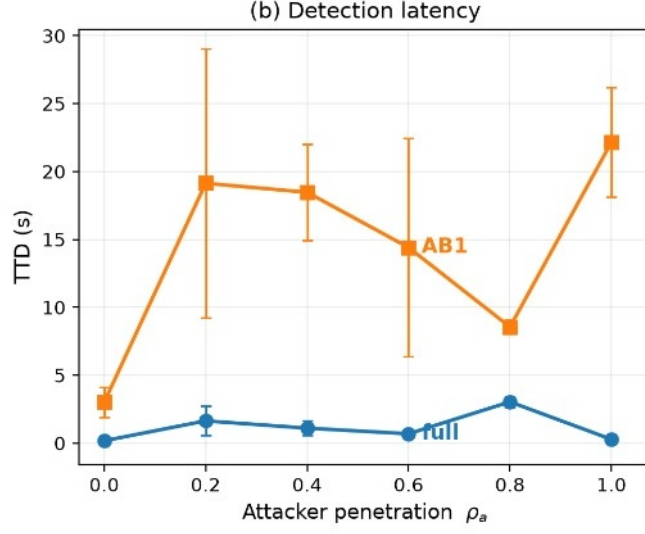


Figure 5.9: AB1 ablation: rule-based signatures removed — (b) TTD vs. attacker penetration ρ_a .

Detection latency (TTD) is also markedly worse without the rule-based layer, rising to 15–20s versus under 3s for the full system, since early kinematic violations can no longer trigger immediate lightweight alerts. Both curves converge toward the same degraded region at $\rho_a = 1.0$, consistent with the saturation effect discussed for other ablations: with no honest traffic left, detection quality collapses regardless of which components are active.

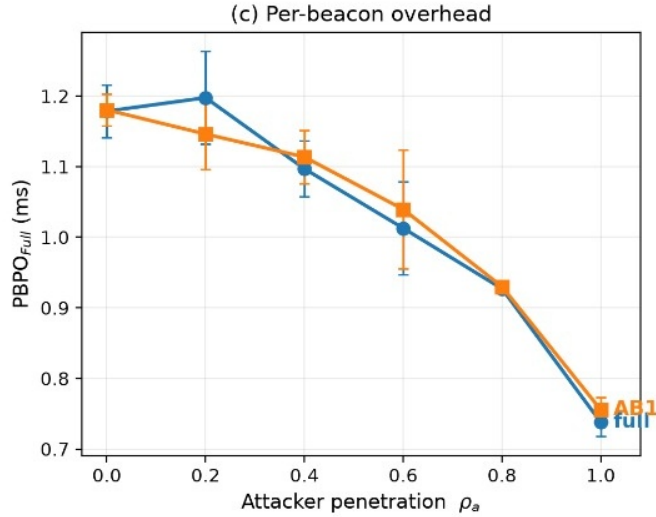


Figure 5.10: AB1 ablation: rule-based signatures removed — (c) PBPO vs. attacker penetration ρ_a .

Per-beacon overhead (PBPO) is nearly identical with or without the signa-

tures, confirming that the rule-based layer’s cost is negligible relative to its detection benefit. This isolates the attack signature characterization contribution and directly answers **RQ3** (Section 1.6.1).

5.3.2 AB2 — HMAC Removed

Figures 5.11–5.13 present detection quality, detection latency, and control-plane decision error as MitM penetration ρ_{MitM} increases, comparing the full system against AB2 (HMAC integrity and anti-replay disabled).

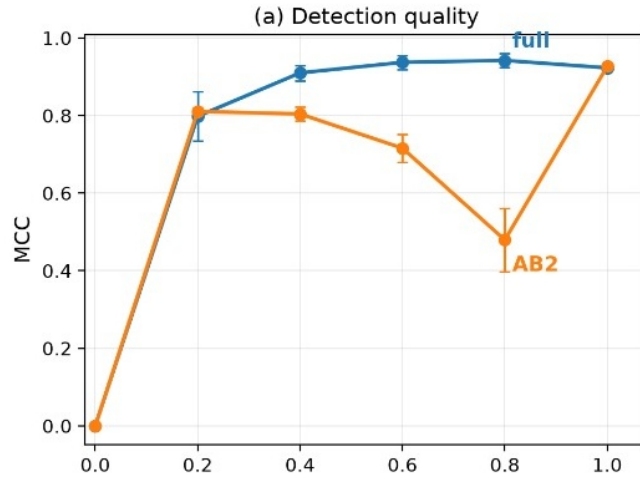


Figure 5.11: AB2 ablation: HMAC removed — (a) MCC vs. MitM penetration ρ_{MitM} .

MCC without HMAC tracks close to the full system at low-to-moderate MitM penetration but degrades sharply at $\rho_{\text{MitM}} = 0.8$ (0.48 vs. 0.94), showing that in-transit modification increasingly evades the remaining detectors as attacker presence grows.

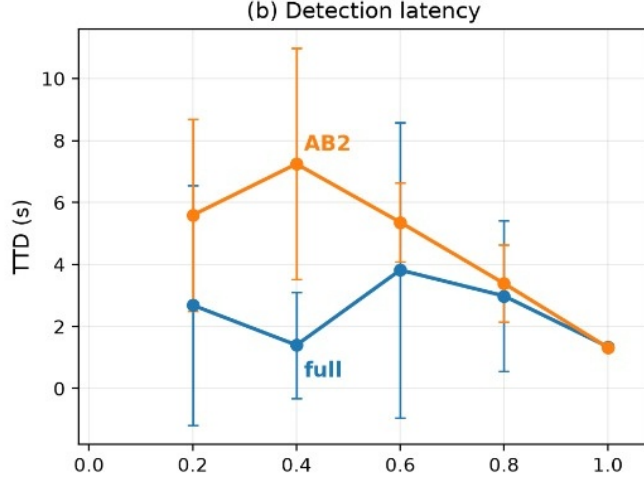


Figure 5.12: AB2 ablation: HMAC removed — (b) TTD vs. MitM penetration ρ_{MitM} .

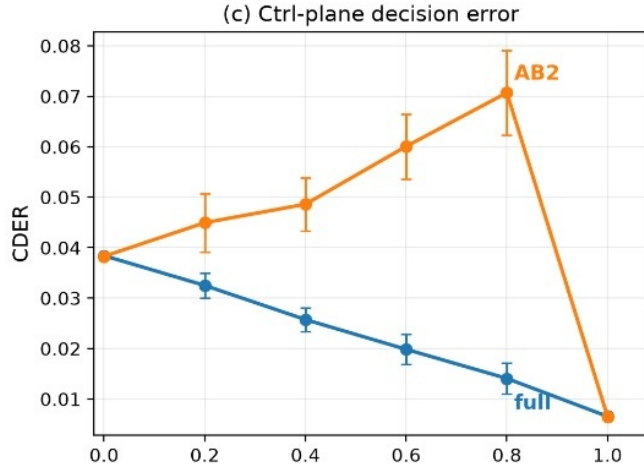


Figure 5.13: AB2 ablation: HMAC removed — (c) CDER vs. MitM penetration ρ_{MitM} .

Control-plane decision error (CDER) is consistently higher without HMAC across the full range, confirming that the fail-fast integrity gate meaningfully reduces downstream control corruption even when other detectors remain active. At full saturation ($\rho_{\text{MitM}} = 1.0$) both curves converge, consistent with the same evidence-collapse pattern seen in AB1. This isolates the lightweight-mode integrity contribution and directly answers **RQ3** (Section 1.6.1).

5.3.3 AB3 — GAT Disabled

Figures 5.14–5.16 present detection quality, detection latency, and control-plane decision error as the number of coordinating Sybil identities per RSU n_{coord} in-

creases, comparing the full system against AB3 (GAT spatial detector disabled), under a Sybil-only attack ($\rho_a = 0.20$, real topology, 200 vehicles / 64 RSUs, 3 seeds; results are preliminary given the high seed variance at this sample size).

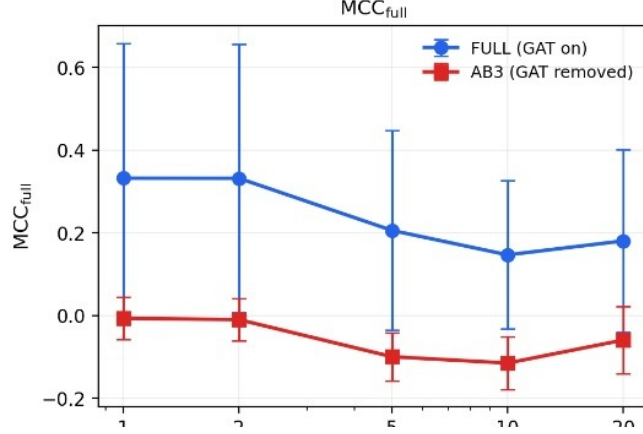


Figure 5.14: AB3 ablation: GAT disabled (preliminary, $n = 3$ seeds) — (a) MCC vs. coordinated Sybil identities n_{coord} .

The full system maintains an MCC of approximately 0.15–0.33 across all coordination levels, while AB3 (GAT removed) stays near zero or slightly negative throughout, confirming that individual per-beacon and per-vehicle checks alone cannot detect coordinated Sybil identities regardless of how many attackers coordinate.

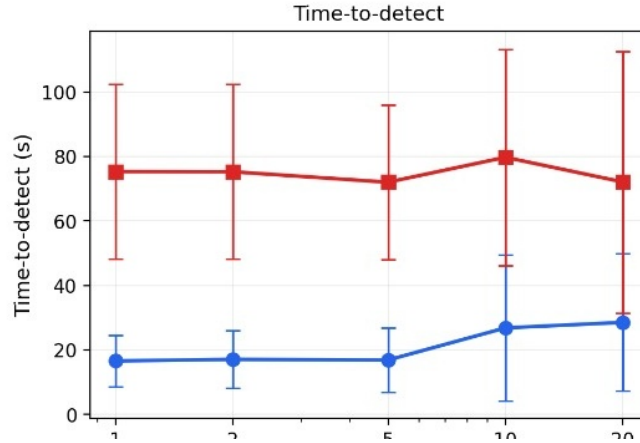


Figure 5.15: AB3 ablation: GAT disabled (preliminary, $n = 3$ seeds) — (b) TTD vs. coordinated Sybil identities n_{coord} .

Detection latency (TTD) for AB3 is consistently around 70–80 s, roughly 4–5 $\times$ slower than the full system’s 15–28 s, since without spatial relational analysis the system must wait for other signals to accumulate sufficient evidence.

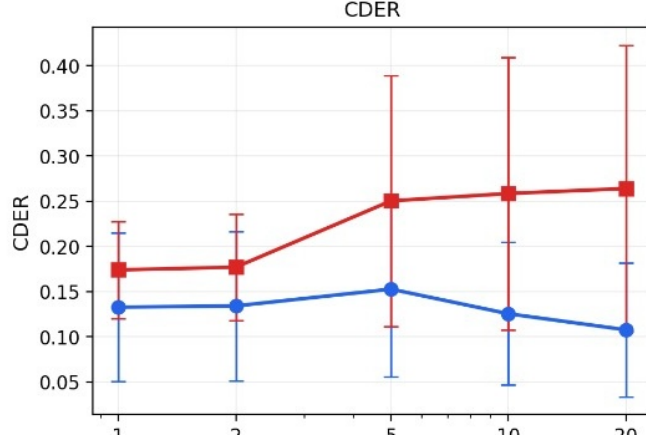


Figure 5.16: AB3 ablation: GAT disabled (preliminary, $n = 3$ seeds) — (c) CDER vs. coordinated Sybil identities n_{coord} .

Control-plane decision error (CDER) diverges further as n_{coord} increases, from a small gap at $n_{\text{coord}} = 1$ to nearly double by $n_{\text{coord}} = 20$, isolating the GAT’s specific contribution to detecting multi-vehicle spatial configurations that individually pass all non-relational checks. Given the high variance at this seed count, these results should be treated as directional rather than final; a larger seed sweep is planned to confirm the trend. This isolates the spatial relational detector’s contribution and directly answers **RQ4** (Section 1.6.1).

5.3.4 AB4 — LSTM-AE Disabled

Figures 5.17–5.20 present detection quality, detection latency, control-plane decision error, and trajectory prediction error as the stealth drift bound ϵ_{max} increases, comparing the full system against AB4 (LSTM temporal autoencoder disabled), under stealth trajectory poisoning ($\rho_a = 0.20$, attack TP-S1, 5 seeds).

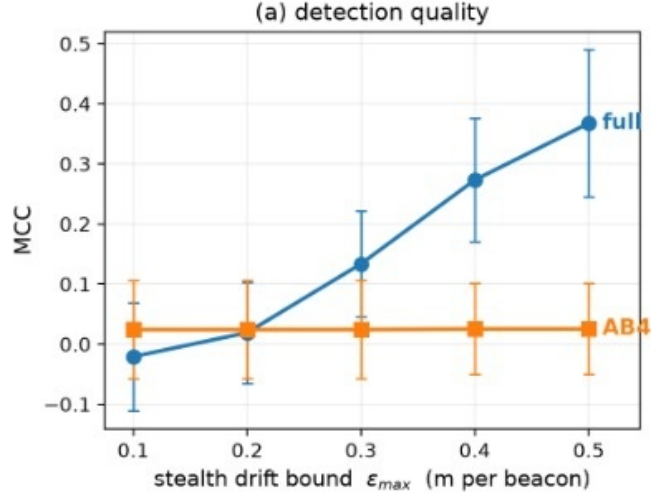


Figure 5.17: AB4 ablation: LSTM-AE removed — (a) MCC vs. stealth drift bound $\epsilon_{\max}$.

AB4 produces a flat MCC of approximately 0.02 regardless of drift magnitude, while the full system’s MCC rises from near-zero to 0.37 as $\epsilon_{\max}$ increases, directly demonstrating that without the LSTM-AE, gradual stealth drift within per-beacon kinematic bounds goes undetected irrespective of how large the cumulative drift becomes.

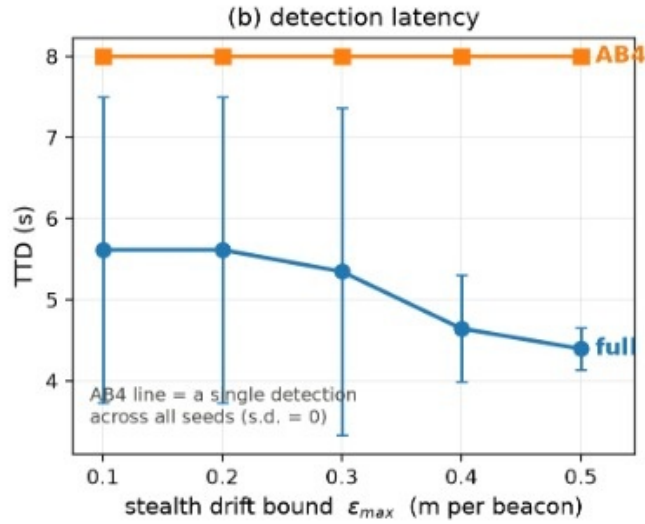


Figure 5.18: AB4 ablation: LSTM-AE removed — (b) TTD vs. stealth drift bound $\epsilon_{\max}$.

TTD for AB4 is fixed at the 8s window ceiling with zero variance across seeds, indicating the ablated system never raises a genuine alert within the observation window rather than detecting late.

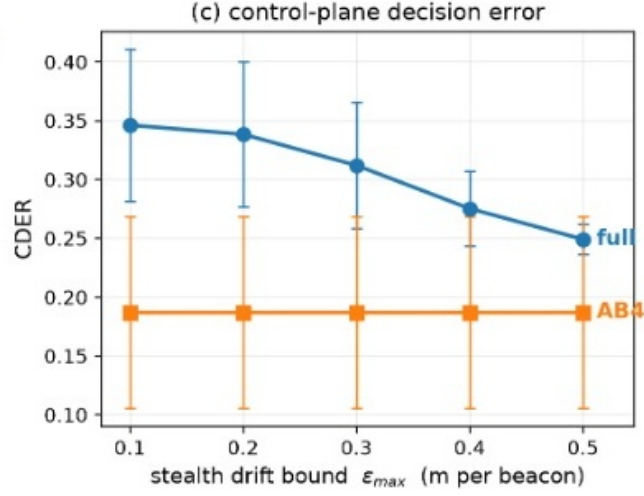


Figure 5.19: AB4 ablation: LSTM-AE removed — (c) CDER vs. stealth drift bound ϵ_{max} .

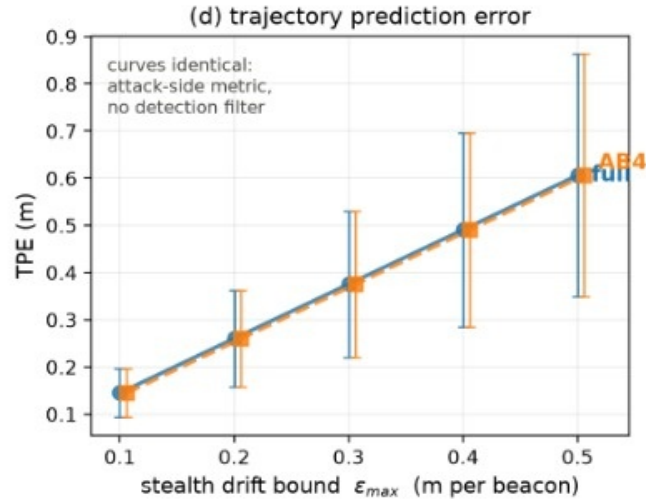


Figure 5.20: AB4 ablation: LSTM-AE removed — (d) TPE vs. stealth drift bound ϵ_{max} .

Trajectory prediction error (TPE) is identical between the two systems, since TPE is a pure attack-severity metric measured before any detection filter is applied. This isolates the LSTM-AE’s unique contribution to catching temporal drift that stays within kinematic bounds throughout the attack, and directly answers **RQ4** (Section 1.6.1).

5.3.5 AB5 — Full AI Layer Removed (Speed Regime)

Figures 5.21–5.26 present all six primary metrics as vehicle speed increases, comparing the full system (full AI layer active) against AB5 (lightweight mode only, GAT and LSTM-AE removed), under combined attack conditions (3 seeds).

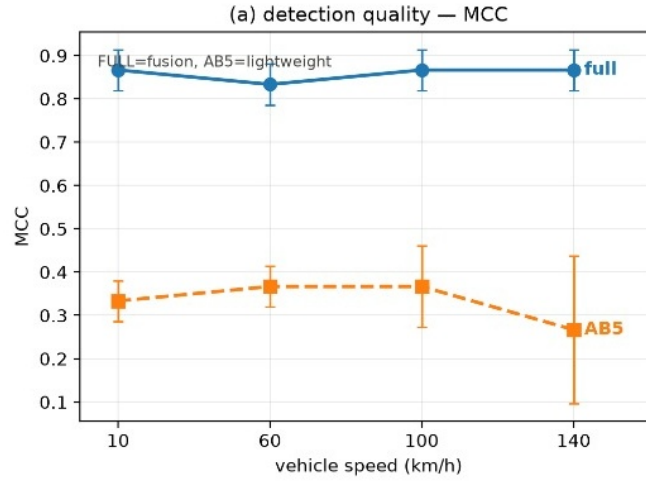


Figure 5.21: AB5 ablation: full AI layer removed — (a) MCC vs. vehicle speed.

The full system maintains MCC above 0.83 across all speeds, while AB5 stays between 0.26 and 0.36 throughout, confirming that the AI layer’s contribution is not merely additive but structurally necessary.

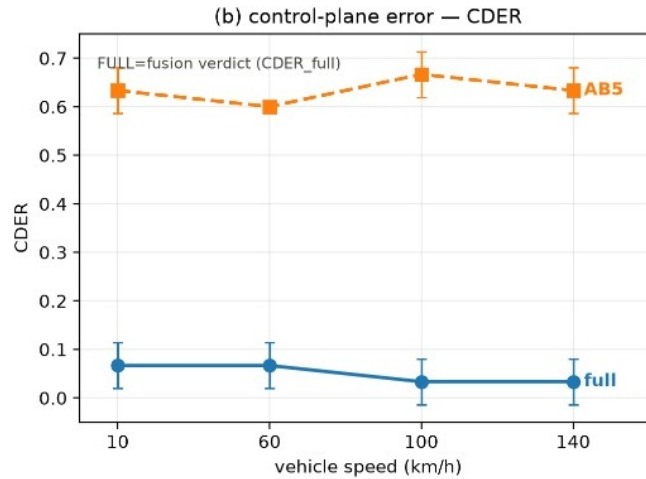


Figure 5.22: AB5 ablation: full AI layer removed — (b) CDER vs. vehicle speed.

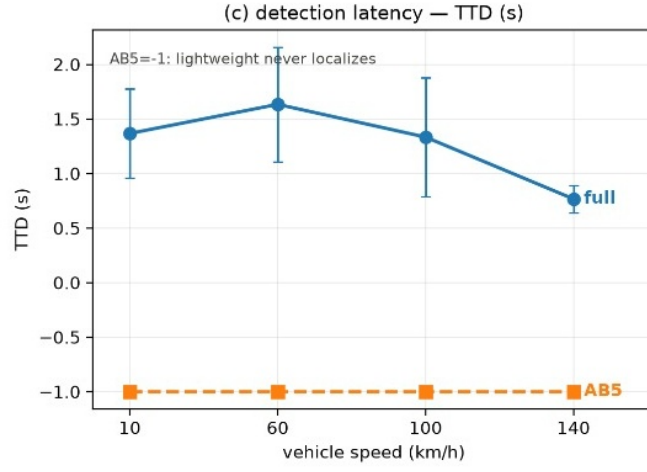


Figure 5.23: AB5 ablation: full AI layer removed — (c) TTD vs. vehicle speed.

TTD for AB5 is fixed at -1 across all speeds, indicating the lightweight-only system never localizes a genuine detection event within the observation window rather than detecting late; this directly validates the Detection Infeasible Zone prediction of Eq. 3.4, since AI-based sequential anomaly scoring is precisely the mechanism designed to recover detection power when per-beacon evidence alone is insufficient.

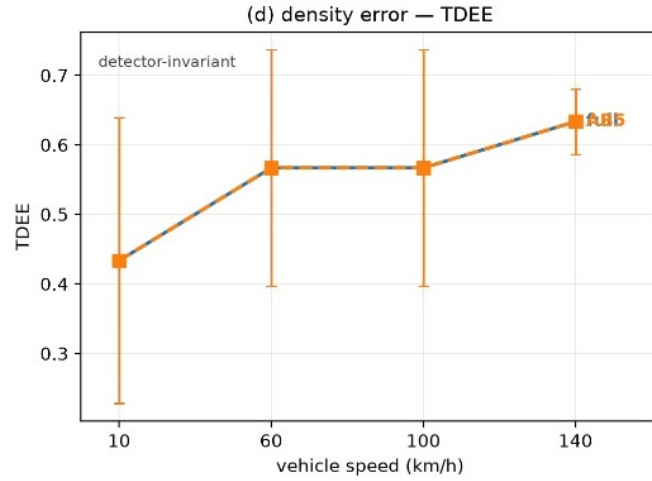


Figure 5.24: AB5 ablation: full AI layer removed — (d) TDEE vs. vehicle speed.

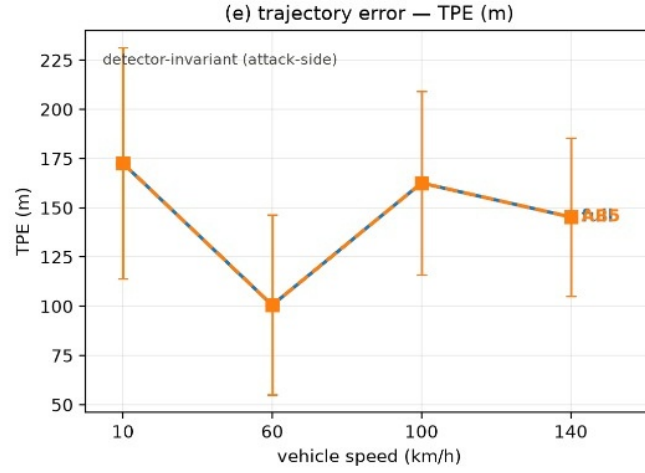


Figure 5.25: AB5 ablation: full AI layer removed — (e) TPE vs. vehicle speed.

TDEE and TPE are attack-severity metrics measured independent of the detector and are therefore near-identical between the two systems, confirming these are correctly attack-side rather than detector-side quantities.

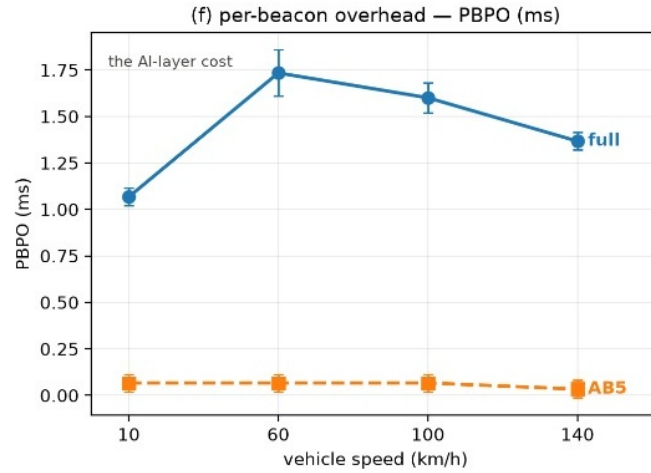


Figure 5.26: AB5 ablation: full AI layer removed — (f) PBPO vs. vehicle speed.

Per-beacon overhead (PBPO) shows the AI layer’s cost directly: the full system’s overhead (1.1–1.7 ms) sits well within the lightweight mode’s much smaller footprint (under 0.1 ms), quantifying the latency cost of enabling GAT and LSTM-AE detection. This isolates the combined contribution of the full AI layer and directly answers **RQ3** and **RQ4** (Section 1.6.1).

5.3.6 AB6 — TRS Threshold Signing Gate Removed

Figures 5.27–5.29 present poisoned aggregate rejection rate, control decision error, and cryptographic operation overhead as the compromised ring-member fraction f/n increases ($n = 4$, combined attack, $\rho_a = 0.40$, 60 km/h, 3 seeds), comparing the full TRS+FHE pipeline against AB6 (single-signer, no quorum).

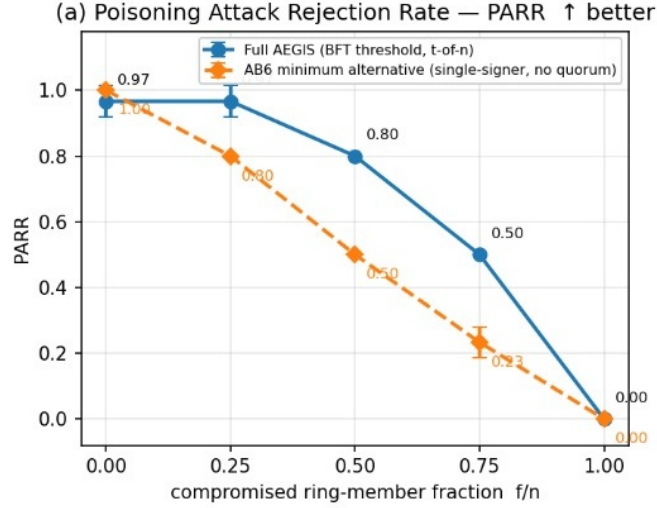


Figure 5.27: AB6 ablation: TRS threshold gate weakened to single-signer — (a) PARR vs. compromised ring-member fraction f/n .

The full threshold-signing gate rejects a consistently higher fraction of poisoned aggregates than the single-signer alternative at every compromise level (e.g. PARR of 0.80 vs. 0.50 at $f/n = 0.5$), confirming that requiring t -of- n collaborative signing structurally raises the bar against a growing compromised ring minority.

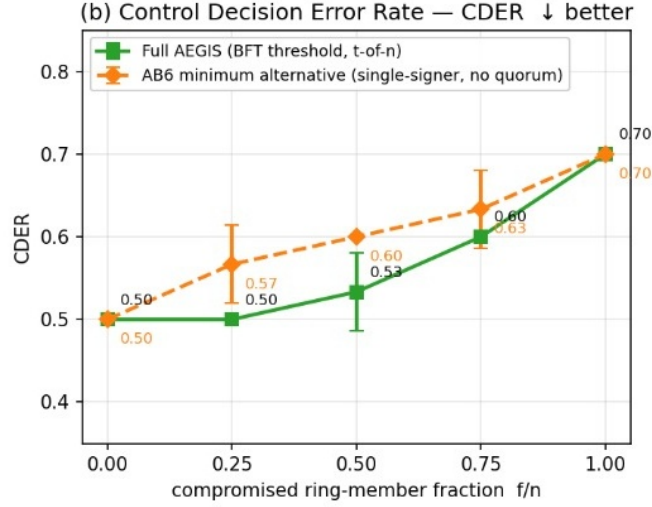


Figure 5.28: AB6 ablation: TRS threshold gate weakened to single-signer — (b) CDER vs. compromised ring-member fraction f/n .

Control decision error (CDER) tracks closely between the two until $f/n \geq 0.5$, after which the full system maintains a visible advantage.

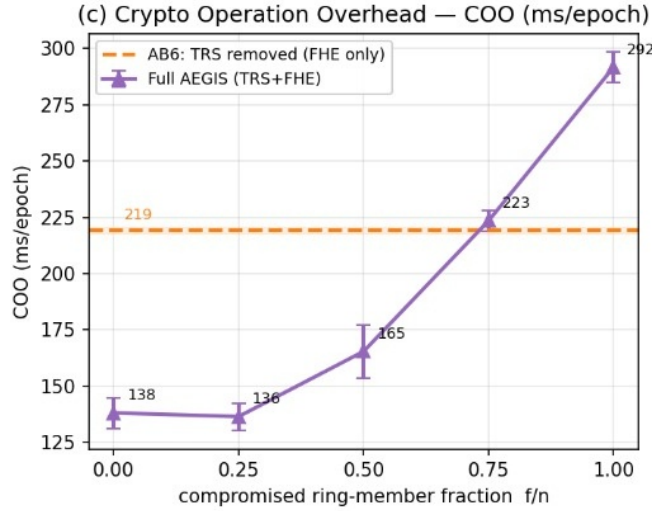


Figure 5.29: AB6 ablation: TRS threshold gate weakened to single-signer — (c) COO vs. compromised ring-member fraction f/n .

Cryptographic overhead (COO) increases with f/n for the full TRS+FHE pipeline (138 ms to 292 ms per epoch) since larger compromised fractions require more signing coordination overhead, while the FHE-only alternative (TRS removed) remains flat at approximately 219 ms, quantifying the additional cost of threshold signing directly. This isolates the threshold-signing gate’s contribution and directly answers **RQ5** (Section 1.6.1).

5.3.7 AB7 — FHE Pre-Coordination Encryption Removed

Figures 5.30–5.33 present per-epoch cryptographic cost, TRS signing latency, communication overhead ratio, and one-time distributed key generation setup latency as the RSU signing-ring size n increases ($n \in \{4, 6, 8, 10, 12\}$), comparing the full post-quantum PQ-FHE-TRS pipeline against a classical ECDSA-class baseline (Shamir-Schnorr-P256) at each ring size.

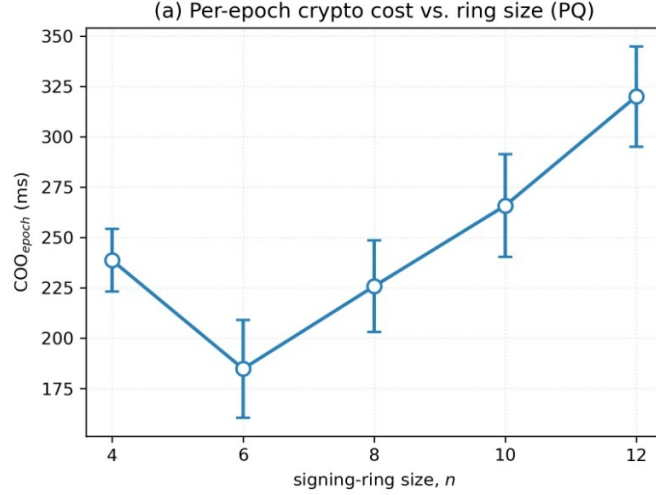


Figure 5.30: AB7 ablation: post-quantum vs. classical cryptographic overhead — (a) per-epoch crypto cost (COO_{epoch}) vs. signing-ring size n .

The per-epoch cryptographic cost COO_{epoch} for the post-quantum pipeline is non-monotonic across ring size, dipping from approximately 238 ms at $n = 4$ to a minimum of 185 ms at $n = 6$ before rising to 320 ms at $n = 12$, since a larger signing ring requires aggregating more Dilithium partial signatures per epoch (Eq. 3.55) even though FHE ciphertext encryption cost is fixed per RSU.

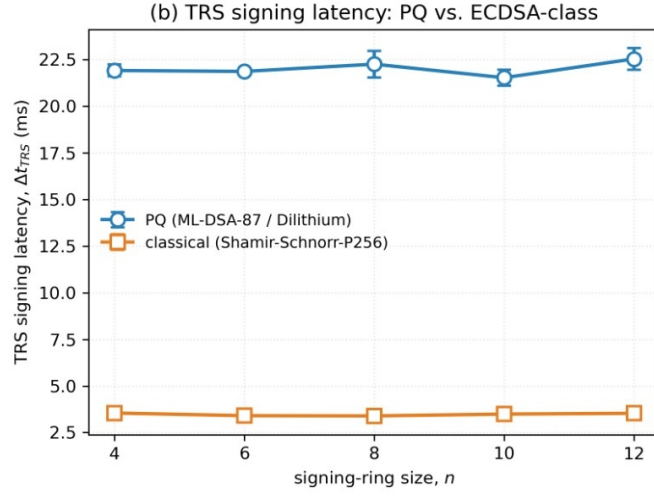


Figure 5.31: AB7 ablation: post-quantum vs. classical cryptographic overhead — (b) TRS signing latency (Δt_{TRS}), PQ vs. ECDSA-class, vs. signing-ring size n .

The per-RSU TRS signing latency itself is flat at approximately 22 ms for the post-quantum scheme regardless of n , versus a flat 3.5 ms for the classical baseline, a consistent $6.3\times$ overhead that confirms each RSU signs independently under Eq. 3.54 irrespective of ring membership, so the post-quantum signing premium is a fixed per-signer cost rather than a ring-size-dependent one.

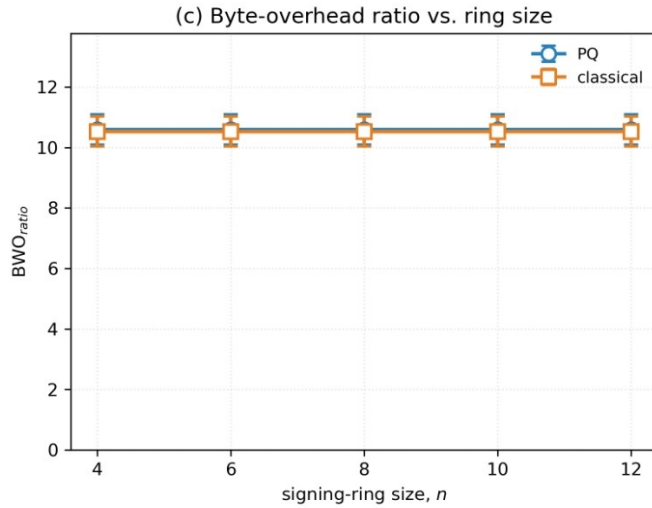


Figure 5.32: AB7 ablation: post-quantum vs. classical cryptographic overhead — (c) byte-overhead ratio ($\text{BWO}_{\text{ratio}}$) vs. signing-ring size n .

The byte-overhead ratio $\text{BWO}_{\text{ratio}}$ is nearly identical between the post-quantum and classical pipelines, both settling at approximately $10.5\times$ the plain-BSM baseline and flat across ring size, indicating that communication overhead is dominated

by the constant per-RSU FHE ciphertext expansion (Eq. 3.51) rather than by how many RSUs participate in the ring.

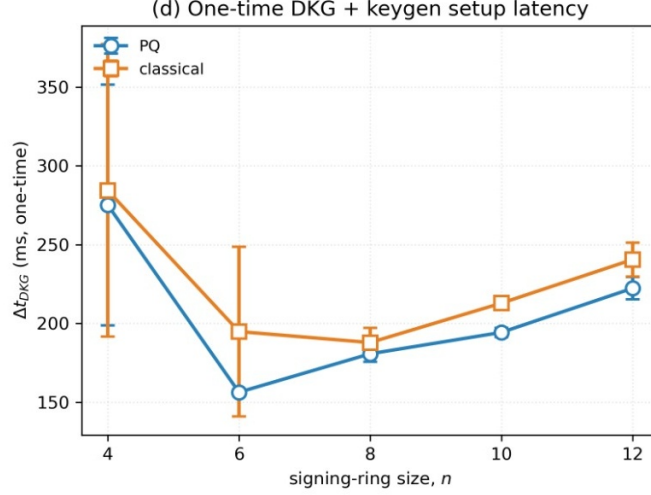


Figure 5.33: AB7 ablation: post-quantum vs. classical cryptographic overhead — (d) one-time DKG + keygen setup latency vs. signing-ring size n .

The one-time DKG and key generation setup cost (Eqs. 3.25, 3.26) follows the same dip-then-rise pattern as panel (a), with the post-quantum and classical curves tracking closely for $n \geq 6$ and both showing high variance at $n = 4$, reflecting one-time setup noise that averages out once the ring is large enough to amortise coordination overhead. Together, these results isolate FHE’s specific contribution to per-epoch computation and communication overhead independent of ring size scaling, and show that post-quantum TRS settles at a predictable, bounded latency premium over classical signing (approximately $6\times$ for signing, negligible for byte overhead) that does not itself grow with the BFT security parameter f , supporting deployment feasibility at larger ring sizes. This isolates the post-quantum FHE pre-coordination overhead and directly answers **RQ5** (Section 1.6.1).

5.3.8 AB9 — Multi-Controller Architecture Removed

Figures 5.34–5.35 present control-plane decision error and overall detection quality as controller compromise onset time $t_{\text{comp}}/T_{\text{sim}}$ varies, comparing the full multi-controller architecture against AB9 (single fixed controller, no revocation pathway).

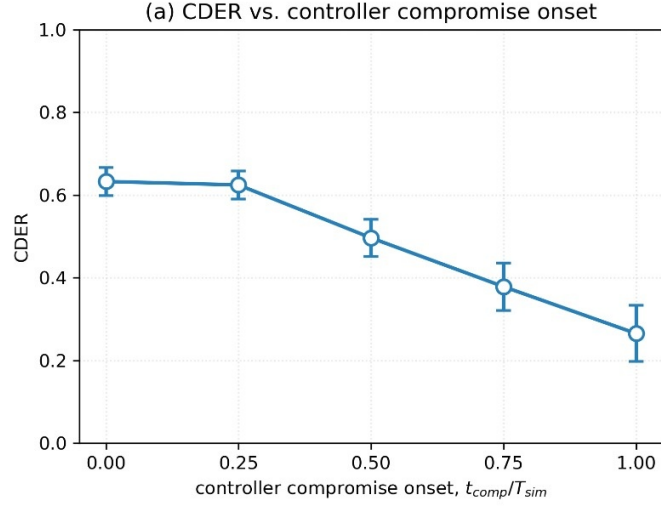


Figure 5.34: AB9 ablation: multi-controller architecture removed — (a) CDER vs. controller compromise onset $t_{\text{comp}}/T_{\text{sim}}$.

CDER decreases as compromise onset moves later in the simulation (from 0.63 at onset= 0 to 0.27 at onset= 1.0), reflecting that a controller compromised earlier has more time to corrupt control decisions before the run ends — exactly the behaviour the multi-controller revocation pathway is designed to bound.

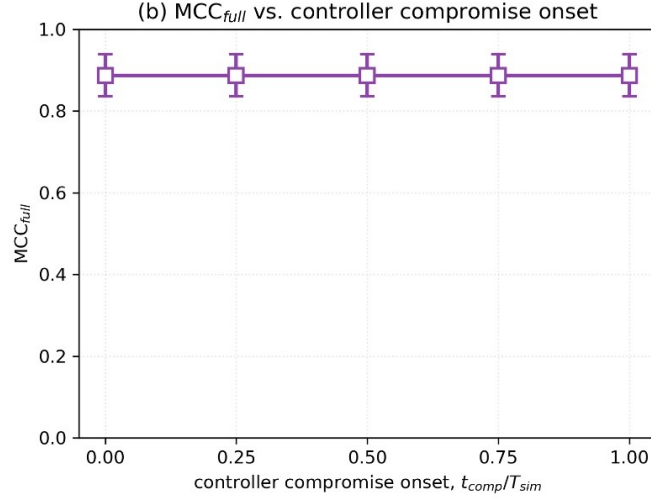


Figure 5.35: AB9 ablation: multi-controller architecture removed — (b) MCC vs. controller compromise onset $t_{\text{comp}}/T_{\text{sim}}$.

Detection quality (MCC) remains flat at approximately 0.88 regardless of compromise onset time, confirming that detection is independent of when the controller is compromised; the multi-controller architecture’s contribution is specifically to mitigation (bounding control-plane damage) rather than detection, consistent with its design purpose of providing a controller revocation and reassignment pathway

rather than an additional detection signal. This isolates the multi-controller architecture’s contribution and directly answers **RQ6** (Section 1.6.1).

5.3.9 AB10 — Blockchain Trust Layer Removed

Figures 5.36–5.40 present control decision error, false revocation and demotion rates, on-chain confirmation latency, and detection quality as attack penetration ρ_a increases, comparing the full blockchain-enabled trust layer against AB10 (SC-Trust/SC-Revoke disabled, centralized controller-side revocation).

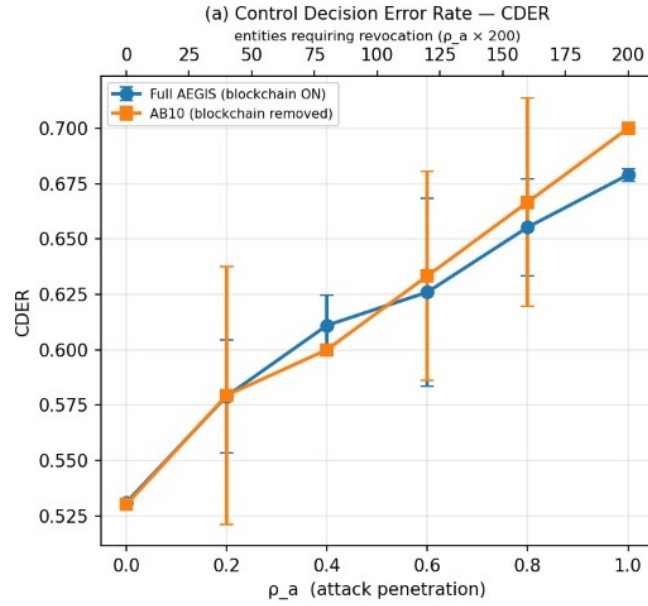


Figure 5.36: AB10 ablation: blockchain trust layer removed — (a) CDER vs. attack penetration ρ_a .

CDER is nearly identical between the two systems at low-to-moderate penetration but AB10 shows a visibly higher error at $\rho_a \geq 0.6$, consistent with the loss of BFT-conditioned revocation.

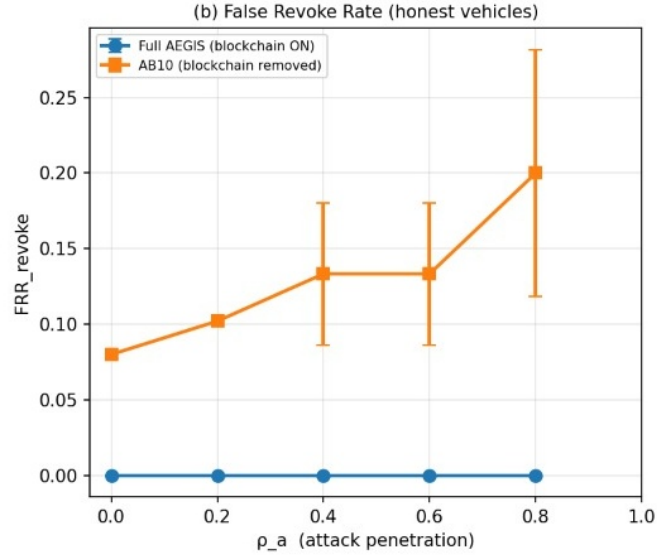


Figure 5.37: AB10 ablation: blockchain trust layer removed — (b) $\text{FRR}_{\text{revoke}}$ vs. attack penetration ρ_a .

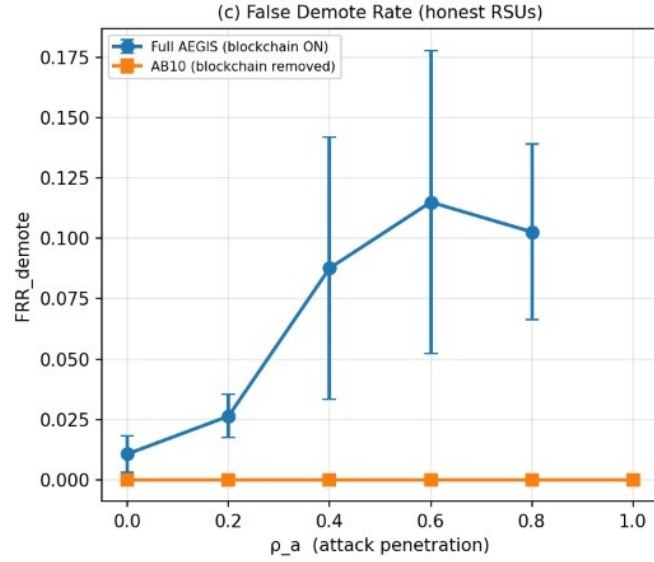


Figure 5.38: AB10 ablation: blockchain trust layer removed — (c) $\text{FRR}_{\text{demote}}$ vs. attack penetration ρ_a .

The false revocation and demotion rates reveal the clearest safety difference: AB10 shows a rising false revoke rate on honest vehicles (up to 0.20 at $\rho_a = 0.8$) since centralized controller-side revocation lacks the $2f + 1$ peer-endorsement

safeguard, while the full system shows zero false revocations throughout. The full system does show a non-zero false demotion rate on honest RSUs (up to 0.115), which is the expected and less severe outcome of the three-state trust lifecycle design — a demoted RSU can recover, whereas AB10 has no equivalent demotion mechanism at all (flat zero, since the mechanism doesn't exist).

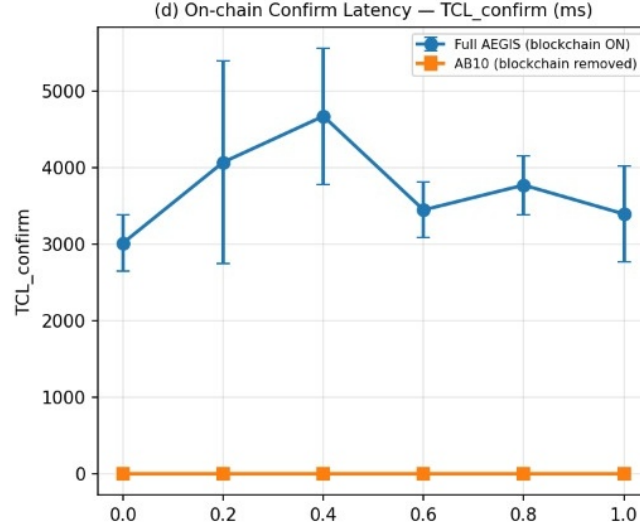


Figure 5.39: AB10 ablation: blockchain trust layer removed — (d) TCL_{confirm} vs. attack penetration ρ_a .

On-chain confirmation latency is naturally zero for AB10 by definition (no blockchain transactions), establishing the latency cost of the full system's on-chain layer at roughly 3–4.7 s per confirmation.

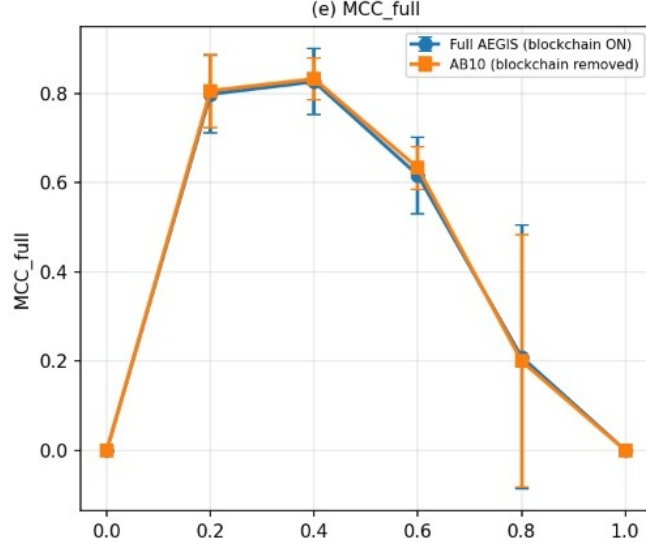


Figure 5.40: AB10 ablation: blockchain trust layer removed — (e) MCC vs. attack penetration ρ_a .

Detection quality (MCC) is nearly identical between the two systems, confirming that the blockchain layer’s contribution is to trust management and revocation safety rather than detection itself. This isolates the blockchain trust layer’s contribution and directly answers **RQ6** (Section 1.6.1).

5.3.10 AB11 — LKH Replaced with Unicast Key Distribution

Figures 5.41–5.42 present the empirical rekey message count and resulting channel occupancy as vehicle zone density increases, comparing the Logical Key Hierarchy (LKH) against AB11 (direct per-vehicle unicast rekeying).

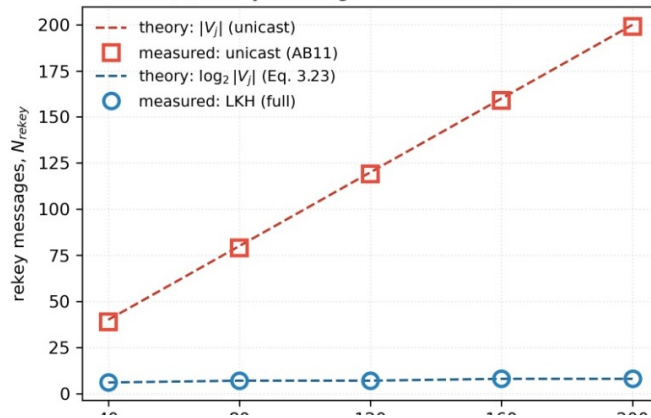


Figure 5.41: AB11 ablation: LKH replaced with unicast key distribution — (a) rekey message count vs. vehicles registered per RSU zone $|V_j|$.

The measured LKH rekey cost tracks the theoretical $\log_2 |\mathcal{V}_j|$ curve almost exactly (Eq. 3.24), while the unicast alternative closely follows the theoretical $O(|\mathcal{V}_j|)$ growth, empirically confirming the closed-form complexity claim. At $|\mathcal{V}_j| = 200$, LKH requires only 8 rekey messages compared to 200 for unicast, a $25\times$ reduction.

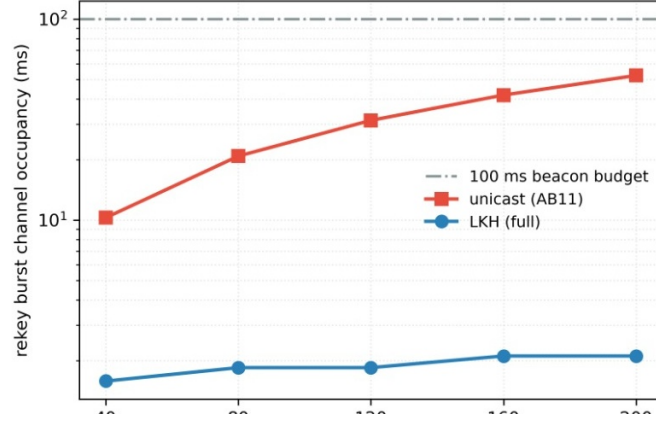


Figure 5.42: AB11 ablation: LKH replaced with unicast key distribution — (b) rekey burst channel occupancy vs. vehicles registered per RSU zone $|\mathcal{V}_j|$.

Panel (b) translates this into channel occupancy against the 100 ms beacon budget: unicast rekeying already consumes over half the beacon budget (55 ms) at $|\mathcal{V}_j| = 200$, whereas LKH remains under 3 ms throughout, confirming that LKH is the only scalable option for dense urban RSU zones without threatening the real-time beacon processing constraint. This isolates the LKH key-management scheme’s contribution and directly answers **RQ5** (Section 1.6.1).

Chapter 6

Timeline and Resource Required

6.1 Timeline

PROJECT ACTIVITY	PROJECT WEEK																																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		
Project Selection Submission	■																																	
Conduct comprehensive literature review		■	■	■																														
Submission of project proposal report for supervisor's comments				■	■	■																												
Submission of project proposal report					■	■																												
Project Proposal Presentation and VIVA						■	■																											
Attack simulation from NS-3							■	■	■	■	■																							
Design Graph Attention Network (GAT) model									■	■	■	■	■																					
Design Temporal Autoencoder for anomaly detection										■	■	■	■	■	■	■																		
Project Progress presentations and VIVA																■	■																	
Submission of project progress report																	■	■																
Integration of GAT with Temporal Autoencoder																		■	■	■	■	■												
Blockchain framework design																				■	■	■	■	■										
System integration (SDVN + AI model + Blockchain)																					■	■	■	■	■									
Final system testing and validation																									■	■	■	■	■					
Submission of draft of the final report for supervisor's comments																														■				
Submission of Final report																																■		
Final presentation and VIVA																																	■	■
Final Demonstration																																	■	■

Figure 6.1: Project Timeline

6.2 Resource Required

This project mainly requires software tools and computing resources to design, simulate, and evaluate the proposed mitigation framework. No specialized hardware is required since the evaluation is conducted through simulations.

6.2.1 Software Resources

The following software tools are needed for the project:

- Network Simulator 3 (NS-3) for simulating the Software Defined Vehicular Network environment and attack scenarios.
- Simulation of Urban Mobility (SUMO) for generating realistic vehicular mobility and traffic patterns.
- Python programming environment for implementing machine learning models, including the Graph Attention Network and temporal autoencoder.
- Machine learning libraries liked PyTorch or TensorFlow for training and evaluating the learning models.
- Blockchain frameworks or libraries (e.g., Hyperledger Fabric or Web3 libraries) for implementing the blockchain based trust mechanism.
- LaTeX for preparing project documentation and reports.

6.2.2 Hardware Resources

The project requires a standard personal computer or laptop.

6.2.3 Estimated Cost

Since all required tools are open-source or freely available for academic use, the estimated financial cost of the project is minimal. The primary resources are time, computational effort, and access to a personal computer. Therefore, no additional budget allocation is required for this project.

Chapter 7

Conclusion

This research addresses mobility pattern and trajectory poisoning attacks in software-defined vehicular networks (SDVNs), a security gap in modern intelligent transport systems. SDVNs increasingly rely on a centralized controller to manage a dynamic environment with large numbers of vehicles, which exposes them to attacks that gradually corrupt mobility and trajectory information. Such attacks are dangerous precisely because they slip past the prevailing security model, which verifies who sent a message but not whether its content is true, and can lead the controller to learn incorrect mobility patterns and issue inefficient or unsafe control decisions.

The literature reviewed in this thesis shows a consistent gap: existing SDVN security solutions rarely address both data-plane and control-plane attacks together. Most focus on a single defense strategy, such as rule-based mobility verification, generic intrusion detection, or cryptographic authentication, and none of the reviewed work accounts for an insider attack originating from the controller itself.

To address these shortcomings, this work proposes a mitigation framework built on three technologies:

1. Use of blockchain for providing distributed trust, integrity of data, and traceability, which will minimize the need for a centralized controller.
2. Graph Attention Networks (GAT) for representing spatial relationships between vehicles and roadside units (RSUs).
3. Temporal Autoencoders learn normal mobility and trajectory patterns over time and detect subtle deviations that indicate poisoning attacks.

The framework has been assessed via NS-3/SUMO simulation experiments across ten completed ablation studies (AB1–AB7 and AB9–AB11, Section 5.3) and a preliminary per-attack-variant comparison against three external baselines (Experiment 5, Section 5.1), simulating attack cases across both data and control planes. A related engineering finding from this evaluation round, correcting cross-run contamination of shared per-RSU detector state, reduces the honest false positive rate from 0.2272 to 0.0432 at the 100s evaluation horizon (Section 5.2) and is applied throughout the reported results. The completed ablation

results consistently confirm the necessity of every framework component. Removing the rule-based signature layer (AB1) drops MCC from 0.78 to 0.43 at moderate penetration; removing HMAC integrity verification (AB2) allows MitM detection quality to collapse at high penetration; removing the GAT spatial detector (AB3) and the LSTM temporal autoencoder (AB4) each independently reduce coordinated-attack and stealth-drift detection toward zero, confirming that spatial and temporal analysis catch distinct attack classes that per-beacon rule checks alone cannot; and removing the full AI layer (AB5) reproduces the predicted Detection Infeasible Zone at high speed, where lightweight-only detection never localizes a genuine alert within the observation window. On the mitigation side, weakening the TRS threshold-signing gate to a single signer (AB6) roughly halves the poisoned-aggregate rejection rate under ring compromise, and removing FHE pre-coordination encryption (AB7) isolates a bounded, ring-size-independent post-quantum signing premium (approximately $6\times$ classical latency) that does not itself grow with the BFT security parameter. Removing the multi-controller revocation pathway (AB9) leaves control-decision error uncorrected once a controller is compromised, and removing the blockchain trust layer (AB10) reintroduces false revocations against honest vehicles that the BFT-conditioned SC-Revoke design eliminates entirely. Finally, replacing the Logical Key Hierarchy with unicast rekeying (AB11) empirically confirms the predicted logarithmic-versus-linear scaling gap, with LKH requiring $25\times$ fewer rekey messages at dense RSU zone occupancy. Complementing these component-level results, the preliminary per-attack-variant comparison shows MPTD-PQS matching or exceeding all three external baselines across every evaluated attack type, with the sharpest advantage on the two control-plane variants that the data-plane-only baselines have no mechanism to detect at all. Taken together, these completed results substantiate the research project’s central claim: integrating distributed trust, spatial learning, and temporal anomaly identification measurably improves the security of learning-based SDVNs. The RSU three-state trust lifecycle ablation (AB8) and the rural-scenario RAPTOR generalisation experiments remain outstanding and are positioned to extend rather than establish this conclusion.

A critical and previously uncharacterized aspect of this threat class is the role of vehicle kinematics as an attack amplification vector. As formally derived in this work, increasing vehicle speed simultaneously reduces RSU contact duration and shrinks the observable beacon sampling window. In high-speed highway regimes, the observable beacon count N_b falls below the minimum detection threshold $N_{\min}$ (Eq. 3.4), creating a Detection Infeasible Zone where sequential anomaly detection becomes unreliable, while the attacker’s falsification budget simultaneously peaks. This mobility-induced dual unfavorability means that existing threshold-based defenses are not merely insufficient but are systematically miscalibrated for real SDVN operating conditions, particularly under high-speed scenarios where Sybil fraction f_{sybil} also approaches 1 at low traffic density, allowing a compromised RSU

to dominate the controller’s entire view of local traffic. The AB5 ablation results above provide direct empirical confirmation of this predicted Detection Infeasible Zone.

To address these challenges, we propose MPTD-PQS, a two-mode detection and mitigation system for securing SDVNs with six significant contributions. Firstly, a multi-source, dual-plane threat model incorporates attacks against malicious vehicles, compromised RSUs, MitM attackers, and malicious controllers concurrently in both data and control planes. Secondly, a formal formulation of mobility-induced threat amplification quantifies closed-form bounds on how the speed of a vehicle, handover frequency, and congestion jointly reduce the availability of detection information while expanding the scope of adversary actions. Thirdly, an attack signature characterization identifies nine formally defined detection signatures (TP-S1 to MP-S4), which create a composite anomaly score based on principled reasoning. Fourthly, a dual-mode detection approach combines simple rule-based RSU-side detection with more complex full-mode detection using GAT spatial and LSTM temporal autoencoder anomaly scoring, and the fused detection scores form a decision variable. Fifthly, a post-quantum TRS and FHE cryptographically mitigated system structurally ensures that a minority of compromised RSUs cannot reach the controller regardless of AI detection efficacy. Sixthly, a permissioned blockchain with smart contracts (SC-Trust and SC-Revoke) encodes a full-state machine with RSU-direct Byzantine fault-tolerant consensus, thereby breaking the trust bottleneck that all reviewed defenses require.

References

- [1] OpenFHE, “OpenFHE: Open-source fully homomorphic encryption library.” <https://github.com/openfheorg/openfhe-development>, 2022.
- [2] H. Hartenstein and L. Laberteaux, “A tutorial survey on vehicular ad hoc networks,” *IEEE Communications Magazine*, vol. 46, no. 6, pp. 164–171, 2008.
- [3] I. Wahid, S. Tanvir, A. Hameed, and M. Ahmed, “Software-defined networks and named data networks in vehicular ad hoc network routing: Comparative study and future directions,” *Security and Communication Networks*, vol. 2022, pp. 1–14, 06 2022.
- [4] D. Cabarkapa and D. Rancic, “Towards software-defined vehicular networks: Recent achievements,” 06 2019.
- [5] M. Arif, G. Wang, O. Geman, V. E. Balas, P. Tao, A. Brezulianu, and J. Chen, “SDN-based VANETs, security attacks, applications, and challenges,” *Applied Sciences*, vol. 10, no. 9, p. 3217, 2020.
- [6] B. Ravi, B. Varghese, I. Murturi, P. K. Donta, S. Dustdar, C. K. Dehury, and S. N. Srirama, “Stochastic modeling for intelligent software-defined vehicular networks: A survey,” *Computers*, vol. 12, no. 8, p. 162, 2023.
- [7] M. Arif, G. Wang, M. Bhuiyan, T. Wang, and J. Chen, “A survey on security attacks in vanets: Communication, applications and challenges,” *Vehicular Communications*, vol. 19, p. 100179, 09 2019.
- [8] F. A. Ghaleb, M. A. Razzaque, and A. Zainal, “Mobility pattern based misbehavior detection in vehicular adhoc networks to enhance safety,” in *2014 International Conference on Connected Vehicles and Expo (ICCVE)*, pp. 894–901, IEEE, 2014.
- [9] G. Raja, S. Anbalagan, G. Vijayaraghavan, P. Dhanasekaran, Y. D. Al-Otaibi, and A. K. Bashir, “Energy-efficient end-to-end security for software defined vehicular networks,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 8, pp. 5730–5737, 2021.

- [10] S. Hong, L. Xu, H. Wang, and G. Gu, "Poisoning network visibility in software-defined networks: New attacks and countermeasures," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, (San Diego, CA, USA), pp. 1–15, 2015.
- [11] M. A. Faleh, M. Yousif, M. A. Al-Shareeda, M. A. Almaiah, and M. Obaidat, "CSAS-V: Certificateless schnorr aggregate signature for VANETs," *International Journal of Innovative Research and Scientific Studies*, vol. 8, no. 2, pp. 4000–4011, 2025.
- [12] W. Ahmed, W. Di, and D. Mukathe, "A blockchain-enabled incentive trust management with threshold ring signature scheme for traffic event validation in VANETs," *Sensors*, vol. 22, no. 17, p. 6715, 2022.
- [13] A. Al Mamun, K. Yates, A. Rakotondrafara, M. Chowdhury, R. Cartor, and S. Gao, "Experimental evaluation of post-quantum homomorphic encryption for privacy-preserving I2I communication in ITS," *arXiv preprint arXiv:2508.02461*, 2025.
- [14] N. u. Sehar, O. Khalid, I. A. Khan, F. Rehman, M. A. B. Fayyaz, A. R. Ansari, and R. Nawaz, "Blockchain enabled data security in vehicular networks," *Scientific Reports*, vol. 13, p. 4412, 2023.
- [15] Y. Yahiatene and A. Rachedi, "Towards a blockchain and software-defined vehicular networks approaches to secure vehicular social network," in *Proc. IEEE Conf. Standards for Communications and Networking (CSCN)*, pp. 1–7, 2018.
- [16] J. Bhatia, Y. Modi, S. Tanwar, and M. Bhavsar, "Software defined vehicular networks: A comprehensive review," *International Journal of Communication Systems*, vol. 32, no. 12, p. e4005, 2019.
- [17] Z. He, J. Cao, and X. Liu, "SDVN: Enabling rapid network innovation for heterogeneous vehicular communication," *IEEE Network*, vol. 30, no. 4, pp. 10–15, 2016.
- [18] K. Yamamoto, S. Kuwada, and Y. Nakamura, "Modeling, prediction, and anomaly detection of manned-vehicle behavior in open field based on velocity vector and variance tensor fields," *Advanced Robotics*, vol. 34, no. 6, pp. 343–357, 2020.
- [19] S. Ercan, M. Ayaida, and N. Messai, "Misbehavior detection for position falsification attacks in VANETs using machine learning," *IEEE Access*, vol. 10, pp. 1893–1904, 2022.
- [20] R. Frenken, S. G. Bhatti, H. Zhang, and Q. Ahmed, "KD-GAT: Combining knowledge distillation and graph attention transformer for a controller area

- network intrusion detection system,” in *Proc. 28th IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2025. arXiv:2507.19686.
- [21] R. Frenken, S. G. Bhatti, H. Zhang, and Q. Ahmed, “Multi-stage knowledge-distilled VGAE and GAT for robust controller-area-network intrusion detection,” *arXiv preprint arXiv:2508.04845*, 2025.
 - [22] P. Sharma and H. Liu, “A machine-learning-based data-centric misbehavior detection model for Internet of Vehicles,” *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4991–4999, 2021.
 - [23] H. Zhong, S. Zhang, J. Cui, L. Wei, and L. Liu, “Broadcast encryption scheme for V2I communication in VANETs,” *IEEE Transactions on Vehicular Technology*, vol. 71, no. 3, pp. 2749–2760, 2022.
 - [24] IEEE, “IEEE standard for information technology—telecommunications and information exchange between systems—local and metropolitan area networks—specific requirements part 11: Wireless lan medium access control (mac) and physical layer (phy) specifications amendment 6: Wireless access in vehicular environments.” IEEE Std 802.11p-2010, 2010.
 - [25] Federal Communications Commission, “Amendment of the commission’s rules regarding dedicated short-range communication services in the 5.850–5.925 GHz band (5.9 GHz band),” Report and Order FCC 03-324, WT Docket No. 01-90, Federal Communications Commission, 2004.
 - [26] J. B. Kenney, “Dedicated short-range communications (DSRC) standards in the United States,” *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.
 - [27] D. Jiang and L. Delgrossi, “IEEE 802.11p: Towards an international standard for wireless access in vehicular environments,” in *Proc. IEEE Vehicular Technology Conference (VTC Spring)*, pp. 2036–2040, 2008.
 - [28] COST Action 231, “Digital mobile radio towards future generation systems, final report,” Tech. Rep. EUR 18957, European Communities, 1999.
 - [29] SAE International, “Dedicated short range communications (DSRC) message set dictionary,” Tech. Rep. J2735_201601, SAE International, 2016.
 - [30] C. Sommer, R. German, and F. Dressler, “Bidirectionally coupled network and road traffic simulation for improved IVC analysis,” *IEEE Transactions on Mobile Computing*, vol. 10, no. 1, pp. 3–15, 2011.